

Cómo Conectar Django Backend con Templates: Guía Completa Paso a Paso

Esta guía exhaustiva te llevará de la mano a través del proceso completo de conexión entre tu backend de Django y plantillas HTML, permitiéndote crear aplicaciones web dinámicas y profesionales con facilidad y confianza.

Creado por **Ivan Malaver Fierro** | Suite Gpt

Bienvenido a Esta Aventura de Aprendizaje

¿Para quién es esta guía?

Esta guía está diseñada específicamente para desarrolladores web y estudiantes con conocimientos básicos de Python y Django que desean dominar la conexión entre el backend y el frontend. No necesitas ser un experto, pero sí debes tener familiaridad con conceptos básicos de programación.

¿Qué aprenderás?

Al finalizar esta guía, serás capaz de configurar completamente el sistema de templates de Django, crear vistas dinámicas, organizar tu código de manera profesional y construir interfaces web que respondan a datos del backend de forma elegante y eficiente.

¿Qué es una Plantilla en Django?

Una plantilla (template) en Django es un archivo HTML especial que puede contener variables y etiquetas especiales para mostrar datos dinámicos provenientes del backend. Piensa en ella como un molde o esqueleto de página web que se llena con información específica cada vez que alguien visita tu sitio.

Esta separación entre lógica y presentación sigue el patrón MVT (Modelo-Vista-Template) que Django implementa para mantener un código limpio, organizado y fácil de mantener. Es como tener diferentes personas especializadas en diferentes tareas: una maneja los datos, otra procesa las solicitudes y otra se encarga del diseño visual.

`{{ variable }}`

Sintaxis para mostrar datos dinámicos del backend en tu HTML

`{% etiqueta %}`

Sintaxis para lógica de programación como bucles y condicionales

`{{ variable|filtro }}`

Sintaxis para transformar datos antes de mostrarlos

El Patrón MVT: El Corazón de Django



Django utiliza el patrón arquitectónico MVT (Modelo-Vista-Template), una variación del popular patrón MVC (Modelo-Vista-Controlador). Este patrón separa las responsabilidades de tu aplicación en tres componentes distintos, facilitando el desarrollo, mantenimiento y escalabilidad.

01

Modelo (Model)

Define la estructura de tus datos y maneja la interacción con la base de datos

02

Vista (View)

Contiene la lógica de negocio y procesa las solicitudes del usuario

03

Template (Template)

Maneja la presentación visual y la interfaz de usuario

Ventajas de Usar Templates en Django



Separación de Responsabilidades

Los desarrolladores pueden trabajar en el backend mientras los diseñadores se enfocan en el frontend, sin pisarse los pies. Esto acelera el desarrollo y reduce errores.



Reutilización de Código

Crea templates base que pueden ser extendidos por múltiples páginas, evitando duplicación de código HTML y manteniendo consistencia visual en toda tu aplicación.



Seguridad Integrada

Django automáticamente escapa contenido peligroso en tus templates, protegiéndote contra ataques XSS (Cross-Site Scripting) sin esfuerzo adicional de tu parte.



Rendimiento Optimizado

El sistema de templates de Django compila y cachea las plantillas, asegurando que tu aplicación responda rápidamente incluso con alto tráfico de usuarios.



Preparando Tu Entorno de Desarrollo

Antes de comenzar a trabajar con templates, asegúrate de tener tu entorno de desarrollo correctamente configurado. Esto te ahorrará dolores de cabeza más adelante y te permitirá concentrarte en aprender sin interrupciones técnicas.

Python Instalado

Verifica que tienes Python 3.8 o superior instalado ejecutando `python --version` en tu terminal. Django requiere una versión moderna de Python para funcionar correctamente.

Django Framework

Instala Django usando pip: `pip install django`. Esto instalará la última versión estable del framework junto con todas sus dependencias necesarias.

Editor de Código

Usa un editor como VS Code, PyCharm o Sublime Text con soporte para Python y Django. Las extensiones adecuadas harán tu vida mucho más fácil con autocompletado y detección de errores.

Estructura del Proyecto Django: Visión General

Cuando creas un proyecto Django con `django-admin startproject myproject`, se genera automáticamente una estructura de directorios específica. Entender esta estructura es fundamental para saber dónde colocar tus templates y cómo organizar tu código.

Cada carpeta y archivo tiene un propósito específico en el ecosistema Django. El archivo **settings.py** contiene toda la configuración, **urls.py** define las rutas de tu aplicación, y **wsgi.py** maneja el despliegue en servidores de producción.

```
myproject/  
├── myproject/  
│   ├── __init__.py  
│   ├── settings.py  
│   ├── urls.py  
│   ├── asgi.py  
│   └── wsgi.py  
├── manage.py  
└── db.sqlite3
```

Creando Tu Primera Aplicación Django

En Django, un proyecto puede contener múltiples aplicaciones. Cada aplicación es un módulo independiente que realiza una función específica. Por ejemplo, podrías tener una aplicación para el blog, otra para usuarios, y otra para el carrito de compras.

Ejecuta el Comando de Creación

Abre tu terminal en la carpeta del proyecto y ejecuta: `python manage.py startapp myapp`. Este comando genera automáticamente toda la estructura básica de tu aplicación.

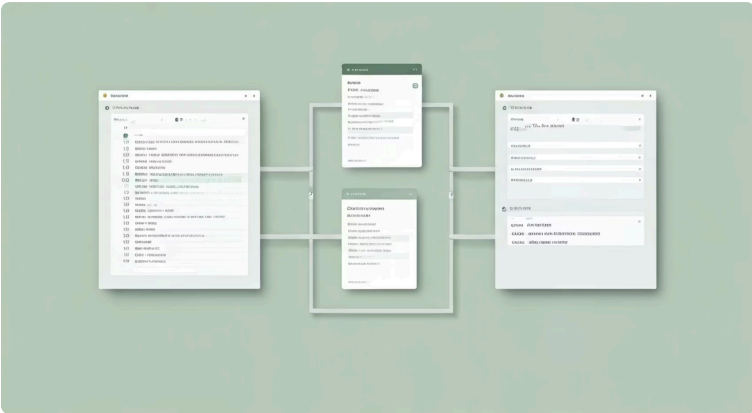
Registra la Aplicación

Abre `settings.py` y añade 'myapp' a la lista `INSTALLED_APPS`. Este paso es crucial porque Django necesita saber qué aplicaciones existen en tu proyecto.

Explora la Estructura

Tu nueva aplicación contendrá archivos como `models.py`, `views.py`, `tests.py` y `admin.py`. Cada uno tiene un rol específico en el funcionamiento de tu aplicación.

Estructura de Carpetas para Templates



Organizar correctamente tus templates es esencial para mantener un proyecto escalable y fácil de mantener. Existen dos enfoques principales: crear una carpeta global de templates en la raíz del proyecto, o mantener templates específicos dentro de cada aplicación.

Enfoque Global

```
myproject/  
├── myapp/  
├── templates/  
│   ├── base.html  
│   ├── home.html  
│   └── about.html  
└── settings.py
```

Ideal para templates compartidos entre múltiples aplicaciones o para proyectos pequeños donde la centralización simplifica la gestión.

Enfoque por Aplicación

```
myproject/  
├── myapp/  
│   ├── templates/  
│   │   ├── myapp/  
│   │   │   ├── index.html  
│   │   │   └── detail.html  
└── settings.py
```

Recomendado para proyectos grandes con múltiples aplicaciones, donde cada una mantiene sus propios templates organizados y encapsulados.

Mejores Prácticas de Organización

1

Usa Namespacing

Cuando creas templates dentro de aplicaciones, siempre crea una subcarpeta con el nombre de la app dentro de templates/. Esto evita conflictos de nombres entre diferentes aplicaciones.

2

Separa por Funcionalidad

Crea subcarpetas dentro de templates para diferentes secciones: `auth/` para autenticación, `blog/` para posts, `includes/` para componentes reutilizables como navbars y footers.

3

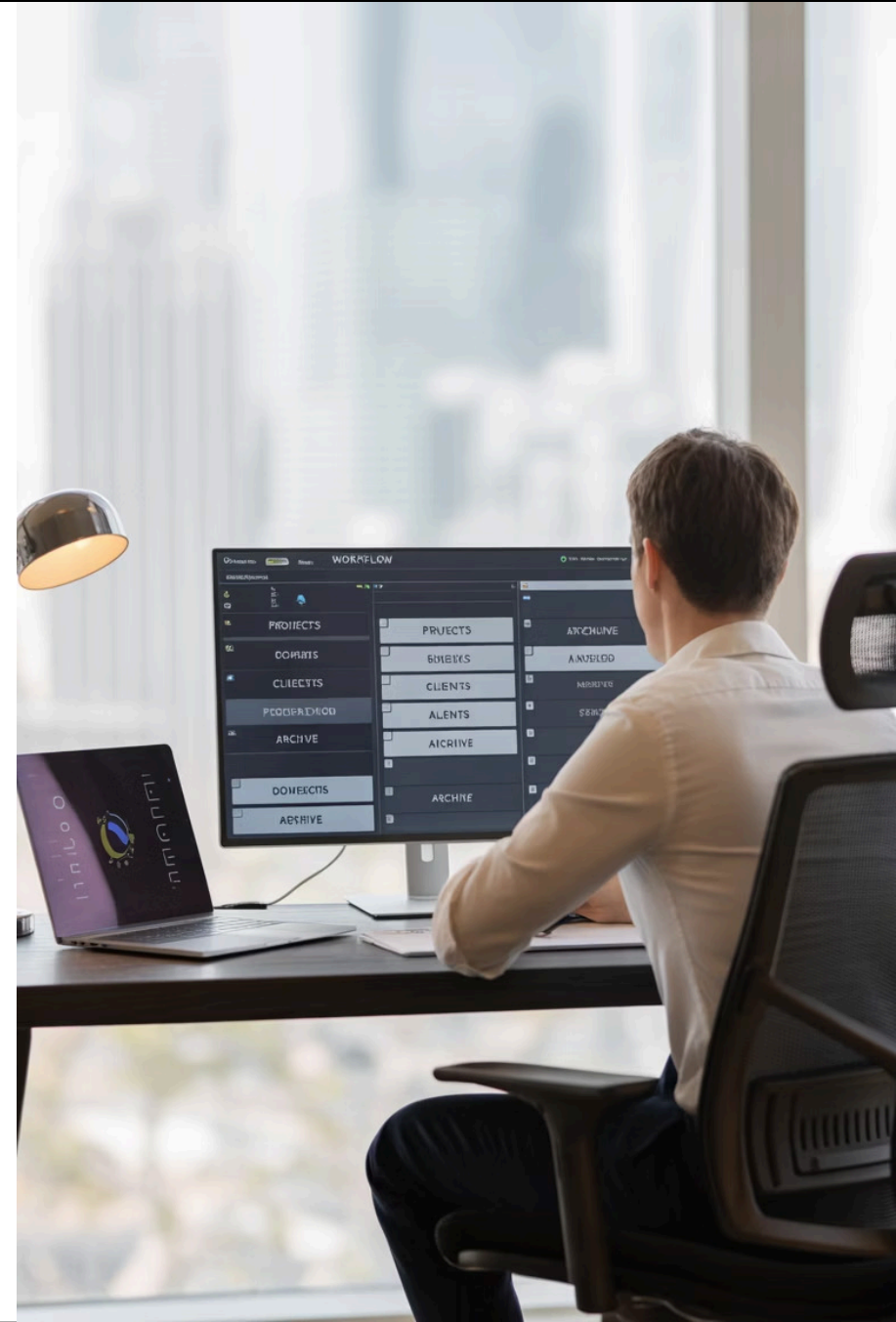
Nombres Descriptivos

Usa nombres claros y descriptivos para tus templates: `article_list.html`, `user_profile.html`, `contact_form.html`. Evita nombres genéricos como `page1.html` o `template.html`.

4

Templates Base

Mantén templates base como `base.html`, `base_admin.html` en la raíz de templates/. Estos sirven como fundamento para el resto de tus páginas mediante herencia.



Configuración Básica: Abriendo settings.py

El archivo `settings.py` es el centro de control de tu proyecto Django. Aquí defines cómo Django debe comportarse, qué aplicaciones usar, cómo conectarse a la base de datos, y crucialmente para nosotros, dónde encontrar los templates.

Este archivo es generado automáticamente cuando creas tu proyecto, pero necesitarás modificarlo frecuentemente a medida que tu aplicación crece. Es importante familiarizarte con su estructura y las diferentes secciones que contiene.

- **Ubicación del archivo**

Se encuentra en `myproject/myproject/settings.py`, dentro de la carpeta del proyecto que tiene el mismo nombre

- **Formato Python**

Es un archivo Python normal, lo que significa que puedes usar variables, importaciones y lógica Python estándar

- **Sensible a mayúsculas**

Todas las configuraciones usan MAYÚSCULAS por convención (DEBUG, INSTALLED_APPS, TEMPLATES, etc.)

Localizando la Configuración TEMPLATES

Dentro de `settings.py`, busca la variable `TEMPLATES`. Esta es una lista que contiene diccionarios de configuración para diferentes motores de templates. Django viene con su propio motor de templates, pero también puede usar otros como Jinja2.

La configuración por defecto ya está preparada para funcionar, pero necesitarás modificar la clave `DIRS` para indicarle a Django dónde buscar tus templates personalizados.

```
TEMPLATES = [  
    {  
        'BACKEND':  
        'django.template.backends.django.Dj  
        angoTemplates',  
        'DIRS': [],  
        'APP_DIRS': True,  
        'OPTIONS': {  
        'context_processors': [  
            ...  
        ],  
    },  
},  
]
```

Entendiendo Cada Configuración de TEMPLATES

BACKEND

Especifica qué motor de templates usar. El valor por defecto es el motor de templates de Django, pero podrías usar Jinja2 u otros si lo prefieres.

DIRS

Lista de directorios donde Django buscará templates. Aquí es donde añadirás la ruta a tu carpeta templates/. Puede estar vacía si solo usas templates de aplicaciones.

APP_DIRS

Si es True, Django buscará automáticamente carpetas templates/ dentro de cada aplicación instalada. Generalmente se deja en True para máxima flexibilidad.

OPTIONS

Configuraciones adicionales como context_processors, que son funciones que añaden variables disponibles en todos los templates automáticamente.

Modificando DIRS: Método con os.path

Para que Django encuentre tus templates en una carpeta global, necesitas añadir su ruta a la lista `DIRS`. El método más común y compatible usa el módulo `os.path` de Python para construir rutas de manera independiente del sistema operativo.

Paso 1: Importar os

Al inicio de `settings.py`, verifica que existe esta línea:

```
import os
```

Si no está, añádela en la parte superior del archivo junto con las demás importaciones.

Paso 2: Modificar DIRS

Encuentra la línea `'DIRS': []`, y cámbiala por:

```
'DIRS': [os.path.join(BASE_DIR, 'templates')],
```

Esto crea una ruta absoluta a la carpeta templates en la raíz de tu proyecto.

Modificando DIRS: Método Moderno con pathlib

Django moderno (versión 3.1+) usa `pathlib` en lugar de `os.path` por defecto. Este es un módulo más reciente y pythónico para manejar rutas de archivos, con mejor legibilidad y funcionalidad.

1

BASE_DIR ya está definido

En la parte superior de `settings.py` encontrarás:

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
```

2

Modifica DIRS

Usa la sintaxis moderna de `pathlib`:

```
'DIRS': [BASE_DIR / 'templates'],
```

El operador `/` une rutas de manera elegante y multiplataforma.

¿Qué es BASE_DIR?

`BASE_DIR` es una variable fundamental en Django que representa la ruta absoluta al directorio raíz de tu proyecto. Es calculada automáticamente al crear el proyecto y sirve como punto de referencia para todas las demás rutas.

Esta variable es crucial porque hace tu configuración portable. Sin importar dónde se ejecute tu proyecto (tu computadora, un servidor, Docker, etc.), `BASE_DIR` siempre apuntará correctamente a la raíz del proyecto.

Se define usando `__file__`, que es la ruta del propio archivo `settings.py`, y luego sube dos niveles (`.parent.parent`) para llegar a la raíz del proyecto donde está `manage.py`.



📌 **Consejo Pro:** Nunca uses rutas hardcodeadas como `'/Users/tu-usuario/proyecto/templates'`. Siempre construye rutas relativas a `BASE_DIR` para mantener tu proyecto portable.

Configuración Completa: Ejemplo Visual

```
# En la parte superior de settings.py
from pathlib import Path
import os

BASE_DIR = Path(__file__).resolve().parent.parent

# Más abajo en el archivo...
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'], # Método pathlib (recomendado)
        # 'DIRS': [os.path.join(BASE_DIR, 'templates')], # Método alternativo
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Verificando la Configuración

Después de modificar `settings.py`, es importante verificar que no haya errores de sintaxis. Django te alertará si hay problemas cuando intentes ejecutar el servidor.

01

Guarda settings.py

Asegúrate de guardar todos los cambios en el archivo antes de continuar.

02

Ejecuta el servidor

En la terminal, ejecuta: `python manage.py runserver`

03

Revisa la salida

Si ves errores rojos, revisa tu sintaxis.
Si dice "Starting development server...", ¡todo está bien!

Creando Tu Primera Vista: Introducción



Las vistas son el cerebro de tu aplicación Django. Son funciones (o clases) de Python que reciben solicitudes web y devuelven respuestas. En nuestro caso, una vista procesará datos y los enviará a un template para renderizar HTML.

Cada vista tiene acceso al objeto `request`, que contiene información sobre la solicitud HTTP: método (GET/POST), parámetros, cookies, sesión del usuario, etc. Con esta información, la vista decide qué datos preparar y qué template usar.

El Archivo `views.py`

Cuando creaste tu aplicación con `startapp`, Django generó automáticamente un archivo `views.py` dentro de la carpeta de tu aplicación. Este archivo es donde escribirás todas las funciones de vista de esa aplicación específica.

Ubicación

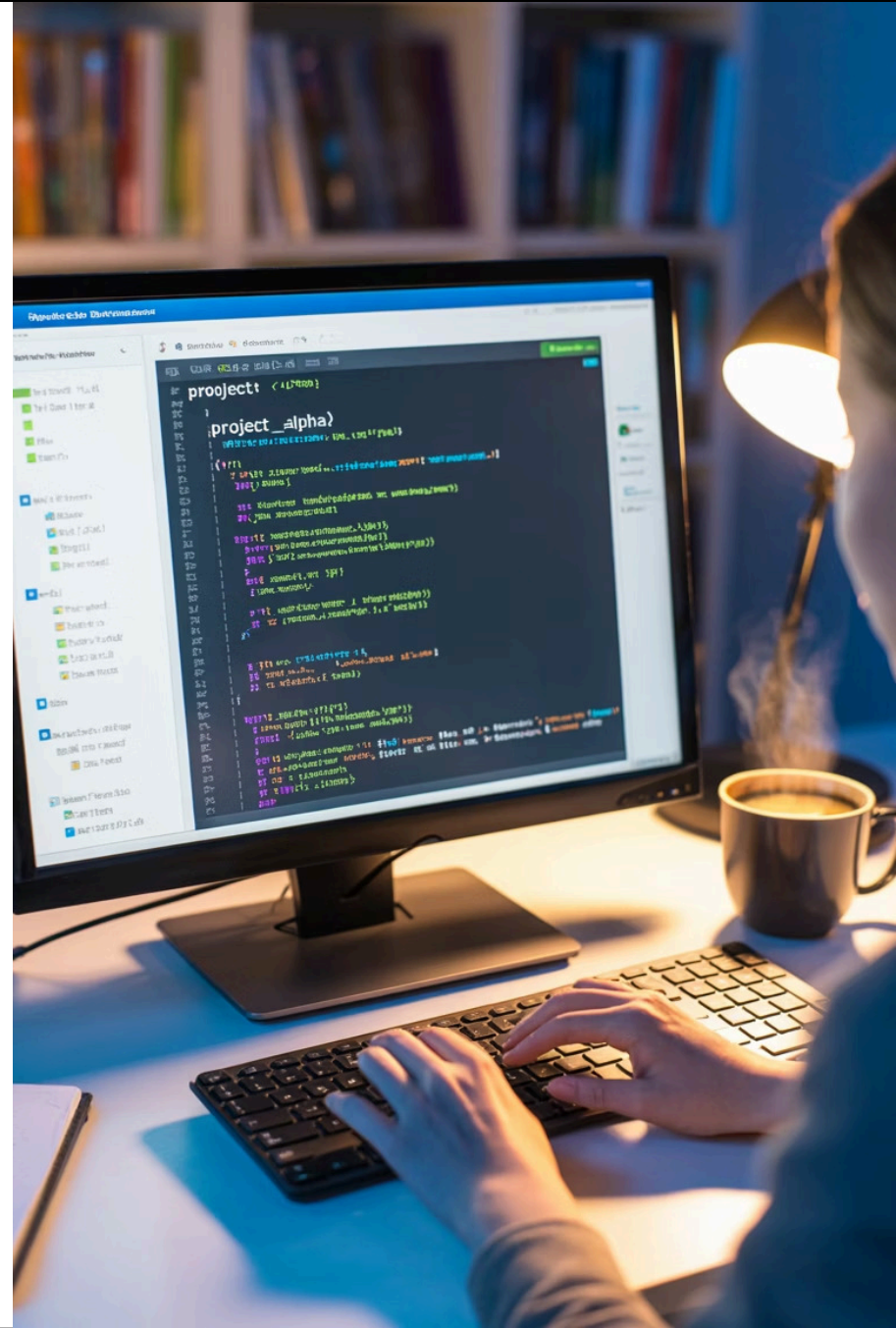
Encontrarás `views.py` en `myproject/myapp/views.py`. Cada aplicación tiene su propio archivo `views.py` independiente.

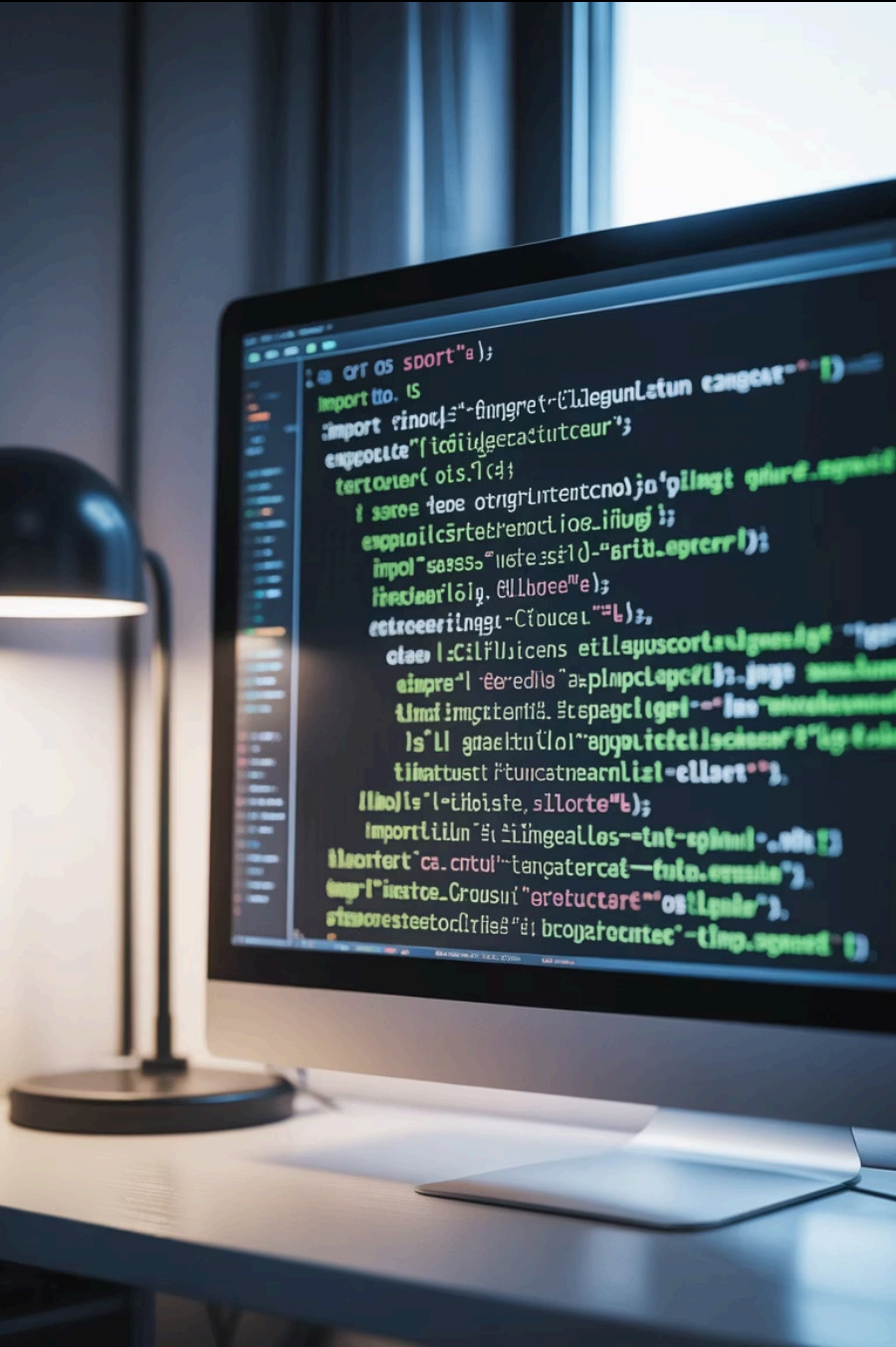
Contenido Inicial

El archivo viene casi vacío, solo con importaciones básicas. Tú añadirás tus propias funciones de vista según las necesidades de tu aplicación.

Organización

Para proyectos grandes, considera crear un paquete `views/` con múltiples archivos (`views_blog.py`, `views_users.py`) para mejor organización.





Importando la Función render

La función `render()` es un atajo super útil proporcionado por Django que combina varios pasos en uno solo. Toma un template, lo llena con datos (contexto), y devuelve una respuesta HTTP completa lista para enviarse al navegador.

Importación Básica

```
from django.shortcuts import  
render
```

Esta línea debe ir al inicio de tu archivo `views.py`. El módulo `shortcuts` contiene funciones auxiliares que simplifican tareas comunes.

¿Por qué usar render?

Sin `render`, necesitarías importar y usar `HttpResponse`, cargar el template manualmente, renderizarlo con contexto, y envolver todo en la respuesta. `Render` hace todo esto en una línea.

Anatomía de una Función de Vista

Una vista típica en Django sigue una estructura predecible. Veamos cada parte con detalle:

```
def mi_vista(request):  
    # 1. Procesar la solicitud (opcional)  
    nombre_usuario = request.GET.get('nombre', 'Invitado')  
  
    # 2. Obtener datos (de base de datos, APIs, etc.)  
    datos_importantes = {  
        'titulo': 'Mi Página',  
        'usuario': nombre_usuario,  
        'items': [1, 2, 3]  
    }  
  
    # 3. Renderizar template con datos  
    return render(request, 'mi_template.html', datos_importantes)
```

1 Firma de la función

Siempre recibe `request` como primer parámetro. Puede tener parámetros adicionales capturados de la URL.

2 Preparar contexto

Crea un diccionario con todos los datos que el template necesitará. Las claves serán nombres de variables en el template.

3 Retornar respuesta

Usa `render()` para combinar `request`, nombre del template y contexto en una respuesta HTTP.

Tu Primera Vista Simple

Empecemos con el ejemplo más básico posible: una vista que simplemente renderiza un template sin pasar datos complejos. Esto te permitirá verificar que toda la configuración está correcta.

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')
```

Esta vista minimalista recibe una solicitud y devuelve el template home.html renderizado. No pasa contexto adicional, así que el template solo podrá usar variables globales del contexto.



- ❑ **Nota:** Aunque esta vista no pasa contexto explícito, los context_processors configurados en settings.py añaden automáticamente variables como 'request', 'user', etc.

Pasando Datos al Template: El Contexto

El contexto es simplemente un diccionario de Python donde las claves son nombres de variables y los valores son los datos que quieres mostrar en el template. Puedes pasar cualquier tipo de dato: strings, números, listas, diccionarios, objetos de modelos, etc.

Ejemplo Completo

```
def home(request):
    contexto = {
        'saludo': '¡Hola Django!',
        'usuario': 'Ivan Malaver',
        'edad': 30,
        'activo': True,
        'lista_items': ['Python', 'Django', 'HTML'],
        'configuracion': {
            'tema': 'oscuro',
            'idioma': 'es'
        }
    }
    return render(request, 'home.html', contexto)
```

En el Template

```
<h1>{{ saludo }}</h1>
<p>Usuario: {{ usuario }}</p>
<p>Edad: {{ edad }}</p>

{% if activo %}
    <span>Activo</span>
{% endif %}

{% for item in lista_items %}
    <li>{{ item }}</li>
{% endfor %}

<p>Tema: {{ configuracion.tema }}</p>
```




Vista con Datos Dinámicos de Base de Datos

En aplicaciones reales, generalmente obtienes datos de la base de datos usando los modelos de Django. Aquí hay un ejemplo más realista que consulta objetos del modelo y los pasa al template:

```
from django.shortcuts import render
from .models import Artículo, Categoria

def lista_articulos(request):
    # Obtener todos los artículos publicados
    articulos = Artículo.objects.filter(publicado=True).order_by('-fecha_creacion')

    # Obtener categorías para el menú
    categorias = Categoria.objects.all()

    # Estadísticas
    total_articulos = articulos.count()

    contexto = {
        'articulos': articulos,
        'categorias': categorias,
        'total': total_articulos,
        'titulo_pagina': 'Blog de Ivan Malavier Fierro'
    }

    return render(request, 'blog/lista_articulos.html', contexto)
```


Vistas Basadas en Clases (CBV): Una Introducción



Además de funciones, Django permite crear vistas usando clases (Class-Based Views). Estas son más poderosas y reutilizables, especialmente para operaciones comunes como listar objetos, mostrar detalles, crear o editar registros.

Las CBV heredan de clases base de Django que ya implementan funcionalidad común, reduciendo drásticamente el código que necesitas escribir. Por ejemplo, `ListView` automáticamente pagina resultados y maneja contexto.

```
from django.views.generic import ListView
from .models import Artículo
```

```
class ListaArticulosView(ListView):
    model = Artículo
    template_name = 'blog/lista.html'
    context_object_name = 'articulos'
    paginate_by = 10
```

Creando Tu Primer Template HTML

Ahora que tienes una vista configurada, necesitas crear el template que renderizará. Un template de Django es un archivo HTML normal con sintaxis especial para manejar datos dinámicos.

Crea un archivo llamado `home.html` dentro de tu carpeta `templates/` (o `templates/myapp/` si usas organización por aplicación). Asegúrate de que el nombre coincida exactamente con el que especificaste en tu vista.

Ubicación correcta

Verifica que el template está en la ruta que configuraste en DIRS o dentro de `templates/` de una aplicación registrada

Extensión .html

Siempre usa la extensión `.html` para que tu editor lo reconozca y proporcione resaltado de sintaxis adecuado

Codificación UTF-8

Guarda tus templates en UTF-8 para soportar caracteres especiales, acentos y emojis sin problemas

Estructura Básica de un Template

Un template HTML de Django comienza como cualquier archivo HTML estándar, con su estructura básica. La diferencia está en las etiquetas especiales de Django que insertamos para hacer el contenido dinámico.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{{ titulo_pagina }}</title>
</head>
<body>
  <header>
    <h1>{{ titulo_principal }}</h1>
  </header>

  <main>
    {% block contenido %}
    <!-- El contenido específico va aquí -->
    {% endblock %}
  </main>

  <footer>
    <p>© 2024 Ivan Malaver Fierro</p>
  </footer>
</body>
</html>
```

[illegible]

Sintaxis de Variables: {{ }}

La sintaxis de doble llave `{{ variable }}` se usa para insertar valores de variables del contexto en tu HTML. Django automáticamente escapa caracteres HTML peligrosos para prevenir ataques XSS.

Ejemplos de Uso

<!-- Variable simple -->

<p>{{ nombre }}</p>

<!-- Acceso a atributos -->

<p>{{ usuario.email }}</p>

<!-- Acceso a métodos (sin paréntesis) -->

<p>{{ articulo.get_fecha_formateada }}</p>

<!-- Acceso a diccionarios -->

<p>{{ config.tema }}</p>

<!-- Acceso a índices de lista -->

<p>{{ items.0 }}</p>



Sintaxis de Etiquetas: {% %}

Las etiquetas {% etiqueta %} proporcionan lógica de programación en tus templates: bucles, condicionales, herencia, bloques, etc. Son más poderosas que las variables pero deben usarse con moderación.

Condicionales

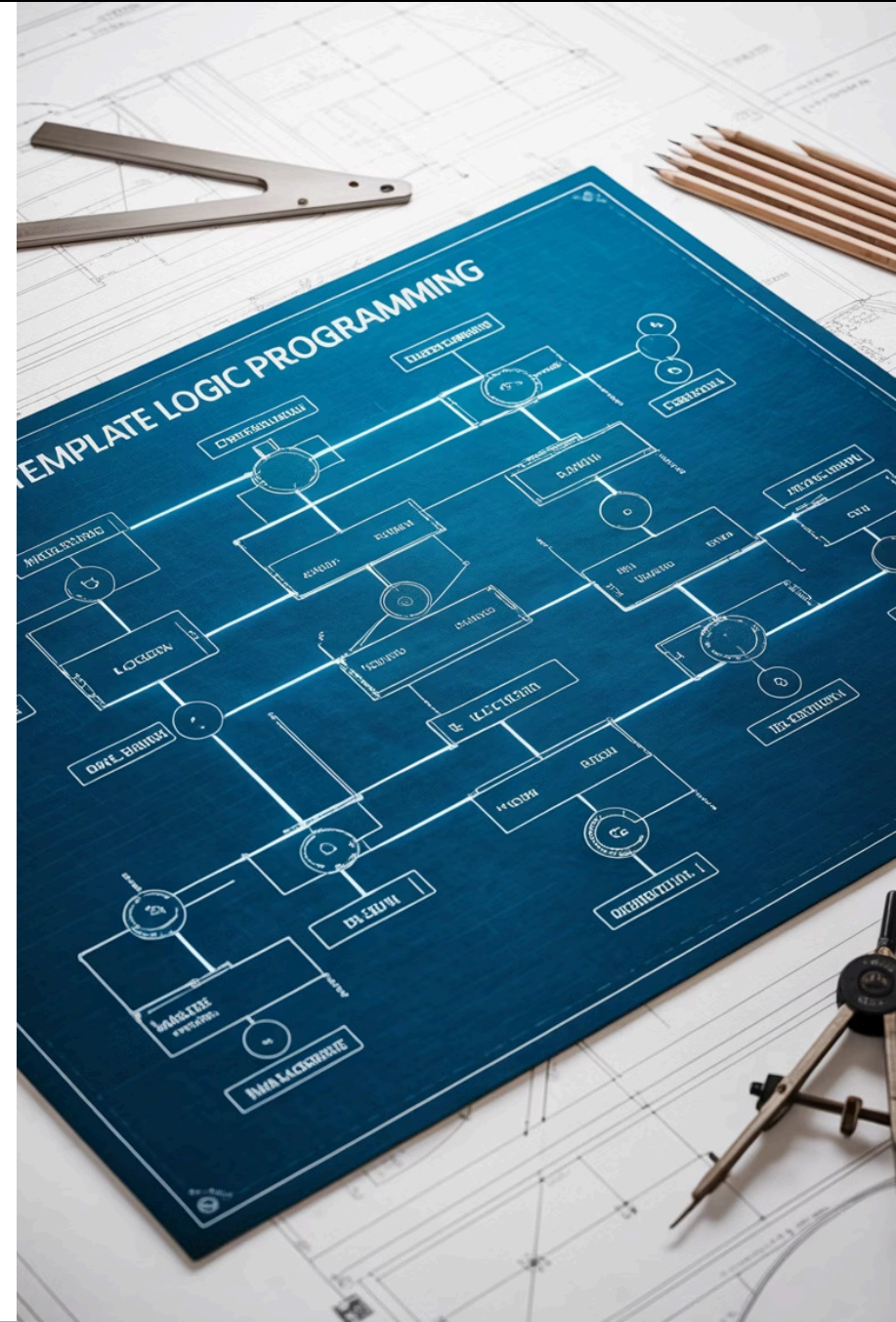
```
{% if usuario.is_authenticated %}
  <p>Bienvenido {{ usuario.username }}
</p>
{% else %}
  <p>Por favor inicia sesión</p>
{% endif %}
```

Bucles

```
{% for item in lista %}
  <li>{{ item.nombre }}</li>
{% empty %}
  <p>No hay items</p>
{% endfor %}
```

Comentarios

```
{# Este es un comentario #}
{% comment %}
  Comentario de
  múltiples líneas
{% endcomment %}
```



Filtros de Template: Transformando Datos

Los filtros modifican variables antes de mostrarlas. Se aplican usando el símbolo pipe `|`. Django incluye docenas de filtros útiles y puedes crear los tuyos propios.

Filtros de Texto

```
{# Mayúsculas #}  
{{ nombre|upper }}  
  
{# Primera letra mayúscula #}  
{{ titulo|title }}  
  
{# Truncar texto #}  
{{ descripcion|truncatewords:30 }}  
  
{# Eliminar HTML #}  
{{ contenido|striptags }}
```

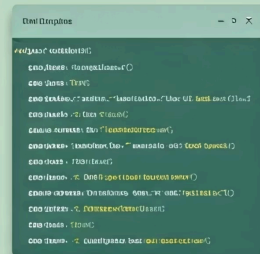
Filtros Numéricos

```
{# Formatear números #}  
{{ precio|floatformat:2 }}  
  
{# Separadores de miles #}  
{{ poblacion|intcomma }}  
  
{# Operaciones matemáticas #}  
{{ cantidad|add:10 }}  
{{ total|divisibleby:2 }}
```

Filtros de Fecha

```
{# Formatear fechas #}  
{{ fecha|date:"d/m/Y" }}  
  
{# Tiempo relativo #}  
{{ creado|timesince }}  
  
{# Fecha por defecto #}  
{{ publicado|default:"No publicado" }}
```


Template con Ejemplo Completo



Aquí está el template home.html completo que usa las variables enviadas desde la vista que creamos anteriormente:

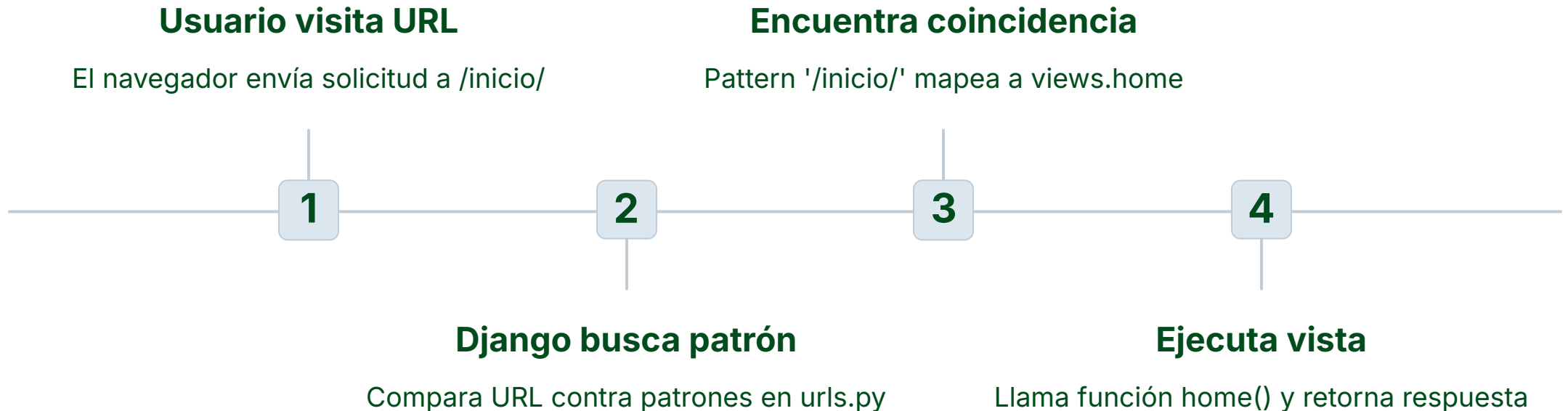
```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Mi Página Django</title>
<style>
body { font-family: Arial, sans-serif; margin: 40px; }
.lista-tech { background: #f0f0f0; padding: 20px; }
</style>
</head>
<body>
<h1>{{ saludo }}</h1>
<p>Esta página fue creada por Ivan Malaver Fierro</p>

<div class="lista-tech">
<h2>Lista de tecnologías:</h2>
<ul>
{% for item in lista_items %}
<li>{{ item }}</li>
{% endfor %}
</ul>
</div>
</body>
</html>
```

Conectando Todo: El Archivo urls.py

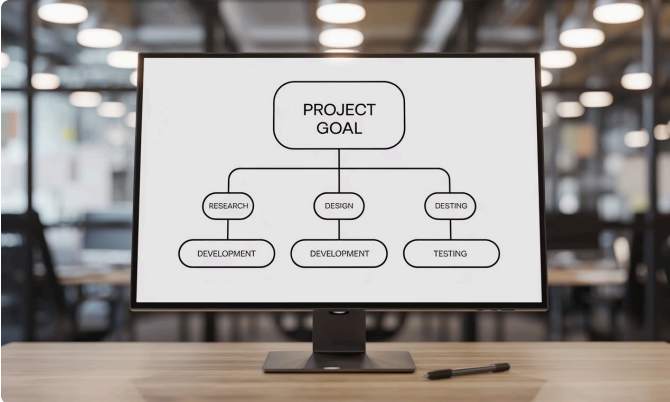
Ahora que tienes una vista y un template, necesitas decirle a Django qué URL debe activar tu vista. Esto se hace en el archivo `urls.py`. Django usa un sistema de enrutamiento basado en patrones para mapear URLs a vistas.

Existen dos niveles de URLs en Django: las URLs del proyecto principal (en la carpeta del proyecto) y las URLs de cada aplicación. La práctica recomendada es mantener las URLs de cada aplicación en su propio archivo `urls.py`.



URLs del Proyecto vs URLs de Aplicación

URLs del Proyecto



El archivo `urls.py` principal está en `myproject/myproject/urls.py`. Este archivo generalmente importa y incluye las URLs de cada aplicación usando `include()`, manteniendo una arquitectura modular.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('myapp.urls')),
    path('blog/', include('blog.urls')),
]
```

URLs de Aplicación



Cada aplicación tiene su propio `urls.py` que debes crear manualmente en `myproject/myapp/urls.py`. Este contiene solo las URLs específicas de esa aplicación.

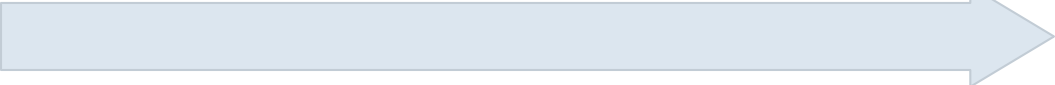
```
from django.urls import path
from . import views

app_name = 'myapp'

urlpatterns = [
    path("", views.home, name='home'),
    path('about/', views.about, name='about'),
]
```

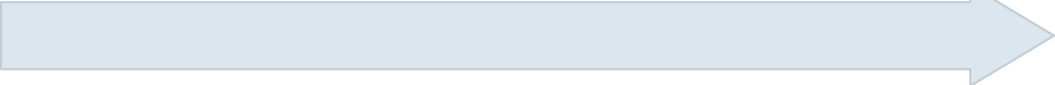
Creando urls.py en Tu Aplicación

Si tu aplicación no tiene un archivo `urls.py` (la mayoría no lo tienen por defecto), necesitas crearlo manualmente. Este paso es fundamental para mantener tus URLs organizadas y modulares.



Crear archivo

En la carpeta de tu aplicación, crea un nuevo archivo llamado `urls.py` al mismo nivel que `views.py`



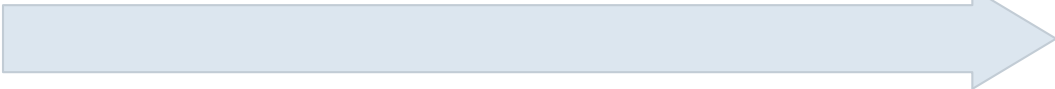
Lista `urlpatterns`

Crea la lista `urlpatterns = []` donde añadirás tus patrones de URL



Importaciones básicas

Añade las importaciones necesarias: `from django.urls import path` y `from . import views`



Incluir en proyecto

Ve al `urls.py` del proyecto y usa `include('myapp.urls')` para conectarlo

Anatomía de un Patrón de URL

Cada patrón de URL en Django se define usando la función `path()`, que toma varios argumentos para configurar cómo se debe manejar una ruta específica.

```
path('ruta/', vista_a_ejecutar, name='nombre_url')
```

1

Ruta (route)

String que define el patrón de URL. No incluye dominio ni parámetros de query. Ejemplo: 'blog/post/' o '' para la raíz. Siempre usa / al final por convención.

2

Vista (view)

La función o clase de vista que se ejecutará cuando coincida este patrón. Puede ser una referencia a la función o el resultado de `.as_view()` para CBV.

3

Nombre (name)

Identificador único opcional pero altamente recomendado. Te permite referenciar URLs en templates y vistas sin hardcodear la ruta. Usa nombre descriptivo en `snake_case`.

Configuración Completa de URLs

myapp/urls.py

```
from django.urls import path
from . import views

# Namespace de la aplicación
app_name = 'myapp'

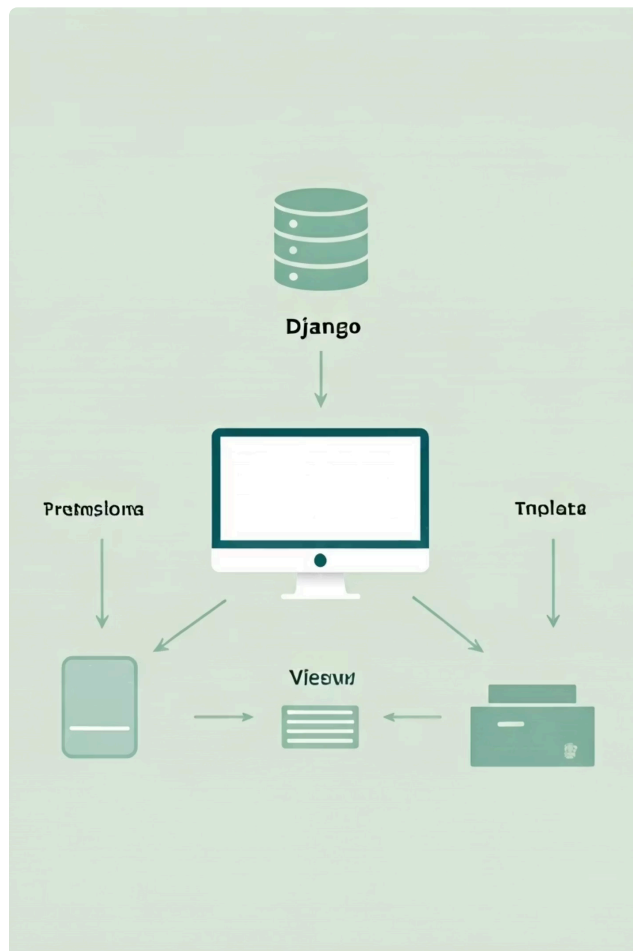
urlpatterns = [
    path("", views.home, name='home'),
    path('about/', views.about, name='about'),
    path('contact/', views.contact, name='contact'),
]
```

myproject/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),

    # Incluir URLs de myapp
    path("", include('myapp.urls')),
]
```



Con esta configuración, Django enrutará correctamente las solicitudes a tus vistas.

URLs con Parámetros Dinámicos

Frecuentemente necesitas capturar valores de la URL para usarlos en tus vistas. Por ejemplo, mostrar un artículo específico basado en su ID. Django proporciona convertidores de path para esto.

En urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    # Captura un entero
    path('articulo/<int:id>/',
         views.detalle_articulo,
         name='detalle_articulo'),

    # Captura un string (slug)
    path('post/<slug:slug>/',
         views.detalle_post,
         name='detalle_post'),

    # Captura cualquier string
    path('usuario/<str:username>/',
         views.perfil_usuario,
         name='perfil'),
]
```

En views.py

```
def detalle_articulo(request, id):
    articulo = Artículo.objects.get(id=id)
    return render(request, 'detalle.html',
                  {'articulo': articulo})

def detalle_post(request, slug):
    post = Post.objects.get(slug=slug)
    return render(request, 'post.html',
                  {'post': post})

def perfil_usuario(request, username):
    usuario =
    User.objects.get(username=username)
    return render(request, 'perfil.html',
                  {'usuario': usuario})
```

aguary

view
view

Campaigns →

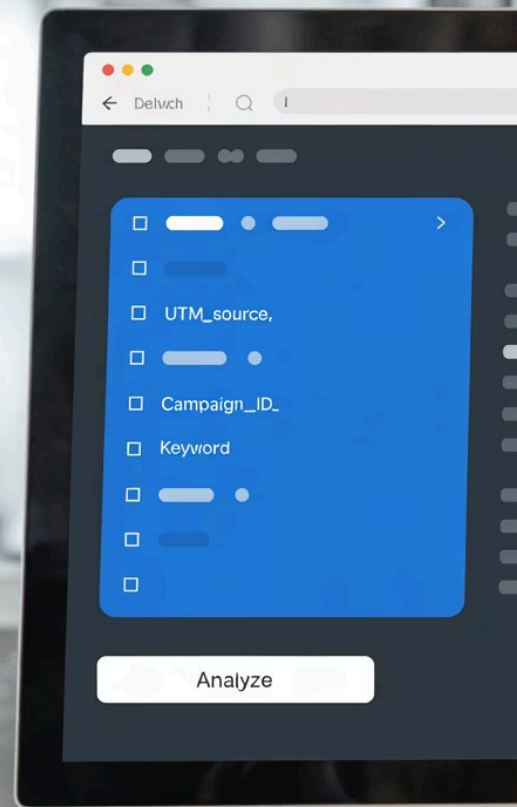
Reports →

Settings →

unaine onser/OW

or Poilo/

Privacy Policy
Privacy Policy
Terms of Service



Convertidores de Path Disponibles



int

Coincide con números enteros positivos. Ejemplo: `<int:id>` captura 5, 100, 999 pero no -5 o 3.14



str

Coincide con cualquier string no vacío excepto '/'. Es el tipo por defecto si no especificas uno. Ejemplo: `<str:nombre>`



slug

Coincide con strings que contienen solo letras, números, guiones y guiones bajos. Ideal para URLs amigables. Ejemplo: `<slug:articulo-slug>`



uuid

Coincide con UUIDs formateados. Útil para identificadores únicos seguros. Ejemplo: `<uuid:token>`



path

Coincide con cualquier string incluyendo '/'. Útil para capturar rutas completas. Ejemplo: `<path:ruta_archivo>`



Nombres de URLs y Reversión

Darle un nombre a cada URL es una práctica fundamental en Django. Permite cambiar la URL sin modificar código en múltiples lugares. Usa la función `reverse()` en vistas y la etiqueta `{% url %}` en templates.

En Templates

```
<!-- Sin parámetros -->
<a href="{% url 'myapp:home'
%}">Inicio</a>

<!-- Con parámetros -->
<a href="{% url 'myapp:detalle_articulo'
id=5 %}">
    Ver artículo
</a>

<!-- Con variables -->
<a href="{% url 'myapp:perfil'
username=usuario.username %}">
    {{ usuario.nombre }}
</a>
```

En Views.py

```
from django.shortcuts import redirect
from django.urls import reverse

def mi_vista(request):
    # Reverse sin parámetros
    url = reverse('myapp:home')

    # Reverse con parámetros
    url = reverse('myapp:detalle_articulo',
kwargs={'id': 5})

    # Redirect usando reverse
    return redirect('myapp:home')
```

Probando Tu Configuración Completa

Es momento de verificar que todo funcione correctamente. Sigue estos pasos para probar tu primera página Django con backend conectado a template.

1

Inicia el servidor de desarrollo

En tu terminal, ejecuta: `python manage.py runserver`. Deberías ver "Starting development server at `http://127.0.0.1:8000/`"

2

Abre el navegador

Visita `http://localhost:8000/` o la URL específica que configuraste. Deberías ver tu template renderizado con los datos de la vista.

3

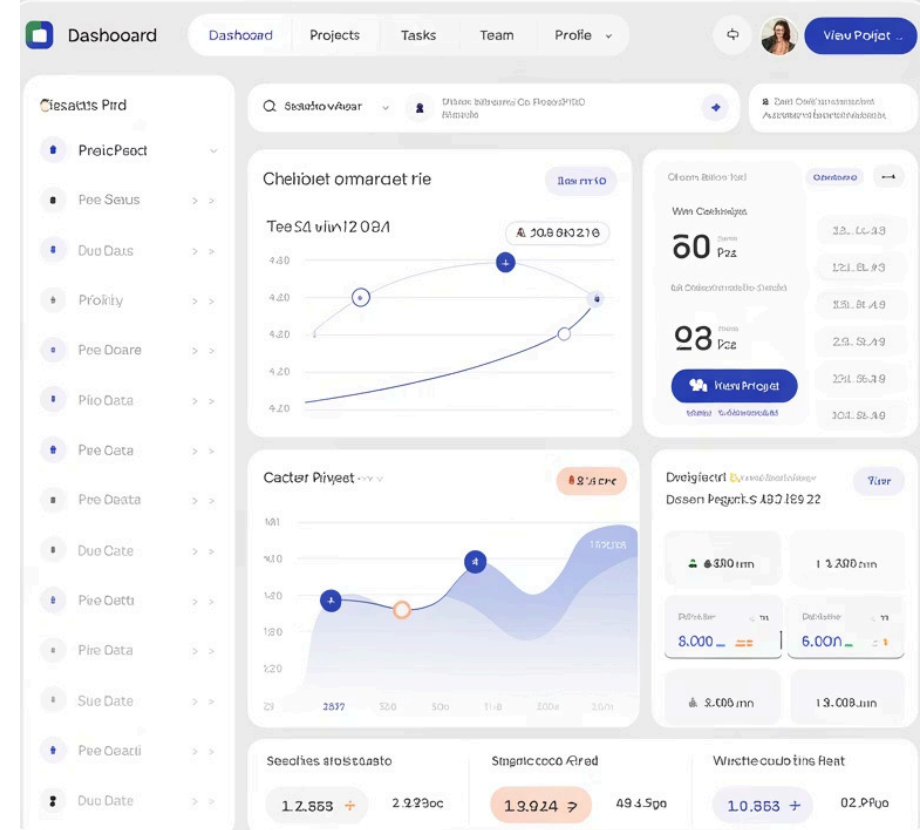
Verifica los datos

Confirma que las variables del contexto aparecen correctamente. Si ves `{{ variable }}` sin renderizar, hay un error en el template.

4

Revisa la consola

Cualquier error aparecerá en la terminal donde ejecutaste `runserver`. Django proporciona mensajes de error muy descriptivos.



Errores Comunes y Soluciones

TemplateDoesNotExist

Causa: Django no encuentra tu template. **Solución:** Verifica la ruta en DIRS de settings.py, asegúrate que el nombre del archivo coincide exactamente (case-sensitive), y que el archivo está en la ubicación correcta.

Page not found (404)

Causa: No hay patrón de URL que coincida. **Solución:** Revisa urls.py, asegúrate de haber incluido las URLs de la app en el proyecto, verifica que la ruta no tenga typos.

Variable {{ }} no renderiza

Causa: La variable no existe en el contexto. **Solución:** Verifica que pasaste la variable en el diccionario de contexto en la vista, revisa el nombre por typos.

NoReverseMatch

Causa: Error al usar {% url %} con nombre incorrecto. **Solución:** Verifica que el nombre de URL existe en urls.py, incluye el namespace si lo usas (app:nombre).

El Flujo Completo: Resumen Visual



Usuario hace solicitud

El navegador envía petición HTTP a tu servidor Django con una URL específica



Django compara URLs

El sistema de URLs busca en urlpatterns hasta encontrar patrón que coincida



Vista procesa lógica

La función de vista se ejecuta, consulta base de datos si es necesario, prepara contexto



Template renderiza

Django carga el template, reemplaza variables con datos del contexto, genera HTML final



Respuesta al navegador

El HTML completo se envía como respuesta HTTP y el navegador lo muestra al usuario

Herencia de Templates: Concepto Fundamental



La herencia de templates es uno de los features más poderosos de Django. Te permite crear un template "padre" con la estructura común (navbar, footer, CSS) y templates "hijos" que solo definen su contenido único.

Esto sigue el principio DRY (Don't Repeat Yourself). En lugar de copiar el mismo header y footer en 50 páginas, lo defines una vez en el template base y todas las páginas lo heredan. Si necesitas cambiar el footer, lo cambias en un solo lugar.

La herencia usa dos etiquetas clave: `{% extends %}` para indicar qué template heredar, y `{% block %}` para definir áreas que los hijos pueden sobrescribir.

Creando un Template Base

El template base (generalmente `base.html`) contiene toda la estructura HTML común que compartirán tus páginas. Define bloques en los lugares donde los templates hijos insertarán contenido específico.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block titulo %}Mi Sitio Django{% endblock %}</title>

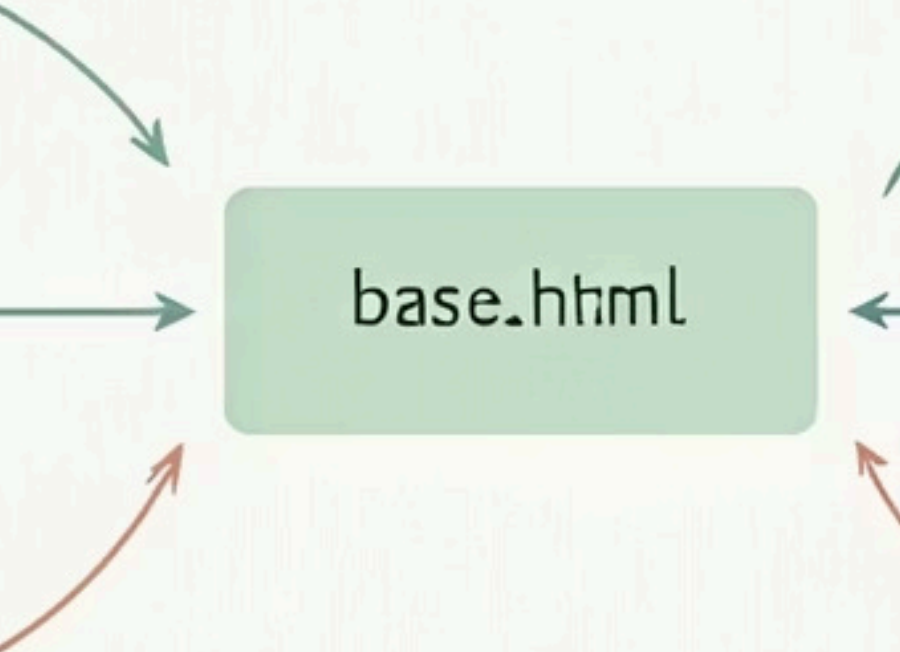
  {% block css_extra %}{% endblock %}
</head>
<body>
  <header>
    <nav>
      <h1>Mi Aplicación Django</h1>
      <ul>
        <li><a href="{% url 'myapp:home' %}">Inicio</a></li>
        <li><a href="{% url 'myapp:about' %}">Acerca de</a></li>
      </ul>
    </nav>
  </header>

  <main>
    {% block contenido %}
    {# Los templates hijos sobrescribirán este bloque #}
    {% endblock %}
  </main>

  <footer>
    <p>© 2024 Ivan Malaver Fierro</p>
  </footer>

  {% block js_extra %}{% endblock %}
</body>
</html>
```

Django Tsnmemdianatio



base.html

Extendiendo el Template Base

Los templates hijos usan `{% extends %}` como primera línea para heredar del base. Luego sobrescriben solo los bloques que necesitan personalizar.

home.html

```
{% extends "base.html" %}

{% block titulo %}
    Inicio - Mi Sitio
{% endblock %}

{% block contenido %}
    <h2>Bienvenido a mi sitio</h2>
    <p>{{ mensaje_bienvenida }}</p>

    <div class="featured">
        <h3>Artículos destacados</h3>
        {% for articulo in destacados %}
            <article>
                <h4>{{ articulo.titulo }}</h4>
                <p>{{ articulo.resumen }}</p>
            </article>
        {% endfor %}
    </div>
{% endblock %}
```

about.html

```
{% extends "base.html" %}

{% block titulo %}
    Acerca de - Mi Sitio
{% endblock %}

{% block contenido %}
    <h2>Acerca de Nosotros</h2>

    <p>{{ descripcion_empresa }}</p>

    <h3>Nuestro Equipo</h3>
    <ul>
        {% for miembro in equipo %}
            <li>
                <strong>{{ miembro.nombre }}</strong>
                - {{ miembro.cargo }}
            </li>
        {% endfor %}
    </ul>
{% endblock %}
```

Bloques Múltiples y Bloques Anidados

Puedes definir múltiples bloques en un template base para permitir personalización granular. Los bloques también pueden ser anidados dentro de otros bloques para mayor flexibilidad.

```
<!-- base.html -->
<!DOCTYPE html>
<html>
<head>
{% block head %}
<title>{% block titulo %}Título por defecto{% endblock %}</title>
{% block css %}
<link rel="stylesheet" href="/static/css/base.css">
{% endblock %}
{% block css_extra %}{% endblock %}
{% endblock %}
</head>
<body>
{% block body %}
{% block header %}
<header>Header por defecto</header>
{% endblock %}

{% block sidebar %}
<aside>Sidebar por defecto</aside>
{% endblock %}

{% block contenido %}
<main>Contenido por defecto</main>
{% endblock %}

{% block footer %}
<footer>Footer por defecto</footer>
{% endblock %}
{% endblock %}
</body>
</html>
```

Super Blocks: Extendiendo sin Sobrescribir

A veces quieres añadir contenido a un bloque del padre en lugar de reemplazarlo completamente. Usa `{{ block.super }}` para incluir el contenido original del bloque padre.

```
<!-- base.html -->
{% block css %}
    <link rel="stylesheet" href="base.css">
{% endblock %}

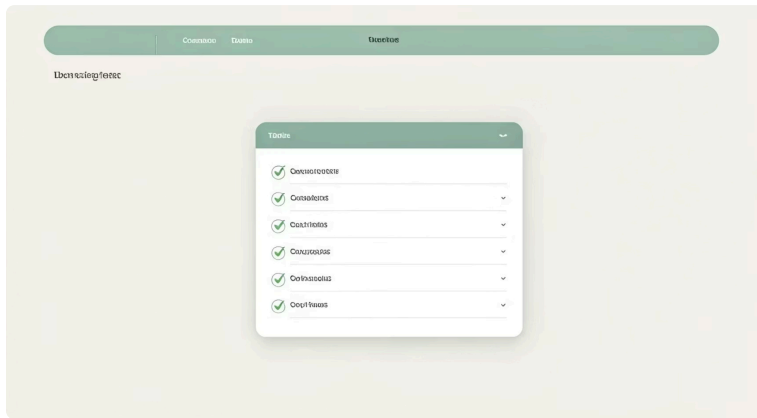
<!-- hijo.html -->
{% extends "base.html" %}

{% block css %}
    {{ block.super }}
    <link rel="stylesheet" href="hijo.css">
{% endblock %}
```

El resultado final incluirá tanto `base.css` como `hijo.css`. Esto es especialmente útil para CSS y JavaScript adicional.



Ejemplo Práctico: Sistema de Blog



Veamos cómo implementar un sistema completo de blog con lista de artículos y vista de detalle usando Django templates. Este ejemplo integra todo lo aprendido: vistas, contexto, herencia y URLs.

models.py

```
from django.db import models
from django.utils import timezone

class Artículo(models.Model):
    titulo = models.CharField(max_length=200)
    slug = models.SlugField(unique=True)
    contenido = models.TextField()
    fecha_publicacion = models.DateTimeField(
        default=timezone.now
    )
    autor = models.CharField(max_length=100)

    def __str__(self):
        return self.titulo

    class Meta:
        ordering = ['-fecha_publicacion']
```

views.py

```
from django.shortcuts import render, get_object_or_404
from .models import Artículo

def lista_articulos(request):
    articulos = Artículo.objects.all()
    contexto = {
        'articulos': articulos,
        'total': articulos.count()
    }
    return render(request, 'blog/lista.html', contexto)

def detalle_articulo(request, slug):
    articulo = get_object_or_404(Articulo, slug=slug)
    contexto = {'articulo': articulo}
    return render(request, 'blog/detalle.html', contexto)
```

Plantillas del Blog: Lista de Artículos

```
<!-- templates/blog/lista.html -->
{% extends "base.html" %}

{% block titulo %}Blog - Todos los artículos{% endblock %}

{% block contenido %}
<h1>Blog de Ivan Malaver Fierro</h1>
<p>Total de artículos: {{ total }}</p>

<div class="articulos-grid">
{% for articulo in articulos %}
<article class="articulo-card">
<h2>
<a href="{% url 'blog:detalle' slug=articulo.slug %}">
{{ articulo.titulo }}
</a>
</h2>

<div class="meta">
<span class="autor">Por {{ articulo.autor }}</span>
<span class="fecha">{{ articulo.fecha_publicacion|date:"d M Y" }}</span>
</div>

<p>{{ articulo.contenido|truncatewords:50 }}</p>

<a href="{% url 'blog:detalle' slug=articulo.slug %}" class="btn">
Leer más →
</a>
</article>
{% empty %}
<p>No hay artículos publicados todavía.</p>
{% endfor %}
</div>
{% endblock %}
```


Templates del Blog: Detalle de Artículo

```
<!-- templates/blog/detalle.html -->
{% extends "base.html" %}

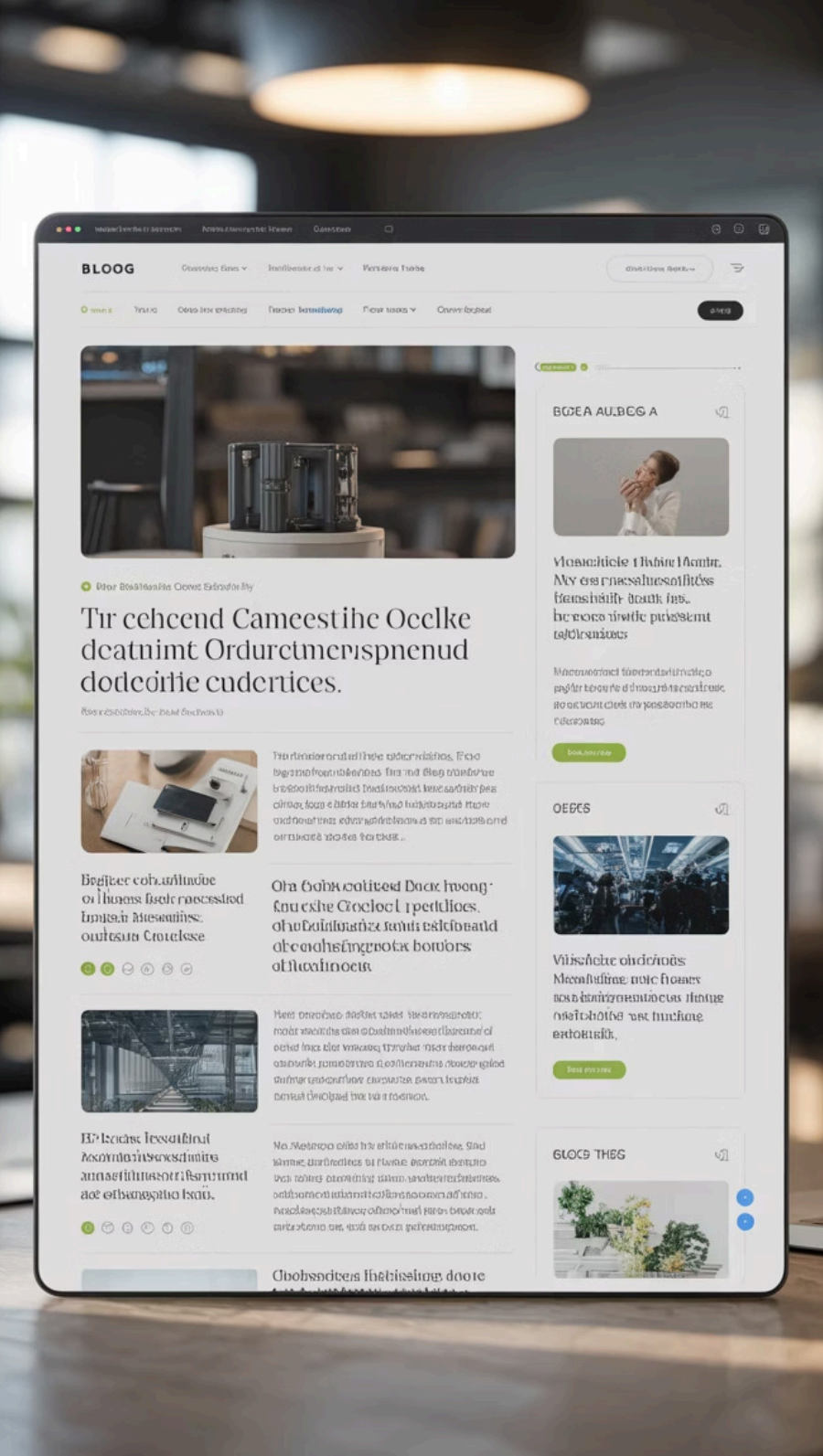
{% block titulo %}{{ articulo.titulo }} - Blog{% endblock %}

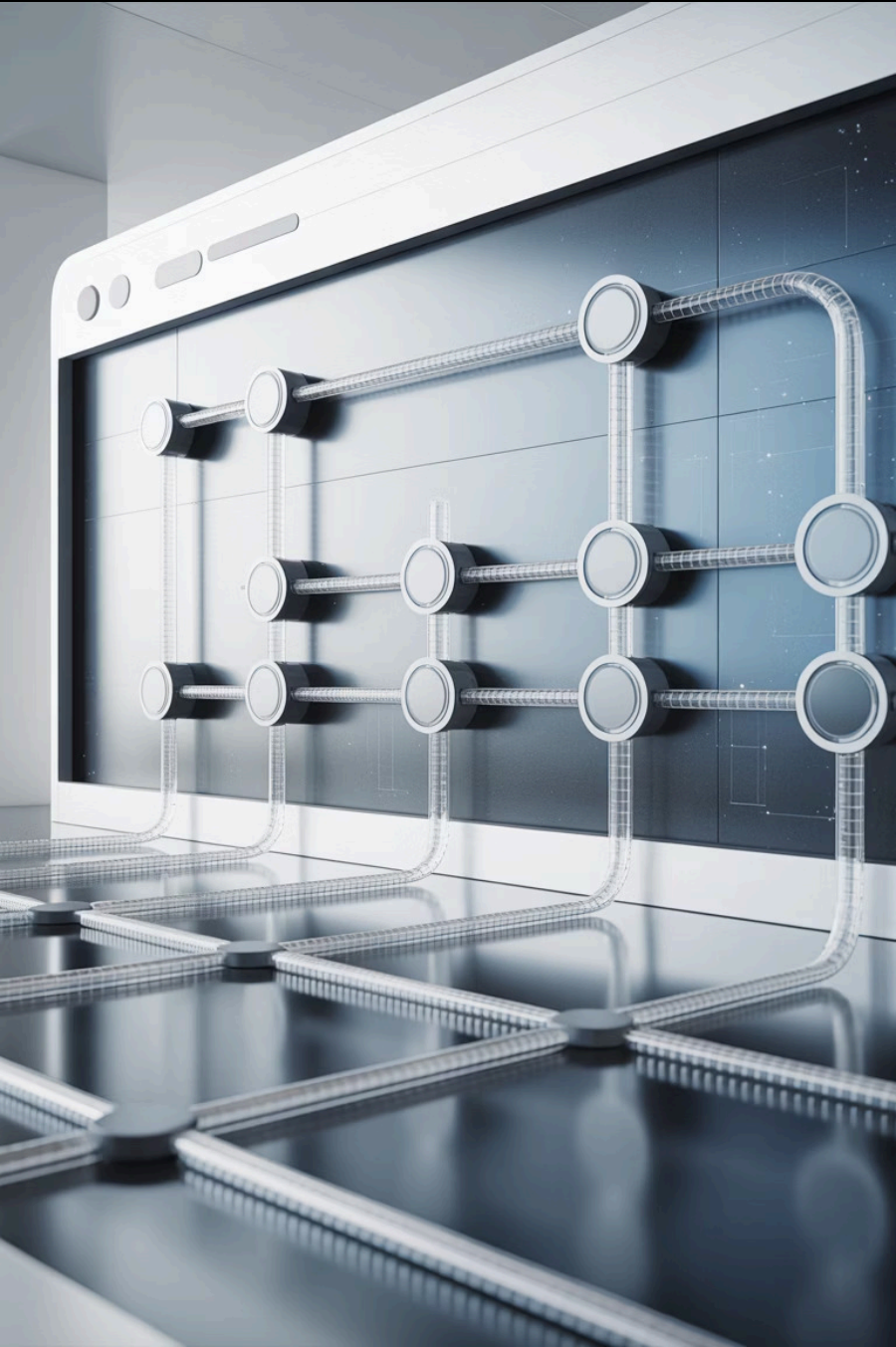
{% block contenido %}
<article class="articulo-detalle">
<header>
<h1>{{ articulo.titulo }}</h1>

<div class="meta">
<span class="autor">
<strong>Autor:</strong> {{ articulo.autor }}
</span>
<span class="fecha">
<strong>Publicado:</strong>
{{ articulo.fecha_publicacion|date:"d de F de Y" }}
</span>
</div>
</header>

<div class="contenido">
{{ articulo.contenido|linebreaks }}
</div>

<footer>
<a href="{% url 'blog:lista' %}" class="btn-volver">
← Volver a la lista
</a>
</footer>
</article>
{% endblock %}
```





URLs del Blog

blog/urls.py

```
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    path("",
        views.lista_articulos,
        name='lista'),

    path('articulo/<slug:slug>',
        views.detalle_articulo,
        name='detalle'),
]
```

Estructura de URLs resultante

- /blog/ → Lista de todos los artículos
- /blog/articulo/mi-primer-post/ → Detalle del artículo con slug "mi-primer-post"
- /blog/articulo/django-templates-guia/ → Otro artículo

Los slugs proporcionan URLs legibles y SEO-friendly en lugar de usar IDs numéricos.

Sistema de Tareas: Otro Ejemplo Completo



Implementemos un sistema de gestión de tareas mostrando cómo manejar datos estructurados y estados booleanos en templates.

views.py

```
def lista_tareas(request):
    tareas = [
        {
            "id": 1,
            "titulo": "Aprender Django",
            "completada": True,
            "prioridad": "alta"
        },
        {
            "id": 2,
            "titulo": "Crear modelos",
            "completada": False,
            "prioridad": "media"
        },
        {
            "id": 3,
            "titulo": "Conectar templates",
            "completada": False,
            "prioridad": "alta"
        },
        {
            "id": 4,
            "titulo": "Hacer deploy",
            "completada": False,
            "prioridad": "baja"
        }
    ]

    contexto = {
        'tareas': tareas,
        'completadas': sum(1 for t in tareas if t['completada']),
        'pendientes': sum(1 for t in tareas if not t['completada'])
    }

    return render(request, 'tareas.html', contexto)
```

templates/tareas.html

```
{% extends "base.html" %}

{% block contenido %}
<h1>Mis Tareas</h1>

<div class="estadisticas">
    <span>Completadas: {{ completadas }}</span>
    <span>Pendientes: {{ pendientes }}</span>
</div>

<ul class="lista-tareas">
{% for tarea in tareas %}
    <li class="{% if tarea.completada %}completada{% endif %}
    prioridad-{{ tarea.prioridad }}">

        {% if tarea.completada %}
        <span class="check">✓</span>
        {% endif %}

        <span class="titulo">{{ tarea.titulo }}</span>

        <span class="badge prioridad">
            {{ tarea.prioridad|upper }}
        </span>
    </li>
{% endfor %}
</ul>
{% endblock %}
```


Includes: Reutilizando Fragmentos de Templates

La etiqueta `{% include %}` permite insertar un template dentro de otro. Es perfecta para componentes que se repiten en múltiples páginas como navbars, footers, cards, modales, etc.

Crear componente

Crea archivos parciales en `templates/includes/` como `_navbar.html`, `_footer.html` (el guión bajo es convención para parciales)

Incluir en templates

Usa `{% include 'includes/_navbar.html' %}` donde necesites el componente. Se renderiza en ese lugar exacto.

Pasar contexto

El include tiene acceso al contexto del template padre. Opcionalmente pasa variables adicionales: `{% include 'card.html' with titulo='Hola' %}`



Ejemplo de Include: Navbar Reutilizable

templates/includes/_navbar.html

```
<nav class="navbar">
  <div class="logo">
    <a href="{% url 'home' %}">
      {{ site_name | default:"Mi Sitio" }}
    </a>
  </div>

  <ul class="nav-links">
    <li><a href="{% url 'home' %}">Inicio</a></li>
    <li><a href="{% url 'blog:lista' %}">Blog</a></li>
    <li><a href="{% url 'about' %}">Acerca de</a></li>
    <li><a href="{% url 'contact' %}">Contacto</a></li>
  </ul>

  {% if user.is_authenticated %}
    <div class="user-menu">
      <span>Hola, {{ user.username }}</span>
      <a href="{% url 'logout' %}">Salir</a>
    </div>
  {% else %}
    <a href="{% url 'login' %}" class="btn">Iniciar Sesión</a>
  {% endif %}
</nav>
```

templates/base.html (usando include)

```
<!DOCTYPE html>
<html>
<head>
  <title>{% block titulo %}Mi Sitio{% endblock %}</title>
</head>
<body>
  {% include 'includes/_navbar.html' %}

  <main>
    {% block contenido %}
    {% endblock %}
  </main>

  {% include 'includes/_footer.html' %}
</body>
</html>
```

Ahora el navbar aparece automáticamente en todas las páginas que extiendan base.html. Si necesitas actualizarlo, solo modificas _navbar.html.

Archivos Estáticos: CSS, JavaScript e Imágenes

Los archivos estáticos son recursos que no cambian: hojas de estilo CSS, archivos JavaScript, imágenes, fuentes, etc. Django tiene un sistema específico para manejarlos de manera eficiente y segura.

01

Crear carpeta static/

En la raíz del proyecto o dentro de cada app, crea una carpeta llamada `static/`

03

Configurar settings.py

Verifica que `STATIC_URL = '/static/'` esté definido. Para producción, configura también `STATIC_ROOT`

02

Organizar por tipo

Dentro de `static/`, crea subcarpetas: `css/`, `js/`, `img/`, `fonts/`

04

Usar en templates

Carga etiqueta static: `{% load static %}` y úsala: `<link rel="stylesheet" href="{% static 'css/style.css' %}">`

Usando Archivos Estáticos en Templates

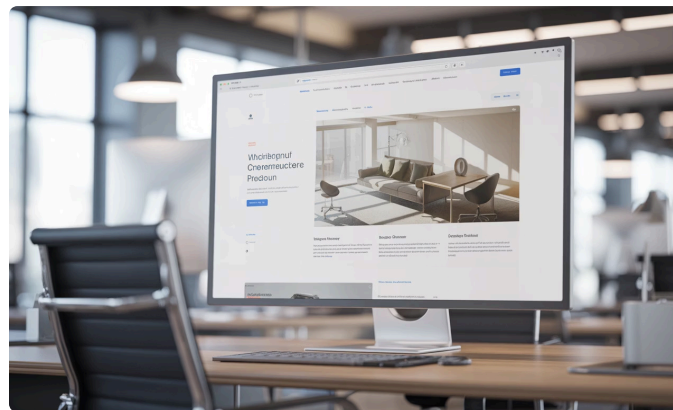
La etiqueta `{% static %}` genera automáticamente la URL correcta para tus archivos estáticos, funcionando tanto en desarrollo como en producción sin cambios de código.

```
{% load static %}

<!DOCTYPE html>
<html>
<head>
  <!-- CSS -->
  <link rel="stylesheet"
        href="{% static 'css/base.css' %}">
  <link rel="stylesheet"
        href="{% static 'css/blog.css' %}">

  <!-- Favicon -->
  <link rel="icon"
        href="{% static 'img/favicon.ico' %}">
</head>
<body>
  <!-- Imagen -->
  

  <!-- JavaScript -->
  <script src="{% static 'js/main.js' %}">
  </script>
</body>
</html>
```



📌 **Importante:** En producción, ejecuta `python manage.py collectstatic` para recopilar todos los archivos estáticos en una ubicación centralizada para el servidor web.

Context Processors: Variables Globales

Los context processors son funciones que añaden variables al contexto de todos tus templates automáticamente. Son perfectos para datos que necesitas en múltiples páginas como configuración del sitio, usuario actual, carrito de compras, etc.

Crear context processor

En tu app, crea `context_processors.py`:

```
def site_info(request):
    return {
        'site_name': 'Blog de Ivan Malaver',
        'site_version': '2.0',
        'year': 2024,
        'contact_email': 'ivan@example.com'
    }
```

Registrar en settings.py

```
TEMPLATES = [{
    'OPTIONS': {
        'context_processors': [
            # ... otros processors
            'myapp.context_processors.site_info',
        ],
    },
}]
```

Ahora todas las variables retornadas están disponibles en todos los templates sin pasarlas explícitamente en cada vista.

Custom Template Tags: Extendiendo Django

Django permite crear tus propias etiquetas de template para lógica personalizada. Esto es útil cuando necesitas funcionalidad que los tags built-in no proporcionan.

Crear estructura

En tu app, crea la estructura: `myapp/templatetags/_init_.py` y `myapp/templatetags/custom_tags.py`

Definir etiqueta

```
from django import template

register = template.Library()

@register.simple_tag
def multiply(a, b):
    return a * b

@register.filter
def highlight(text):
    return f"<mark>{text}</mark>"
```

Usar en templates

```
{% load custom_tags %}

<p>{% multiply 5 3 %}</p>
<p>{{ texto | highlight }}</p>
```



Formularios Django: Conexión con Templates

Los formularios son fundamentales en aplicaciones web. Django proporciona un sistema robusto de formularios que maneja validación, seguridad (CSRF) y renderizado automático.

forms.py

```
from django import forms

class ContactoForm(forms.Form):
    nombre = forms.CharField(
        max_length=100,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Tu nombre'
        })
    )
    email = forms.EmailField(
        widget=forms.EmailInput(attrs={
            'class': 'form-control'
        })
    )
    mensaje = forms.CharField(
        widget=forms.Textarea(attrs={
            'class': 'form-control',
            'rows': 5
        })
    )
```

views.py

```
from django.shortcuts import render, redirect
from .forms import ContactoForm

def contacto(request):
    if request.method == 'POST':
        form = ContactoForm(request.POST)
        if form.is_valid():
            # Procesar datos
            nombre = form.cleaned_data['nombre']
            email = form.cleaned_data['email']
            mensaje = form.cleaned_data['mensaje']
            # Enviar email, guardar en DB, etc.
            return redirect('gracias')
        else:
            form = ContactoForm()

    return render(request, 'contacto.html',
                  {'form': form})
```

Renderizando Formularios en Templates

Django ofrece múltiples formas de renderizar formularios en templates, desde muy automático hasta completamente manual para control total.

Renderizado Automático

```
<form method="post">
    {% csrf_token %}

    {%# Todo el formulario %#%
    {{ form.as_p }}

    <button type="submit">
        Enviar
    </button>
</form>
```

Renderiza todos los campos envueltos en `<p>`. También existe `as_table` y `as_ul`.

Semi-manual

```
<form method="post">
    {% csrf_token %}

    {% for field in form %}
        <div class="field">
            {{ field.label_tag }}
            {{ field }}
            {% if field.errors %}
                <span class="error">
                    {{ field.errors }}
                </span>
            {% endif %}
        </div>
    {% endfor %}

    <button type="submit">Enviar</button>
</form>
```

Completamente Manual

```
<form method="post">
    {% csrf_token %}

    <div>
        <label>Nombre:</label>
        {{ form.nombre }}
        {{ form.nombre.errors }}
    </div>

    <div>
        <label>Email:</label>
        {{ form.email }}
        {{ form.email.errors }}
    </div>

    <div>
        <label>Mensaje:</label>
        {{ form.mensaje }}
        {{ form.mensaje.errors }}
    </div>

    <button type="submit">Enviar</button>
</form>
```

Paginación: Dividiendo Contenido Grande

Cuando tienes muchos objetos para mostrar (artículos, productos, usuarios), la paginación es esencial para la usabilidad y el rendimiento. Django incluye un sistema de paginación muy fácil de usar.

En views.py

```
from django.core.paginator import Paginator

def lista_articulos(request):
    articulos_list = Artículo.objects.all()

    # Crear paginador (10 items por página)
    paginator = Paginator(articulos_list, 10)

    # Obtener número de página de URL
    page_number = request.GET.get('page')

    # Obtener objetos de esa página
    page_obj = paginator.get_page(page_number)

    return render(request, 'lista.html',
                  {'page_obj': page_obj})
```

En template

```
<!-- Mostrar objetos de página actual -->
{% for articulo in page_obj %}
  <div>{{ articulo.titulo }}</div>
{% endfor %}

<!-- Controles de paginación -->
<div class="pagination">
  {% if page_obj.has_previous %}
    <a href="?page=1">Primera</a>
    <a href="?page={{ page_obj.previous_page_number }}">
      Anterior
    </a>
  {% endif %}

  <span>
    Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}
  </span>

  {% if page_obj.has_next %}
    <a href="?page={{ page_obj.next_page_number }}">
      Siguiente
    </a>
    <a href="?page={{ page_obj.paginator.num_pages }}">
      Última
    </a>
  {% endif %}
</div>
```

Mensajes Flash: Feedback al Usuario

El framework de mensajes de Django permite mostrar notificaciones temporales al usuario después de realizar acciones: "Artículo guardado exitosamente", "Error al procesar el formulario", etc.

```
<!-- En views.py -->
from django.contrib import messages

def crear_articulo(request):
    if request.method == 'POST':
        form = ArtículoForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, 'Artículo creado exitosamente!')
            return redirect('lista_articulos')
        else:
            messages.error(request, 'Por favor corrige los errores.')
        else:
            form = ArtículoForm()
            return render(request, 'crear.html', {'form': form})

<!-- En base.html (disponible en todos los templates) -->
{% if messages %}
<div class="messages">
{% for message in messages %}
<div class="alert alert-{{ message.tags }}">
{{ message }}
</div>
{% endfor %}
</div>
{% endif %}
```


Optimización de Rendimiento de Templates

A medida que tu aplicación crece, optimizar el rendimiento de templates se vuelve crucial. Aquí algunas técnicas esenciales:



Minimiza Queries en Bucles

Usa `select_related()` y `prefetch_related()` en tus consultas para evitar el problema N+1. En lugar de hacer una query por cada objeto en un bucle, carga todos los datos relacionados de una vez.



Caching de Templates

Usa el sistema de caché de Django para fragmentos que no cambian frecuentemente: `{% load cache %} {% cache 500 sidebar %}...{% endcache %}`



Reduce Complejidad

Mueve lógica compleja de templates a vistas o métodos de modelo. Los templates deben ser principalmente presentacionales, no contener lógica de negocio pesada.



Comprime Estáticos

Usa paquetes como `django-compressor` para minificar y combinar archivos CSS y JavaScript, reduciendo el tamaño y número de requests HTTP.

Seguridad en Templates Django

Django incluye múltiples medidas de seguridad automáticas en sus templates, pero es importante entenderlas y usarlas correctamente.



Auto-escape HTML

Django automáticamente escapa caracteres peligrosos en variables (<, >, ', ", &) previniendo XSS. Si necesitas HTML sin escapar, usa `{{ variable|safe }}` con extrema precaución.



Protección CSRF

Siempre incluye `{% csrf_token %}` en formularios POST. Django rechazará solicitudes sin token válido, previniendo ataques Cross-Site Request Forgery.



SQL Injection

Nunca construyas queries SQL manualmente en templates. Usa el ORM de Django que automáticamente sanitiza y parametriza todas las queries.



No Expongas Secretos

Nunca pongas claves de API, passwords o SECRET_KEY en templates. Usa variables de entorno y manténlas fuera del código fuente.

Debugging de Templates

Cuando algo no funciona en un template, Django proporciona herramientas excelentes para diagnosticar el problema.

1

Modo DEBUG

En settings.py, `DEBUG = True` muestra páginas de error detalladas con información sobre variables de contexto, stack trace completo y línea exacta del error en el template.

2

Django Debug Toolbar

Instala `django-debug-toolbar` para ver queries SQL ejecutadas, tiempo de renderizado, variables de contexto disponibles, y más en una barra lateral conveniente.

3

Print Debugging

Usa `{{ variable | pprint }}` (después de `{% load debug %}`) para imprimir estructura completa de objetos complejos con formato legible.

4

Logs Personalizados

Configura logging en Django para escribir información de debug a archivo: útil cuando `DEBUG=False` en producción y necesitas investigar problemas sin exponer información sensible.

Internacionalización: Sitios Multiidioma

Django tiene soporte robusto para internacionalización (i18n) y localización (l10n), permitiendo que tu aplicación se adapte automáticamente al idioma y región del usuario.

Configuración settings.py

```
LANGUAGE_CODE = 'es'
USE_I18N = True
USE_L10N = True
USE_TZ = True

LANGUAGES = [
    ('es', 'Español'),
    ('en', 'English'),
    ('pt', 'Português'),
]

LOCALE_PATHS = [
    BASE_DIR / 'locale',
]
```

En templates

```
{% load i18n %}

<h1>{% trans "Bienvenido" %}</h1>

<p>
{% blocktrans with name=user.name %}
    Hola {{ name }}, tienes {{ count }} mensajes.
{% endblocktrans %}
</p>

<!-- Selector de idioma -->
<form action="{% url 'set_language' %}" method="post">
    {% csrf_token %}
    <select name="language">
        {% get_current_language as LANGUAGE_CODE %}
        {% get_available_languages as LANGUAGES %}
        {% for lang_code, lang_name in LANGUAGES %}
            <option value="{{ lang_code }}"
                {% if lang_code == LANGUAGE_CODE %}
                    selected
                {% endif %}>
                {{ lang_name }}
            </option>
        {% endfor %}
    </select>
    <button type="submit">Cambiar</button>
</form>
```

Testing de Templates

Escribir tests para tus templates asegura que renderizan correctamente y muestran los datos esperados. Django proporciona herramientas específicas para esto.

```
from django.test import TestCase, Client
from django.urls import reverse
from .models import Artículo

class TemplateTests(TestCase):
    def setUp(self):
        self.client = Client()
        self.articulo = Artículo.objects.create(
            titulo='Test',
            contenido='Contenido de prueba'
        )

    def test_lista_articulos_template(self):
        response = self.client.get(reverse('blog:lista'))

        # Verifica que usa el template correcto
        self.assertTemplateUsed(response, 'blog/lista.html')

        # Verifica código de estado
        self.assertEqual(response.status_code, 200)

        # Verifica que el artículo aparece
        self.assertContains(response, 'Test')
        self.assertContains(response, 'Contenido de prueba')

    def test_contexto_correcto(self):
        response = self.client.get(reverse('blog:lista'))

        # Verifica variables de contexto
        self.assertIn('articulos', response.context)
        self.assertEqual(len(response.context['articulos']), 1)
```

Deployment: Preparando Templates para Producción

Cuando despliegas tu aplicación Django a producción, hay configuraciones específicas que debes ajustar para seguridad y rendimiento.

1 **DEBUG = False**

Absolutamente esencial. En producción, DEBUG debe ser False para no exponer información sensible en páginas de error. Configura ALLOWED_HOSTS con tus dominios.

2 **Recolectar Estáticos**

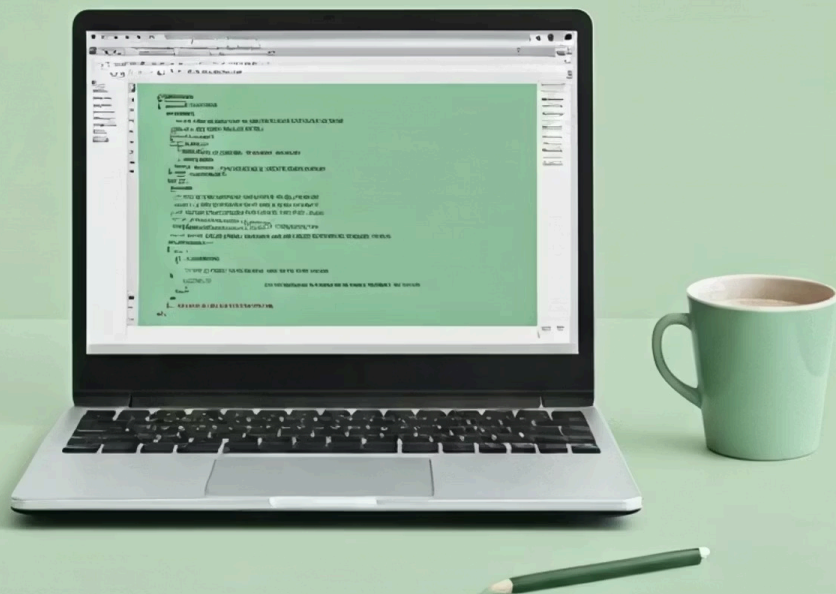
Ejecuta `python manage.py collectstatic` para copiar todos los archivos estáticos a `STATIC_ROOT`. Configura tu servidor web (nginx, Apache) para servir estos archivos directamente.

3 **Cacheo de Templates**

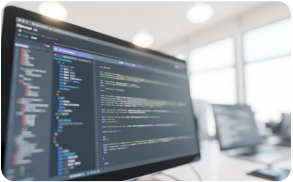
En producción, Django cachea templates compilados automáticamente. Para desarrollo, desactiva esto en settings: `TEMPLATES[0]['OPTIONS']['loaders']`

4 **CDN para Estáticos**

Considera usar un CDN (CloudFront, Cloudflare) para servir archivos estáticos más rápido globalmente. Actualiza `STATIC_URL` para apuntar al CDN.

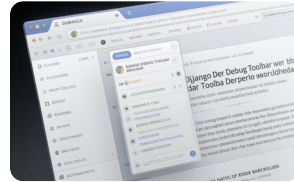


Herramientas y Paquetes Recomendados



django-crispy-forms

Renderiza formularios Django con Bootstrap, Foundation, Tailwind u otros frameworks CSS automáticamente. Elimina la necesidad de escribir HTML repetitivo para formularios.



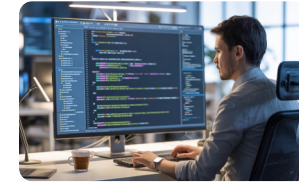
django-debug-toolbar

Panel lateral para debugging que muestra queries SQL, tiempo de renderizado, variables de contexto, caché hits/misses y más. Imprescindible durante desarrollo.



django-extensions

Colección de comandos útiles incluyendo `shell_plus`, `show_urls`, `runserver_plus` y otros que mejoran el workflow de desarrollo.



django-compressor

Comprime y combina archivos CSS y JavaScript automáticamente, reduciendo tamaño y número de requests. Esencial para optimización de rendimiento.

Patrones Avanzados: HTMX con Django

HTMX es una biblioteca JavaScript que permite crear interfaces interactivas usando atributos HTML simples, perfecto para combinar con templates Django sin escribir JavaScript complejo.

Instalación y Setup

```
<!-- En base.html -->
<script
src="https://unpkg.com/htmx.org@1.9.10">
</script>
```

Ejemplo: Carga Dinámica

```
<button
  hx-get="/api/articulos/"
  hx-target="#resultados"
  hx-swap="innerHTML">
  Cargar más
</button>

<div id="resultados"></div>
```

Vista Parcial Django

```
def articulos_parcial(request):
    articulos = Artículo.objects.all()[:10]

    # Renderiza solo el fragmento
    return render(
        request,
        'partials/articulos_list.html',
        {'articulos': articulos}
    )
```

Template Parcial

```
<!-- partials/articulos_list.html -->
{% for articulo in articulos %}
<div class="articulo-card">
<h3>{{ articulo.titulo }}</h3>
<p>{{ articulo.resumen }}</p>
</div>
{% endfor %}
```



Documentación y Recursos Adicionales

Documentación Oficial de Django

La fuente definitiva: <https://docs.djangoproject.com/es/4.2/topics/templates/> - Extremadamente completa con ejemplos, casos de uso y mejores prácticas. Disponible en español.

Django Girls Tutorial

Tutorial excelente para principiantes que cubre Django desde cero, incluyendo templates, con explicaciones claras y paso a paso. Gratuito y en múltiples idiomas.

Two Scoops of Django

Libro de mejores prácticas por Daniel y Audrey Roy Greenfeld. Cubre patrones avanzados, seguridad, testing y organización de proyectos Django profesionales.

Django Forum y Stack Overflow

Comunidades activas donde puedes hacer preguntas específicas. La etiqueta [django] en Stack Overflow tiene miles de respuestas a problemas comunes.

Proyecto de Práctica: Blog Completo

Para consolidar todo lo aprendido, te desafío a crear un blog completo con las siguientes características:

Funcionalidades Core

- Lista de artículos con paginación
- Vista de detalle de artículo
- Sistema de categorías y tags
- Búsqueda de artículos
- Comentarios (opcional)

Templates y Diseño

- Template base con header y footer
- Herencia para todas las páginas
- Sidebar con artículos recientes y categorías
- Responsive design con CSS
- Formulario de contacto

Características Avanzadas

- Sistema de autenticación de usuarios
- Panel de administración personalizado
- RSS feed de artículos
- Compartir en redes sociales
- Estadísticas de visualizaciones

Este proyecto te dará experiencia práctica con todos los conceptos cubiertos en esta guía y te preparará para proyectos Django reales.

Siguientes Pasos en Tu Viaje Django

Has completado esta guía exhaustiva sobre conexión de Django backend con templates. Aquí están tus próximos pasos recomendados para continuar mejorando:



Profundizar en el ORM

Aprende queries avanzadas, optimización de consultas, agregaciones, anotaciones y el sistema de signals de Django para manipular datos eficientemente.



Django REST Framework

Si quieres construir APIs modernas para aplicaciones móviles o SPAs con React/Vue, aprende DRF para crear endpoints RESTful profesionales.



Autenticación y Permisos

Domina el sistema de usuarios de Django, permisos personalizados, autenticación social (Google, Facebook), y control de acceso granular.



Deployment y DevOps

Aprende a desplegar tu aplicación en servicios como Heroku, AWS, DigitalOcean o Railway. Domina Docker, CI/CD, y monitoreo de aplicaciones en producción.

Conclusión: Tu Dominio de Django Templates

Felicitaciones por completar esta guía exhaustiva de 75 secciones sobre cómo conectar Django backend con templates. Has aprendido desde los conceptos más básicos hasta técnicas avanzadas de optimización y despliegue.

Recuerda que la práctica es esencial. Los conceptos que has visto aquí cobrarán vida cuando los implementes en proyectos reales. No tengas miedo de experimentar, cometer errores y buscar ayuda en la comunidad cuando lo necesites.

Django es un framework poderoso y maduro usado por compañías como Instagram, Pinterest, NASA y Mozilla. Con las habilidades que has adquirido en esta guía, estás en el camino correcto para construir aplicaciones web profesionales y escalables.

Creado por Ivan Malaver Fierro - Esta guía fue diseñada específicamente para desarrolladores que buscan dominar la conexión entre backend y frontend en Django con explicaciones claras, ejemplos prácticos y mejores prácticas de la industria.



📄 **Recursos finales:** Guarda esta guía como referencia, únete a comunidades Django en Discord/Reddit, contribuye a proyectos open source, y nunca dejes de aprender. ¡El futuro del desarrollo web está en tus manos!

Aplicación Django con Análisis de Datos: Guía Completa

Integrando Pandas, NumPy y Conexiones Multi-Base de Datos

Esta guía exhaustiva te llevará paso a paso para crear una aplicación Django profesional que integre librerías científicas como pandas, numpy, matplotlib, seaborn y plotly, con conexiones a PostgreSQL, MySQL y SQLite. Aprenderás desde la configuración inicial hasta el despliegue, con 100 ejercicios prácticos explicados línea por línea.

Preparación del Entorno: Instalación y Configuración Inicial

¿Qué vamos a construir?

Una aplicación Django llamada "DataAnalyzer" que permitirá:

- Cargar datasets desde archivos CSV/Excel
- Realizar análisis estadísticos con pandas y numpy
- Generar visualizaciones interactivas con matplotlib, seaborn y plotly
- Conectar con PostgreSQL, MySQL y SQLite
- Dashboard con métricas en tiempo real
- API REST para consumir datos

Requisitos del Sistema:

- Python 3.8 o superior
- pip (gestor de paquetes)
- Git (control de versiones)
- Editor de código (VS Code recomendado)

Verificación de Python:

```
python --version
# Debe mostrar: Python 3.8.x o superior
pip --version
# Debe mostrar la versión de pip
```

¿Por qué estas herramientas?

Python

Lenguaje base, excelente para análisis de datos

Django

Framework web robusto y escalable

pandas

Manipulación y análisis de datos estructurados

numpy

Computación numérica eficiente

matplotlib/seaborn

Visualización de datos estática

plotly

Gráficos interactivos para web

Paso 1: Creación del Entorno Virtual

¿Qué es un entorno virtual?

Un entorno virtual es un espacio aislado donde puedes instalar paquetes de Python sin afectar el sistema global. Es fundamental para evitar conflictos entre versiones de librerías.

Creación del entorno virtual:

```
# Crear carpeta del proyecto
mkdir DataAnalyzer
cd DataAnalyzer
```

```
# Crear entorno virtual
python -m venv venv
```

```
# Activar entorno virtual
# En Windows:
venv\Scripts\activate
# En macOS/Linux:
source venv/bin/activate
```

Explicación línea por línea:

- `mkdir DataAnalyzer`: Crea una carpeta llamada "DataAnalyzer"
- `cd DataAnalyzer`: Entra a la carpeta creada
- `python -m venv venv`: Ejecuta el módulo venv para crear un entorno virtual en la carpeta "venv"
- `venv\Scripts\activate`: Activa el entorno virtual (Windows)
- `source venv/bin/activate`: Activa el entorno virtual (macOS/Linux)

Verificación:

Cuando el entorno esté activo, verás `(venv)` al inicio de tu terminal:

```
(venv) C:\DataAnalyzer>
```

¿Por qué es importante?

- Aislamiento de dependencias
- Evita conflictos entre proyectos
- Facilita el despliegue
- Permite diferentes versiones de librerías por proyecto

Paso 2: Instalación de Dependencias - Librerías Científicas

Creación del archivo requirements.txt:

```
# Framework web
Django==4.2.7
djangoestframework==3.14.0

# Librerías científicas y análisis de datos
pandas==2.1.3
numpy==1.25.2
matplotlib==3.8.2
seaborn==0.13.0
plotly==5.17.0
scipy==1.11.4

# Bases de datos
psycopg2-binary==2.9.9 # PostgreSQL
PyMySQL==1.1.0      # MySQL
sqlite3              # SQLite (incluido en Python)

# Utilidades adicionales
openpyxl==3.1.2      # Leer archivos Excel
xlrd==2.0.1          # Leer archivos Excel antiguos
python-dotenv==1.0.0 # Variables de entorno
Pillow==10.1.0       # Procesamiento de imágenes
requests==2.31.0     # Peticiones HTTP

# Desarrollo y testing
pytest==7.4.3
pytest-django==4.7.0
```

Instalación de dependencias:

```
# Crear el archivo requirements.txt (copia el contenido de arriba)
# Luego instalar todas las dependencias:
pip install -r requirements.txt
```

Explicación de cada librería:

- **Django:** Framework web principal
- **djangoestframework:** Para crear APIs REST
- **pandas:** Manipulación de datos tabulares (DataFrames)
- **numpy:** Operaciones matemáticas y arrays multidimensionales
- **matplotlib:** Gráficos estáticos básicos
- **seaborn:** Gráficos estadísticos elegantes
- **plotly:** Gráficos interactivos para web
- **scipy:** Funciones científicas avanzadas
- **psycopg2-binary:** Conector para PostgreSQL
- **PyMySQL:** Conector para MySQL

Paso 3: Creación del Proyecto Django

Creación del proyecto principal:

```
# Crear proyecto Django
django-admin startproject dataanalyzer_project .

# Crear aplicación principal
python manage.py startapp analytics

# Crear aplicación para APIs
python manage.py startapp api

# Crear aplicación para visualizaciones
python manage.py startapp visualizations
```

Explicación línea por línea:

- `django-admin startproject dataanalyzer_project .`: Crea un proyecto Django en el directorio actual
- `python manage.py startapp analytics`: Crea una aplicación para lógica de análisis
- `python manage.py startapp api`: Crea una aplicación para endpoints REST
- `python manage.py startapp visualizations`: Crea una aplicación para gráficos

Estructura del proyecto resultante:

```
DataAnalyzer/
├── venv/
├── dataanalyzer_project/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── analytics/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── api/
├── visualizations/
├── manage.py
└── requirements.txt
```

¿Por qué múltiples aplicaciones?

Separación de responsabilidades Cada app tiene un propósito específico	Reutilización Las apps pueden reutilizarse en otros proyectos
Mantenimiento Código más organizado y fácil de mantener	Escalabilidad Permite que diferentes equipos trabajen en diferentes apps

Paso 4: Configuración de settings.py - Integración de Librerías

Modificación de settings.py:

```
import os
from pathlib import Path
from dotenv import load_dotenv

# Cargar variables de entorno
load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = os.getenv('SECRET_KEY', 'django-insecure-default-key')

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = ['localhost', '127.0.0.1']

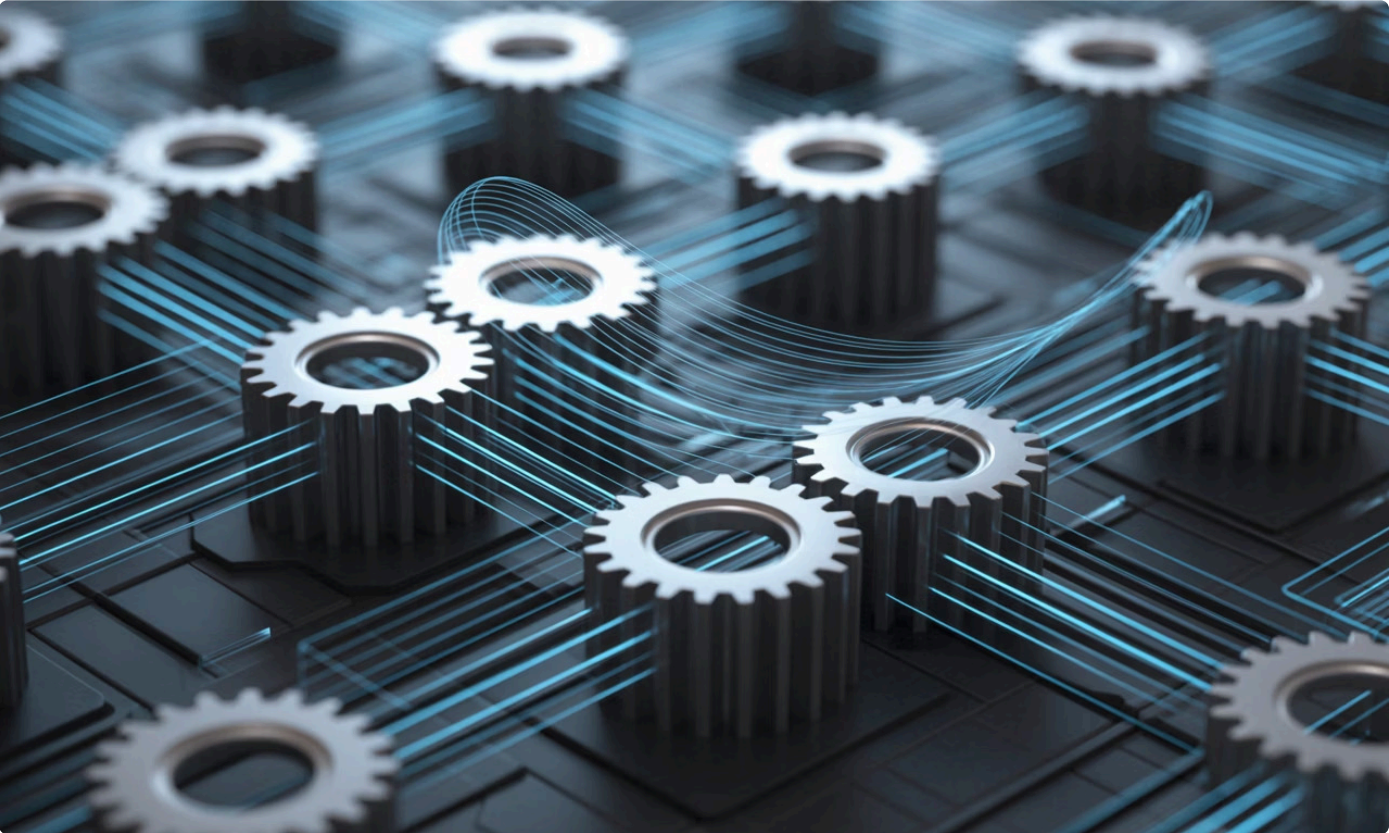
# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third party apps
    'rest_framework',

    # Local apps
    'analytics',
    'api',
    'visualizations',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'dataanalyzer_project.urls'
```



Explicación de cada sección:

- load_dotenv(): Carga variables de entorno desde archivo .env
- INSTALLED_APPS: Lista de aplicaciones instaladas en el proyecto
- rest_framework: Habilita Django REST Framework para APIs
- Nuestras apps locales: analytics, api, visualizations

Paso 5: Configuración de Bases de Datos - PostgreSQL, MySQL y SQLite

Configuración multi-base de datos en settings.py:

```
# Database configuration
DATABASES = {
    # Base de datos principal (SQLite para desarrollo)
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    },

    # PostgreSQL para análisis pesados
    'postgres': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('POSTGRES_DB', 'dataanalyzer_db'),
        'USER': os.getenv('POSTGRES_USER', 'postgres'),
        'PASSWORD': os.getenv('POSTGRES_PASSWORD', 'password'),
        'HOST': os.getenv('POSTGRES_HOST', 'localhost'),
        'PORT': os.getenv('POSTGRES_PORT', '5432'),
    },

    # MySQL para datos históricos
    'mysql': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.getenv('MYSQL_DB', 'dataanalyzer_mysql'),
        'USER': os.getenv('MYSQL_USER', 'root'),
        'PASSWORD': os.getenv('MYSQL_PASSWORD', 'password'),
        'HOST': os.getenv('MYSQL_HOST', 'localhost'),
        'PORT': os.getenv('MYSQL_PORT', '3306'),
        'OPTIONS': {
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'",
        },
    },
}

# Router para dirigir modelos a bases de datos específicas
DATABASE_ROUTERS = ['dataanalyzer_project.routers.DatabaseRouter']
```

Creación del archivo .env:

```
# Archivo .env (crear en la raíz del proyecto)
SECRET_KEY=tu-clave-secreta-super-segura-aqui

# PostgreSQL
POSTGRES_DB=dataanalyzer_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=tu_password_postgres
POSTGRES_HOST=localhost
POSTGRES_PORT=5432

# MySQL
MYSQL_DB=dataanalyzer_mysql
MYSQL_USER=root
MYSQL_PASSWORD=tu_password_mysql
MYSQL_HOST=localhost
MYSQL_PORT=3306
```



SQLite (Default) Base de datos principal (SQLite para desarrollo)	PostgreSQL Utilizada para análisis pesados y grandes volúmenes de datos.
MySQL Diseñada para almacenar datos históricos y de archivo.	Archivo .env Almacena de forma segura las credenciales y configuraciones de las bases de datos.

Creación del Database Router - Enrutamiento Inteligente

Crear archivo `dataanalyzer_project/routers.py`:

```
class DatabaseRouter:
    """
    Router para dirigir modelos a bases de datos específicas
    """

    def db_for_read(self, model, **hints):
        """Sugiere la base de datos para leer objetos del modelo"""
        if model._meta.app_label == 'analytics':
            # Modelos de analytics van a PostgreSQL
            return 'postgres'
        elif model._meta.app_label == 'api':
            # Modelos de API van a MySQL
            return 'mysql'
        return 'default' # SQLite por defecto

    def db_for_write(self, model, **hints):
        """Sugiere la base de datos para escribir objetos del modelo"""
        if model._meta.app_label == 'analytics':
            return 'postgres'
        elif model._meta.app_label == 'api':
            return 'mysql'
        return 'default'

    def allow_relation(self, obj1, obj2, **hints):
        """Permite relaciones si los modelos están en la misma base de datos"""
        db_set = {'default', 'postgres', 'mysql'}
        if obj1._state.db in db_set and obj2._state.db in db_set:
            return True
        return None

    def allow_migrate(self, db, app_label, model_name=None, **hints):
        """Asegura que ciertos modelos solo se migren a bases específicas"""
        if app_label == 'analytics':
            return db == 'postgres'
        elif app_label == 'api':
            return db == 'mysql'
        elif db in ['postgres', 'mysql']:
            return False # No migrar otros modelos a estas bases
        return db == 'default'
```

¿Qué hace cada método?

db_for_read()

Decide qué base de datos usar para leer datos

db_for_write()

Decide qué base de datos usar para escribir datos

allow_relation()

Controla relaciones entre modelos de diferentes bases

allow_migrate()

Controla qué migraciones van a qué base de datos

Modelos de Datos - Integración con Pandas y NumPy

La aplicación analytics requiere un modelo para gestionar los datasets que los usuarios subirán. Este modelo no solo almacenará los metadatos del archivo, sino que también integrará funcionalidades para cargar los datos en estructuras de Pandas y NumPy para su procesamiento.

Crear analytics/models.py:

```
from django.db import models
from django.contrib.auth.models import User
import pandas as pd
import numpy as np
from django.core.files.storage import default_storage
import json

class Dataset(models.Model):
    """Modelo para almacenar información de datasets"""
    name = models.CharField(max_length=200, verbose_name="Nombre del Dataset")
    description = models.TextField(blank=True, verbose_name="Descripción")
    file = models.FileField(upload_to='datasets/', verbose_name="Archivo")
    uploaded_by = models.ForeignKey(User, on_delete=models.CASCADE)
    uploaded_at = models.DateTimeField(auto_now_add=True)
    file_type = models.CharField(max_length=10, choices=[
        ('csv', 'CSV'),
        ('xlsx', 'Excel'),
        ('json', 'JSON'),
    ])

    rows_count = models.IntegerField(null=True, blank=True)
    columns_count = models.IntegerField(null=True, blank=True)

    class Meta:
        verbose_name = "Dataset"
        verbose_name_plural = "Datasets"
        ordering = ['-uploaded_at']

    def __str__(self):
        return self.name

    def load_as_dataframe(self):
        """Carga el archivo como DataFrame de pandas"""
        file_path = self.file.path

        if self.file_type == 'csv':
            df = pd.read_csv(file_path)
        elif self.file_type == 'xlsx':
            df = pd.read_excel(file_path)
        elif self.file_type == 'json':
            df = pd.read_json(file_path)
        else:
            raise ValueError(f"Tipo de archivo no soportado: {self.file_type}")

        return df

    def get_basic_stats(self):
        """Obtiene estadísticas básicas usando pandas"""
        df = self.load_as_dataframe()

        # Estadísticas básicas
        stats = {
            'shape': df.shape,
            'columns': list(df.columns),
            'dtypes': df.dtypes.to_dict(),
            'null_counts': df.isnull().sum().to_dict(),
            'memory_usage': df.memory_usage(deep=True).sum(),
        }

        # Estadísticas numéricas
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        if len(numeric_cols) > 0:
            stats['numeric_stats'] = df[numeric_cols].describe().to_dict()

        return stats
```



Este modelo Dataset es fundamental para la gestión y análisis de datos en nuestra aplicación, ofreciendo una interfaz sencilla para cargar y procesar archivos.



Modelo Dataset

Almacena metadatos esenciales como el nombre, descripción, tipo de archivo y detalles de carga del dataset, junto con el archivo en sí.



load_as_dataframe()

Carga dinámicamente el archivo del dataset (CSV, Excel o JSON) en un DataFrame de Pandas, facilitando su manipulación y análisis.



get_basic_stats()

Genera estadísticas descriptivas del DataFrame, incluyendo forma, columnas, tipos de datos, recuentos de nulos y estadísticas numéricas, utilizando las capacidades de Pandas.

Modelo de Análisis - Almacenando Resultados Científicos

Continuar en analytics/models.py:

```
class Analysis(models.Model):
    """Modelo para almacenar análisis realizados sobre datasets"""
    ANALYSIS_TYPES = [
        ('descriptive', 'Análisis Descriptivo'),
        ('correlation', 'Análisis de Correlación'),
        ('regression', 'Regresión Lineal'),
        ('clustering', 'Clustering'),
        ('time_series', 'Series de Tiempo'),
        ('custom', 'Análisis Personalizado'),
    ]

    dataset = models.ForeignKey(Dataset, on_delete=models.CASCADE, related_name='analyses')
    name = models.CharField(max_length=200, verbose_name="Nombre del Análisis")
    analysis_type = models.CharField(max_length=20, choices=ANALYSIS_TYPES)
    parameters = models.JSONField(default=dict, verbose_name="Parámetros del Análisis")
    results = models.JSONField(default=dict, verbose_name="Resultado del Análisis")
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    execution_time = models.FloatField(null=True, blank=True, verbose_name="Tiempo de Ejecución (segundos)")

    class Meta:
        verbose_name = "Análisis"
        verbose_name_plural = "Análisis"
        ordering = ['-created_at']

    def __str__(self):
        return f"{self.name} - {self.get_analysis_type_display()}"

    def perform_descriptive_analysis(self):
        """Realiza análisis descriptivo usando pandas y numpy"""
        import time
        start_time = time.time()

        df = self.dataset.load_as_dataframe()

        # Análisis descriptivo completo
        results = {
            'basic_info': {
                'shape': df.shape,
                'columns': list(df.columns),
                'dtypes': df.dtypes.astype(str).to_dict(),
            },
            'missing_data': {
                'total_missing': int(df.isnull().sum().sum()),
                'missing_by_column': df.isnull().sum().to_dict(),
                'missing_percentage': (df.isnull().sum() / len(df) * 100).to_dict(),
            },
            'numeric_analysis': {},
            'categorical_analysis': {},
        }

        # Análisis de columnas numéricas
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        if len(numeric_cols) > 0:
            numeric_stats = df[numeric_cols].describe()
            results['numeric_analysis'] = {
                'statistics': numeric_stats.to_dict(),
                'skewness': df[numeric_cols].skew().to_dict(),
                'kurtosis': df[numeric_cols].kurtosis().to_dict(),
            }

        # Análisis de columnas categóricas
        categorical_cols = df.select_dtypes(include=['object', 'category']).columns
        if len(categorical_cols) > 0:
            cat_analysis = {}
            for col in categorical_cols:
                cat_analysis[col] = {
                    'unique_count': int(df[col].nunique()),
                    'most_frequent': df[col].mode().iloc[0] if not df[col].mode().empty else None,
                    'value_counts': df[col].value_counts().head(10).to_dict(),
                }
            results['categorical_analysis'] = cat_analysis

        # Guardar resultados y tiempo de ejecución
        self.results = results
        self.execution_time = time.time() - start_time
        self.save()

    return results
```

Este nuevo modelo `Analysis` está diseñado para almacenar los metadatos y resultados de los diversos análisis que se realizarán sobre los `Datasets`. Esto permite una trazabilidad y reusabilidad de los hallazgos científicos.

Campos Clave del Modelo `Analysis`

- `dataset`: Referencia al `Dataset` sobre el cual se realiza el análisis.
- `name`: Un nombre descriptivo para el análisis.
- `analysis_type`: Clasifica el tipo de análisis (e.g., Descriptivo, Correlación).
- `parameters`: Almacena los parámetros específicos utilizados para ejecutar el análisis, en formato JSON.
- `results`: Contiene los resultados del análisis, también en formato JSON para flexibilidad.
- `created_by`: Usuario que inició el análisis.
- `execution_time`: Duración del análisis en segundos.

El método `perform_descriptive_analysis()` es un ejemplo de cómo se integran las bibliotecas `Pandas` y `NumPy` para calcular y almacenar estadísticas detalladas:

- **Carga de Datos**

Utiliza `self.dataset.load_as_dataframe()` para cargar el dataset en un `DataFrame` de `Pandas`.
- **Información Básica**

Recopila la forma (filas, columnas), nombres de columnas y tipos de datos de las columnas.
- **Datos Faltantes**

Calcula el total de valores nulos, valores nulos por columna y el porcentaje de valores faltantes.
- **Análisis Numérico**

Para columnas numéricas, obtiene estadísticas descriptivas (media, desviación estándar, etc.), asimetría (*skewness*) y curtosis.
- **Análisis Categórico**

Para columnas categóricas, calcula el recuento de valores únicos, el valor más frecuente y los recuentos de frecuencia de los 10 valores principales.
- **Almacenamiento**

Guarda los resultados y el tiempo de ejecución en los campos `results` y `execution_time` del modelo `Analysis`.



Vistas de Analytics - Procesamiento con Pandas

Crear analytics/views.py:

```
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.http import JsonResponse, HttpResponse
from django.views.decorators.csrf import csrf_exempt

import pandas as pd
import numpy as np
import json
import io
import base64
import matplotlib
matplotlib.use('Agg') # Backend no interactivo para servidor
import matplotlib.pyplot as plt
import seaborn as sns
from .models import Dataset, Analysis
from .forms import DatasetUploadForm

@login_required
def dashboard(request):
    """Vista principal del dashboard de analytics"""
    datasets = Dataset.objects.filter(uploaded_by=request.user)
    recent_analyses = Analysis.objects.filter(created_by=request.user)[:5]

    # Estadísticas generales
    stats = {
        'total_datasets': datasets.count(),
        'total_analyses': Analysis.objects.filter(created_by=request.user).count(),
        'total_rows': sum([d.rows_count or 0 for d in datasets]),
    }

    context = {
        'datasets': datasets,
        'recent_analyses': recent_analyses,
        'stats': stats,
    }
    return render(request, 'analytics/dashboard.html', context)

@login_required
def upload_dataset(request):
    """Vista para subir nuevos datasets"""
    if request.method == 'POST':
        form = DatasetUploadForm(request.POST, request.FILES)
        if form.is_valid():
            dataset = form.save(commit=False)
            dataset.uploaded_by = request.user

            # Detectar tipo de archivo
            file_name = dataset.file.name.lower()
            if file_name.endswith('.csv'):
                dataset.file_type = 'csv'
            elif file_name.endswith(('.xlsx', '.xls')):
                dataset.file_type = 'xlsx'
            elif file_name.endswith('.json'):
                dataset.file_type = 'json'

            dataset.save()

            # Procesar archivo para obtener estadísticas básicas
            try:
                df = dataset.load_as_dataframe()
                dataset.rows_count = len(df)
                dataset.columns_count = len(df.columns)
                dataset.save()

                messages.success(request, f'Dataset "{dataset.name}" subido exitosamente!')
                return redirect('analytics:dataset_detail', pk=dataset.pk)
            except Exception as e:
                messages.error(request, f'Error procesando el archivo: {str(e)}')
                dataset.delete() # Eliminar si hay error
            else:
                form = DatasetUploadForm()

    return render(request, 'analytics/upload_dataset.html', {'form': form})
```

Este archivo views.py define las funciones principales para interactuar con los datos y análisis en la plataforma. Utiliza Django para la lógica de la vista y se integra con Pandas y NumPy para el procesamiento de datos.

Dashboard de Analytics

La vista dashboard proporciona una visión general al usuario, mostrando:

- Todos los Datasets subidos por el usuario.
- Los 5 análisis más recientes realizados.
- Estadísticas generales como el número total de datasets, análisis y filas.

Subida de Datasets

La función upload_dataset maneja el proceso de carga de nuevos archivos de datos:

- Validación y guardado:** Procesa el formulario de subida, guarda el dataset y asigna el usuario que lo subió.
- Detección de tipo de archivo:** Identifica automáticamente si el archivo es CSV, XLSX o JSON basándose en su extensión.
- Procesamiento inicial con Pandas:** Carga el archivo en un DataFrame de Pandas para contar filas y columnas, almacenando esta información en el modelo Dataset para un acceso rápido y una descripción inicial.
- Manejo de errores:** Si ocurre un error durante el procesamiento del archivo (ej. archivo corrupto), se notifica al usuario y el dataset se elimina.

Estas vistas son fundamentales para la interacción del usuario con la plataforma de análisis, permitiendo tanto la visualización de un resumen como la incorporación de nuevos datos para su posterior estudio.



Vista de Análisis Detallado - Integración Completa con Librerías

Continuar en analytics/views.py:

```
@login_required
def dataset_detail(request, pk):
    """Vista detallada de un dataset con análisis básico"""
    dataset = get_object_or_404(Dataset, pk=pk, uploaded_by=request.user)

    # Cargar datos con pandas
    try:
        df = dataset.load_as_dataframe()

        # Información básica del dataset
        basic_info = {
            'shape': df.shape,
            'columns': list(df.columns),
            'dtypes': df.dtypes.astype(str).to_dict(),
            'memory_usage': f'{df.memory_usage(deep=True).sum() / 1024 / 1024:.2f} MB',
        }

        # Estadísticas descriptivas para columnas numéricas
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        numeric_stats = None
        if len(numeric_cols) > 0:
            numeric_stats = df[numeric_cols].describe().round(2).to_dict()

        # Información de valores faltantes
        missing_info = {
            'total_missing': int(df.isnull().sum().sum()),
            'missing_by_column': df.isnull().sum().to_dict(),
            'missing_percentage': (df.isnull().sum() / len(df) * 100).round(2).to_dict(),
        }

        # Muestra de los primeros 10 registros
        sample_data = df.head(10).to_dict('records')

        # Análisis de correlación si hay columnas numéricas
        correlation_data = None
        if len(numeric_cols) > 1:
            correlation_matrix = df[numeric_cols].corr().round(3)
            correlation_data = correlation_matrix.to_dict()

        context = {
            'dataset': dataset,
            'basic_info': basic_info,
            'numeric_stats': numeric_stats,
            'missing_info': missing_info,
            'sample_data': sample_data,
            'correlation_data': correlation_data,
            'analyses': dataset.analyses.all(),
        }

    except Exception as e:
        messages.error(request, f'Error cargando el dataset: {str(e)}')
        context = {'dataset': dataset, 'error': str(e)}

    return render(request, 'analytics/dataset_detail.html', context)

@login_required
def perform_analysis(request, dataset_pk):
    """Vista para realizar diferentes tipos de análisis"""
    dataset = get_object_or_404(Dataset, pk=dataset_pk, uploaded_by=request.user)

    if request.method == 'POST':
        analysis_type = request.POST.get('analysis_type')
        analysis_name = request.POST.get('analysis_name', f'Análisis {analysis_type}')

        # Crear objeto Analysis
        analysis = Analysis.objects.create(
            dataset=dataset,
            name=analysis_name,
            analysis_type=analysis_type,
            created_by=request.user,
        )

        try:
            if analysis_type == 'descriptive':
                results = analysis.perform_descriptive_analysis()
            elif analysis_type == 'correlation':
                results = perform_correlation_analysis(analysis)
            elif analysis_type == 'regression':
                results = perform_regression_analysis(analysis, request.POST)
            else:
                raise ValueError(f'Tipo de análisis no soportado: {analysis_type}')

            messages.success(request, f'Análisis "{analysis_name}" completado exitosamente!')
            return redirect('analytics:analysis_detail', pk=analysis.pk)

        except Exception as e:
            analysis.delete() # Eliminar análisis fallido
            messages.error(request, f'Error realizando análisis: {str(e)}')

    return render(request, 'analytics/perform_analysis.html', {'dataset': dataset})
```

Vista de Detalle del Dataset

La función `dataset_detail` proporciona una vista exhaustiva de un dataset cargado, integrando la potencia de Pandas para el análisis:

- **Carga de datos:** Utiliza `dataset.load_as_dataframe()` para cargar los datos en un DataFrame de Pandas.
- **Información básica:** Recopila la forma (filas, columnas), nombres de columnas, tipos de datos y el uso de memoria del DataFrame.
- **Estadísticas descriptivas:** Calcula estadísticas (media, desviación estándar, etc.) para las columnas numéricas.
- **Valores faltantes:** Informa el total de valores nulos, la distribución por columna y su porcentaje.
- **Muestra de datos:** Muestra los primeros 10 registros para una inspección rápida.
- **Análisis de correlación:** Genera una matriz de correlación para las columnas numéricas, si hay más de una.
- **Manejo de errores:** Incluye un bloque `try-except` para gestionar posibles errores durante la carga o procesamiento del dataset.

Realización de Análisis

La función `perform_analysis` permite a los usuarios ejecutar diferentes tipos de análisis sobre sus datasets:

- **Selección de tipo:** Permite al usuario elegir el tipo de análisis (descriptivo, correlación, regresión) y el nombre.
- **Creación del objeto Analysis:** Instancia un nuevo objeto Analysis en la base de datos para registrar el análisis.
- **Despacho de análisis:** Dirige la ejecución a funciones específicas (`perform_descriptive_analysis`, `perform_correlation_analysis`, `perform_regression_analysis`) basadas en el tipo seleccionado.
- **Resultados y redirección:** Tras completar el análisis, guarda los resultados y redirige al usuario a la vista de detalle del análisis.
- **Gestión de errores:** Si el análisis falla, el objeto Analysis se elimina y se muestra un mensaje de error.

Funciones de Análisis Avanzado - Correlación y Regresión

A continuación, se detallan las funciones auxiliares implementadas en analytics/views.py para realizar análisis de correlación y regresión, fundamentales en el procesamiento de datos.

Continuar en analytics/views.py - Funciones auxiliares:

```
def perform_correlation_analysis(analysis):
    """Realiza análisis de correlación usando pandas y numpy"""
    import time
    start_time = time.time()

    df = analysis.dataset.load_as_dataframe()

    # Seleccionar solo columnas numéricas
    numeric_cols = df.select_dtypes(include=[np.number]).columns

    if len(numeric_cols) < 2:
        raise ValueError("Se necesitan al menos 2 columnas numéricas para análisis de correlación")

    # Calcular matriz de correlación
    correlation_matrix = df[numeric_cols].corr()

    # Encontrar correlaciones más fuertes
    correlation_pairs = []
    for i in range(len(correlation_matrix.columns)):
        for j in range(i+1, len(correlation_matrix.columns)):
            col1 = correlation_matrix.columns[i]
            col2 = correlation_matrix.columns[j]
            corr_value = correlation_matrix.iloc[i, j]

            correlation_pairs.append({
                'variable1': col1,
                'variable2': col2,
                'correlation': round(corr_value, 4),
                'strength': get_correlation_strength(abs(corr_value)),
            })

    # Ordenar por correlación absoluta
    correlation_pairs.sort(key=lambda x: abs(x['correlation']), reverse=True)

    results = {
        'correlation_matrix': correlation_matrix.round(4).to_dict(),
        'correlation_pairs': correlation_pairs[:10], # Top 10
        'numeric_columns': list(numeric_cols),
        'total_pairs': len(correlation_pairs),
    }

    # Guardar resultados
    analysis.results = results
    analysis.execution_time = time.time() - start_time
    analysis.save()

    return results
```

La función `perform_correlation_analysis` se encarga de calcular la matriz de correlación entre las columnas numéricas de un dataset. Sus características principales incluyen:

- Carga del DataFrame:** Utiliza `analysis.dataset.load_as_dataframe()` para obtener los datos.
- Filtrado de columnas:** Selecciona únicamente las columnas con tipos de datos numéricos.
- Validación:** Asegura que existan al menos dos columnas numéricas para poder realizar el análisis.
- Cálculo de correlación:** Calcula la matriz de correlación usando el método `.corr()` de Pandas.
- Identificación de pares:** Extrae los pares de variables con sus respectivos valores de correlación y clasifica su fuerza (usando `get_correlation_strength`).
- Resultados:** Retorna la matriz de correlación y el top 10 de pares con mayor correlación absoluta, junto con la lista de columnas numéricas.
- Persistencia:** Guarda los resultados y el tiempo de ejecución en el objeto `analysis`.

```
def perform_regression_analysis(analysis, post_data):
    """Realiza análisis de regresión lineal usando scipy y numpy"""
    from scipy import stats
    import time

    start_time = time.time()
    df = analysis.dataset.load_as_dataframe()

    # Obtener variables del formulario
    target_variable = post_data.get('target_variable')
    predictor_variables = post_data.getlist('predictor_variables')

    if not target_variable or not predictor_variables:
        raise ValueError("Debe seleccionar variable objetivo y al menos una variable predictora")

    # Verificar que las variables existen y son numéricas
    all_vars = [target_variable] + predictor_variables
    numeric_cols = df.select_dtypes(include=[np.number]).columns

    for var in all_vars:
        if var not in numeric_cols:
            raise ValueError(f"La variable '{var}' no es numérica")

    # Preparar datos (eliminar valores nulos)
    analysis_df = df[all_vars].dropna()

    if len(analysis_df) == 0:
        raise ValueError("No hay datos válidos después de eliminar valores nulos")

    y = analysis_df[target_variable]
    results = {
        'target_variable': target_variable,
        'predictor_variables': predictor_variables,
        'sample_size': len(analysis_df),
        'regressions': {},
    }

    # Realizar regresión para cada variable predictora
    for predictor in predictor_variables:
        x = analysis_df[predictor]

        # Regresión lineal
        slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

        # Predicciones
        y_pred = slope * x + intercept

        # Métricas adicionales
        mse = np.mean((y - y_pred) ** 2)
        rmse = np.sqrt(mse)
        mae = np.mean(np.abs(y - y_pred))

        results['regressions'][predictor] = {
            'slope': round(slope, 6),
            'intercept': round(intercept, 6),
            'r_squared': round(r_value ** 2, 6),
            'p_value': round(p_value, 6),
            'std_error': round(std_err, 6),
            'mse': round(mse, 6),
            'rmse': round(rmse, 6),
            'mae': round(mae, 6),
            'equation': f"y = {slope:.4f}x + {intercept:.4f}",
        }

    # Guardar resultados
    analysis.results = results
    analysis.execution_time = time.time() - start_time
    analysis.save()

    return results
```

La función `perform_regression_analysis` implementa el análisis de regresión lineal para predecir una variable objetivo a partir de una o varias variables predictoras. Sus pasos clave son:

- Carga del DataFrame:** Obtiene el dataset como `DataFrame`.
- Extracción de variables:** Recupera la variable objetivo y las predictoras de los datos POST del formulario.
- Validación de entrada:** Verifica que se hayan seleccionado variables y que todas sean numéricas.
- Preparación de datos:** Crea un sub-`DataFrame` con las variables seleccionadas y elimina las filas con valores nulos para asegurar la integridad del análisis.
- Cálculo de regresión:** Para cada variable predictora, realiza una regresión lineal simple utilizando `scipy.stats.linregress`.
- Cálculo de métricas:** Determina la pendiente, el intercepto, el valor R^2 , el p-valor, el error estándar, el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el Error Absoluto Medio (MAE).
- Resultados:** Almacena los coeficientes de regresión y las métricas de evaluación.
- Persistencia:** Guarda los resultados y el tiempo de ejecución en el objeto `analysis`.

```
def get_correlation_strength(correlation_value):
    """Clasifica la fuerza de correlación"""
    abs_corr = abs(correlation_value)
    if abs_corr >= 0.8:
        return "Muy fuerte"
    elif abs_corr >= 0.6:
        return "Fuerte"
    elif abs_corr >= 0.4:
        return "Moderada"
    elif abs_corr >= 0.2:
        return "Débil"
    else:
        return "Muy débil"
```

La función `get_correlation_strength` es una utilidad que clasifica la fuerza de una correlación basada en su valor absoluto:

- "Muy fuerte"**: para valores absolutos de correlación mayores o iguales a 0.8.
- "Fuerte"**: para valores absolutos de correlación entre 0.6 y 0.79.
- "Moderada"**: para valores absolutos de correlación entre 0.4 y 0.59.
- "Débil"**: para valores absolutos de correlación entre 0.2 y 0.39.
- "Muy débil"**: para valores absolutos de correlación menores a 0.2.

1	2
Análisis de Correlación Identifica la relación estadística entre dos o más variables numéricas, revelando la dirección y fuerza de su asociación. - Requiere al menos 2 columnas numéricas. - Calcula matriz de correlación. - Clasifica la fuerza de la correlación.	Análisis de Regresión Lineal Modela la relación entre una variable dependiente y una o más variables independientes, permitiendo la predicción y el entendimiento de su impacto. - Define variable objetivo y predictoras. - Elimina valores nulos para consistencia. - Genera métricas como R^2, p-valor, MSE, RMSE, MAE.

Sistema de Notificaciones y Reportes Automatizados: Guía Completa

WhatsApp, PDFs Encriptados y Correos Automáticos con Django

Esta guía exhaustiva te enseñará a crear un sistema completo de notificaciones automatizadas que incluye:

- Envío de mensajes por WhatsApp usando Twilio
- Generación automática de PDFs con información del cliente
- Encriptación de PDFs con contraseña (número de identificación)
- Envío automático de correos electrónicos con adjuntos
- Estados de cuenta y extractos de productos personalizados
- Sistema de tareas asíncronas con Celery
- Dashboard de seguimiento de notificaciones

Casos de uso reales:

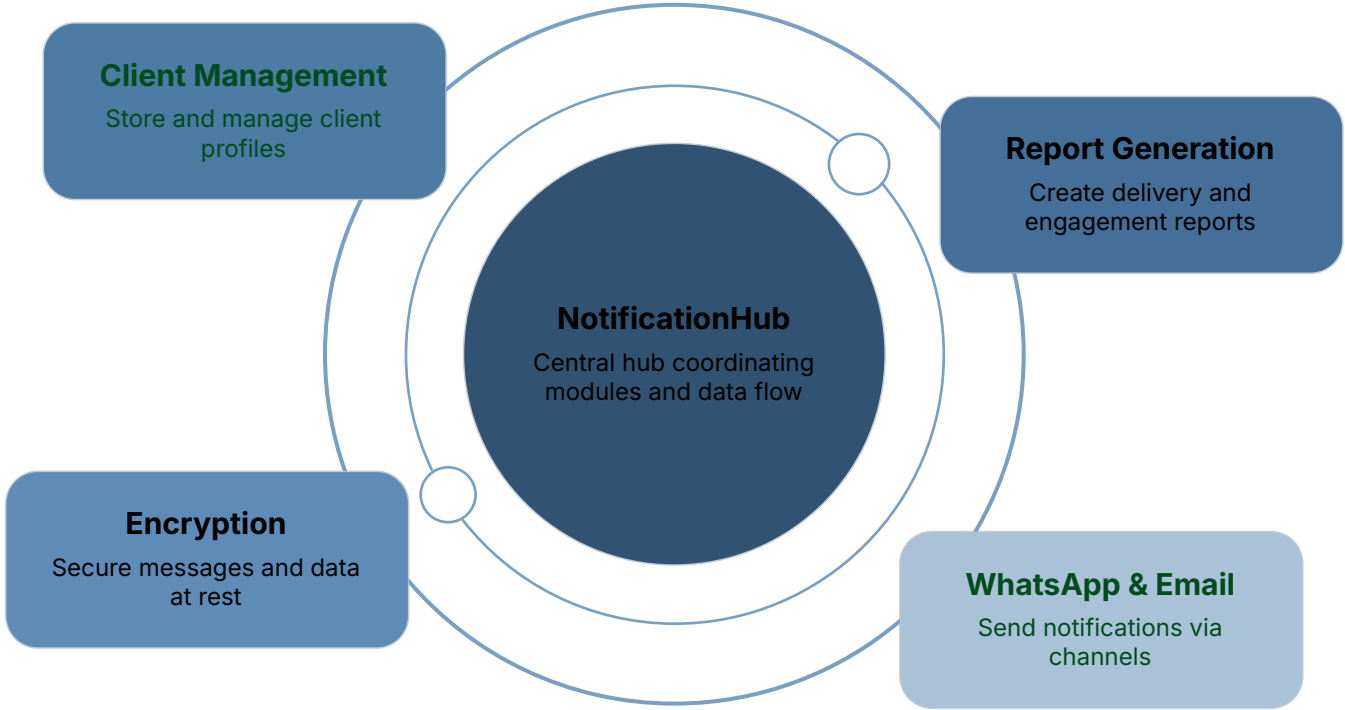
- Bancos enviando extractos bancarios
- Empresas de servicios con facturas mensuales
- Clínicas con resultados médicos
- E-commerce con reportes de compras



Arquitectura del Sistema: Componentes y Flujo de Trabajo

¿Qué vamos a construir?

Un sistema integral llamado "NotificationHub" que automatiza completamente el proceso de comunicación con clientes:



Componentes principales:



Módulo de Clientes
Gestión de usuarios con datos personales y de contacto



Generador de Reportes
Creación automática de PDFs personalizados



Sistema de Encriptación
Protección de documentos con contraseñas



Notificador WhatsApp
Integración con Twilio para mensajería



Sistema de Email
Envío automático con adjuntos encriptados



Cola de Tareas
Procesamiento asíncrono con Celery y Redis



Dashboard de Monitoreo
Seguimiento de entregas y estados

Flujo de trabajo típico:

- Sistema detecta evento (fin de mes, nueva transacción, etc.)
- Genera PDF personalizado con datos del cliente
- Encripta PDF con número de identificación del cliente
- Envía notificación por WhatsApp informando sobre el documento
- Envía email con PDF adjunto encriptado
- Registra el envío en base de datos
- Actualiza dashboard con estadísticas

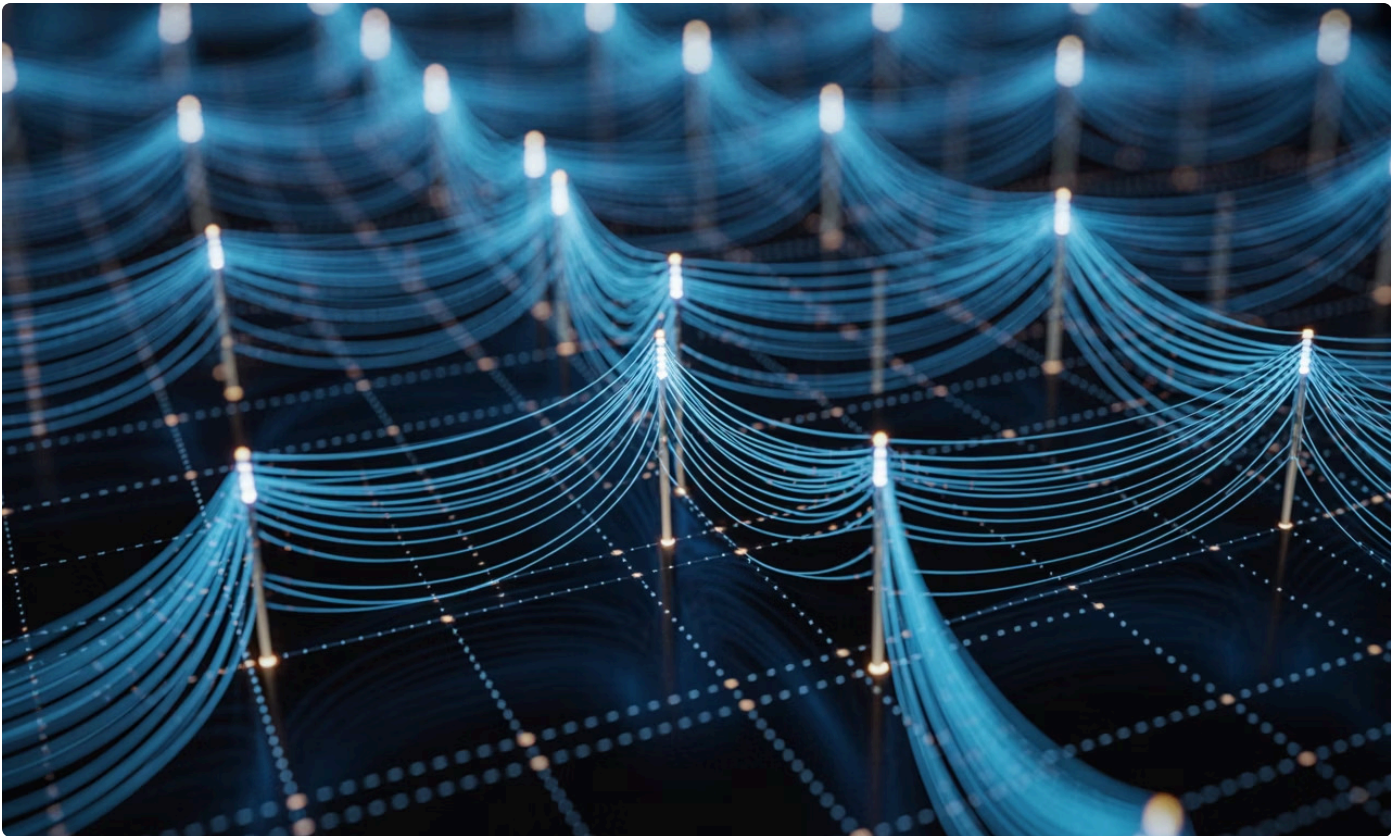
Este flujo describe la secuencia de acciones que el sistema NotificationHub realiza automáticamente desde la detección de un evento hasta la actualización del dashboard, asegurando una comunicación eficiente y segura con el cliente.

Tecnologías que integraremos:

- Django**: Framework principal
- Twilio**: API para WhatsApp Business
- ReportLab**: Generación de PDFs
- PyPDF2**: Encriptación de PDFs
- Celery**: Tareas asíncronas
- Redis**: Broker de mensajes
- SMTP**: Envío de correos
- Bootstrap**: Interfaz de usuario



Paso 1: Configuración del Entorno y Dependencias



Instalación de dependencias especializadas:

Crear archivo `requirements.txt` con todas las librerías necesarias:

```
# Framework base
Django==4.2.7
djangoRESTframework==3.14.0

# Notificaciones WhatsApp
twilio==8.10.0

# Generación y manipulación de PDFs
reportlab==4.0.7
PyPDF2==3.0.1
pypdf==3.17.1

# Envío de correos
django-ses==3.5.0
premailer==3.10.0

# Tareas asíncronas
celery==5.3.4
redis==5.0.1
django-celery-beat==2.5.0
django-celery-results==2.5.1

# Utilidades adicionales
python-dotenv==1.0.0
Pillow==10.1.0
qrcode==7.4.2
python-barcode==0.15.1

# Desarrollo y testing
django-debug-toolbar==4.2.0
```

Instalación paso a paso:

```
# 1. Crear y activar entorno virtual
python -m venv notification_env
# Windows:
notification_env\Scripts\activate
# macOS/Linux:
source notification_env/bin/activate

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Crear proyecto Django
django-admin startproject notification_hub .

# 4. Crear aplicaciones especializadas
python manage.py startapp clients
python manage.py startapp notifications
python manage.py startapp reports
python manage.py startapp dashboard
```

¿Para qué sirve cada librería?

- twilio**
API oficial para WhatsApp Business y SMS
- reportlab**
Generación profesional de PDFs con gráficos
- PyPDF2**
Manipulación y encriptación de PDFs existentes
- celery**
Procesamiento de tareas en segundo plano
- redis**
Base de datos en memoria para colas de tareas
- django-ses**
Integración con Amazon SES para emails masivos

Paso 2: Configuración de settings.py - Integración Completa

Configuración completa de settings.py:

```
import os
from pathlib import Path
from dotenv import load_dotenv

# Cargar variables de entorno
load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

# Configuración de seguridad
SECRET_KEY = os.getenv('SECRET_KEY', 'django-insecure-change-in-production')
DEBUG = os.getenv('DEBUG', 'True').lower() == 'true'
ALLOWED_HOSTS = os.getenv('ALLOWED_HOSTS', 'localhost,127.0.0.1').split(',')

# Aplicaciones instaladas
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third party apps
    'rest_framework',
    'django_celery_beat',
    'django_celery_results',

    # Local apps
    'clients',
    'notifications',
    'reports',
    'dashboard',
]

# Configuración de base de datos
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Configuración de Celery
CELERY_BROKER_URL = os.getenv('REDIS_URL', 'redis://localhost:6379/0')
CELERY_RESULT_BACKEND = os.getenv('REDIS_URL', 'redis://localhost:6379/0')
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = 'America/Bogota'

# Configuración de Twilio (WhatsApp)
TWILIO_ACCOUNT_SID = os.getenv('TWILIO_ACCOUNT_SID')
TWILIO_AUTH_TOKEN = os.getenv('TWILIO_AUTH_TOKEN')
TWILIO_WHATSAPP_FROM = os.getenv('TWILIO_WHATSAPP_FROM') # ej: whatsapp:+14155238886

# Configuración de Email
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = os.getenv('EMAIL_HOST', 'smtp.gmail.com')
EMAIL_PORT = int(os.getenv('EMAIL_PORT', '587'))
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.getenv('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = os.getenv('EMAIL_HOST_PASSWORD')
DEFAULT_FROM_EMAIL = os.getenv('DEFAULT_FROM_EMAIL', EMAIL_HOST_USER)
```



Archivo .env - Variables de Entorno Seguras

Crear archivo .env en la raíz del proyecto:

```
# Configuración Django
SECRET_KEY=tu-clave-secreta-super-segura-de-50-caracteres-aqui
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# Configuración Redis (para Celery)
REDIS_URL=redis://localhost:6379/0

# Configuración Twilio WhatsApp
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=tu_auth_token_de_twilio_aqui
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886

# Configuración Email SMTP
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=tu_email@gmail.com
EMAIL_HOST_PASSWORD=tu_password_de_aplicacion
DEFAULT_FROM_EMAIL=NotificationHub <tu_email@gmail.com>

# Configuración de archivos
MEDIA_ROOT=/ruta/completa/a/media/
STATIC_ROOT=/ruta/completa/a/static/
```

¿Cómo obtener las credenciales?

Para Twilio WhatsApp:

1. Registrarse en <https://www.twilio.com/>
2. Crear una cuenta de WhatsApp Business
3. Obtener Account SID y Auth Token del dashboard
4. Configurar número de WhatsApp sandbox para pruebas

Para Gmail SMTP:

1. Activar verificación en 2 pasos en tu cuenta Google
2. Generar contraseña de aplicación específica
3. Usar esa contraseña en EMAIL_HOST_PASSWORD

Para Redis:

```
# Instalar Redis en Ubuntu/Debian:
sudo apt update
sudo apt install redis-server

# Instalar Redis en macOS:
brew install redis

# Instalar Redis en Windows:
# Descargar desde https://redis.io/download
```

Verificación de configuración:

```
# Probar conexión Redis
redis-cli ping
# Debe responder: PONG
```

```
# Probar variables de entorno
python manage.py shell
>>> import os
>>> print(os.getenv('TWILIO_ACCOUNT_SID'))
```


Modelos de Datos - Sistema de Clientes y Notificaciones

Crear clients/models.py:

```
from django.db import models
from django.contrib.auth.models import User
from django.core.validators import RegexValidator
import uuid

class Client(models.Model):
    """Modelo para gestionar información de clientes"""

    # Información básica
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    identification_number = models.CharField(
        max_length=20,
        unique=True,
        verbose_name="Número de Identificación",
        help_text="Cédula, pasaporte o documento de identidad"
    )
    first_name = models.CharField(max_length=100, verbose_name="Nombres")
    last_name = models.CharField(max_length=100, verbose_name="Apellidos")
    email = models.EmailField(verbose_name="Correo Electrónico")

    # Información de contacto
    phone_regex = RegexValidator(
        regex=r'^\+?1?\d{9,15}$',
        message="Formato: '+9999999999'. Hasta 15 dígitos."
    )
    phone_number = models.CharField(
        validators=[phone_regex],
        max_length=17,
        verbose_name="Número de Teléfono"
    )
    whatsapp_number = models.CharField(
        validators=[phone_regex],
        max_length=17,
        blank=True,
        verbose_name="Número de WhatsApp",
        help_text="Si es diferente al teléfono principal"
    )

    # Configuración de notificaciones
    accepts_whatsapp = models.BooleanField(
        default=True,
        verbose_name="Acepta notificaciones por WhatsApp"
    )
    accepts_email = models.BooleanField(
        default=True,
        verbose_name="Acepta notificaciones por Email"
    )

    # Información adicional
    address = models.TextField(blank=True, verbose_name="Dirección")
    birth_date = models.DateField(null=True, blank=True, verbose_name="Fecha de Nacimiento")

    # Metadatos
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    is_active = models.BooleanField(default=True, verbose_name="Cliente Activo")

    class Meta:
        verbose_name = "Cliente"
        verbose_name_plural = "Clientes"
        ordering = ['last_name', 'first_name']

    def __str__(self):
        return f"{self.first_name} {self.last_name} - {self.identification_number}"

    @property
    def full_name(self):
        return f"{self.first_name} {self.last_name}"

    @property
    def whatsapp_formatted(self):
        """Retorna número de WhatsApp en formato para Twilio"""
        number = self.whatsapp_number or self.phone_number
        if not number.startswith('+'):
            number = '+57' + number # Código de Colombia por defecto
        return f"whatsapp:{number}"
```



Modelo de Cuentas y Transacciones - Datos Financieros

Para extender la funcionalidad de la plataforma de notificaciones y gestionar datos financieros de los clientes, se introducen dos nuevos modelos en `clients/models.py`: `Account` para representar las diferentes cuentas de un cliente (bancarias, servicios, etc.) y `Transaction` para registrar los movimientos asociados a cada cuenta.

Modelo de Cuenta (Account)

Gestiona información detallada sobre las cuentas de los clientes, incluyendo su tipo, saldo, estado y fechas importantes.

Modelo de Transacción (Transaction)

Registra todos los movimientos financieros (depósitos, retiros, pagos) asociados a una cuenta específica, con detalles como tipo, monto y descripción.

Continuar en `clients/models.py`:

```
class Account(models.Model):
    """Modelo para cuentas de clientes (bancarias, servicios, etc.)"""

    ACCOUNT_TYPES = [
        ('savings', 'Cuenta de Ahorros'),
        ('checking', 'Cuenta Corriente'),
        ('credit', 'Tarjeta de Crédito'),
        ('loan', 'Préstamo'),
        ('service', 'Cuenta de Servicios'),
        ('investment', 'Cuenta de Inversión'),
    ]

    STATUS_CHOICES = [
        ('active', 'Activa'),
        ('inactive', 'Inactiva'),
        ('suspended', 'Suspendida'),
        ('closed', 'Cerrada'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    client = models.ForeignKey(Client, on_delete=models.CASCADE, related_name='accounts')
    account_number = models.CharField(max_length=20, unique=True, verbose_name="Número de Cuenta")
    account_type = models.CharField(max_length=20, choices=ACCOUNT_TYPES, verbose_name="Tipo de Cuenta")
    balance = models.DecimalField(max_digits=15, decimal_places=2, default=0, verbose_name="Saldo")
    credit_limit = models.DecimalField(max_digits=15, decimal_places=2, null=True, blank=True, verbose_name="Límite de Crédito")
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='active', verbose_name="Estado")

    # Fechas importantes
    opened_date = models.DateField(verbose_name="Fecha de Apertura")
    last_statement_date = models.DateField(null=True, blank=True, verbose_name="Última Fecha de Extracto")

    # Metadatos
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Cuenta"
        verbose_name_plural = "Cuentas"
        ordering = ['-created_at']

    def __str__(self):
        return f'{self.account_number} - {self.get_account_type_display()}'

class Transaction(models.Model):
    """Modelo para transacciones de cuentas"""

    TRANSACTION_TYPES = [
        ('deposit', 'Depósito'),
        ('withdrawal', 'Retiro'),
        ('transfer_in', 'Transferencia Recibida'),
        ('transfer_out', 'Transferencia Enviada'),
        ('payment', 'Pago'),
        ('fee', 'Comisión'),
        ('interest', 'Interés'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    account = models.ForeignKey(Account, on_delete=models.CASCADE, related_name='transactions')
    transaction_type = models.CharField(max_length=20, choices=TRANSACTION_TYPES, verbose_name="Tipo de Transacción")
    amount = models.DecimalField(max_digits=15, decimal_places=2, verbose_name="Monto")
    description = models.CharField(max_length=200, verbose_name="Descripción")
    reference_number = models.CharField(max_length=50, unique=True, verbose_name="Número de Referencia")

    # Información adicional
    balance_after = models.DecimalField(max_digits=15, decimal_places=2, verbose_name="Saldo Después")
    external_reference = models.CharField(max_length=100, blank=True, verbose_name="Referencia Externa")

    # Fechas
    transaction_date = models.DateTimeField(verbose_name="Fecha de Transacción")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = "Transacción"
        verbose_name_plural = "Transacciones"
        ordering = ['-transaction_date']

    def __str__(self):
        return f'{self.reference_number} - {self.amount}'
```

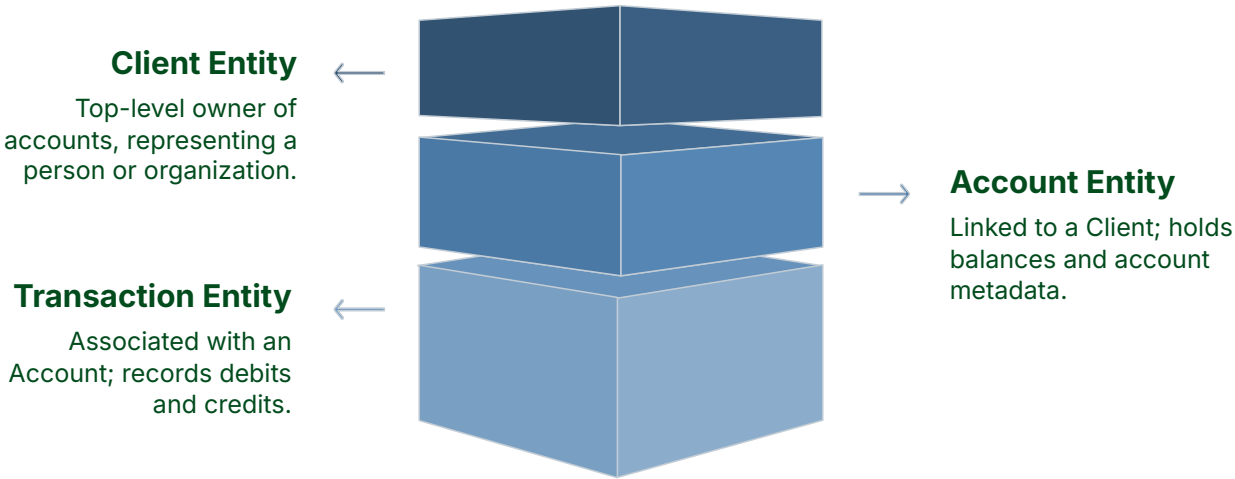
El modelo `Account` permite clasificar las cuentas por su `account_type` y mantener un registro de su balance y status. Los tipos de cuenta definidos son:

- `savings`: Cuenta de Ahorros
- `checking`: Cuenta Corriente
- `credit`: Tarjeta de Crédito
- `loan`: Préstamo
- `service`: Cuenta de Servicios
- `investment`: Cuenta de Inversión

Mientras que el modelo `Transaction` registra cada movimiento con un `transaction_type`, `amount` y una `description`, manteniendo un `balance_after` para la auditoría. Los tipos de transacción incluyen:

- `deposit`: Depósito
- `withdrawal`: Retiro
- `transfer_in`: Transferencia Recibida
- `transfer_out`: Transferencia Enviada
- `payment`: Pago
- `fee`: Comisión
- `interest`: Interés

Estos modelos establecen una relación de uno a muchos: un `Client` puede tener múltiples `Accounts`, y cada `Account` puede tener múltiples `Transactions`.



Modelo de Notificaciones - Registro de Envíos

Para complementar el sistema de gestión de clientes y transacciones, se introduce un módulo de notificaciones. Este módulo permitirá definir plantillas de mensajes y registrar el historial de todas las comunicaciones enviadas a los clientes. El siguiente código define los modelos necesarios para este módulo en notifications/models.py.

Plantillas de Notificación (NotificationTemplate)
Define estructuras reutilizables para diferentes tipos de mensajes, especificando contenido para WhatsApp y correo electrónico, así como configuraciones adicionales.

Registro de Notificaciones (NotificationLog)
Almacena un historial detallado de cada notificación enviada, incluyendo su estado, canal utilizado, contenido final y cualquier error ocurrido durante el proceso.

Crear notifications/models.py:

```
from django.db import models
from django.contrib.auth.models import User
from clients.models import Client
import uuid

class NotificationTemplate(models.Model):
    """Plantillas para diferentes tipos de notificaciones"""

    TEMPLATE_TYPES = [
        ('account_statement', 'Estado de Cuenta'),
        ('transaction_alert', 'Alerta de Transacción'),
        ('payment_reminder', 'Recordatorio de Pago'),
        ('welcome', 'Bienvenida'),
        ('security_alert', 'Alerta de Seguridad'),
        ('promotional', 'Promocional'),
        ('custom', 'Personalizado'),
    ]

    CHANNEL_TYPES = [
        ('whatsapp', 'WhatsApp'),
        ('email', 'Email'),
        ('both', 'Ambos'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=100, verbose_name="Nombre de la Plantilla")
    template_type = models.CharField(max_length=30, choices=TEMPLATE_TYPES, verbose_name="Tipo de Plantilla")
    channel = models.CharField(max_length=20, choices=CHANNEL_TYPES, verbose_name="Canal de Envío")

    # Contenido WhatsApp
    whatsapp_message = models.TextField(
        blank=True,
        verbose_name="Mensaje de WhatsApp",
        help_text="Usar {{variable}} para campos dinámicos"
    )

    # Contenido Email
    email_subject = models.CharField(max_length=200, blank=True, verbose_name="Asunto del Email")
    email_body = models.TextField(
        blank=True,
        verbose_name="Cuerpo del Email",
        help_text="HTML permitido. Usar {{variable}} para campos dinámicos"
    )

    # Configuración
    is_active = models.BooleanField(default=True, verbose_name="Plantilla Activa")
    requires_pdf = models.BooleanField(default=False, verbose_name="Requiere PDF Adjunto")

    # Metadatos
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Plantilla de Notificación"
        verbose_name_plural = "Plantillas de Notificación"
        ordering = ['name']

    def __str__(self):
        return f'{self.name} - {self.get_template_type_display()}'

class NotificationLog(models.Model):
    """Registro de todas las notificaciones enviadas"""

    STATUS_CHOICES = [
        ('pending', 'Pendiente'),
        ('processing', 'Procesando'),
        ('sent', 'Enviado'),
        ('delivered', 'Entregado'),
        ('failed', 'Fallido'),
        ('cancelled', 'Cancelado'),
    ]

    CHANNEL_CHOICES = [
        ('whatsapp', 'WhatsApp'),
        ('email', 'Email'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    client = models.ForeignKey(Client, on_delete=models.CASCADE, related_name='notifications')
    template = models.ForeignKey(NotificationTemplate, on_delete=models.CASCADE)
    channel = models.CharField(max_length=20, choices=CHANNEL_CHOICES, verbose_name="Canal Utilizado")

    # Contenido enviado
    message_content = models.TextField(verbose_name="Contenido del Mensaje")
    subject = models.CharField(max_length=200, blank=True, verbose_name="Asunto (Email)")

    # Estado y seguimiento
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending', verbose_name="Estado")
    external_id = models.CharField(max_length=100, blank=True, verbose_name="ID Externo (Twilio/SMTP)")
    error_message = models.TextField(blank=True, verbose_name="Mensaje de Error")

    # Archivos adjuntos
    pdf_file = models.FileField(upload_to='notifications/pdfs/', blank=True, verbose_name="Archivo PDF")
    pdf_password = models.CharField(max_length=50, blank=True, verbose_name="Contraseña del PDF")

    # Fechas importantes
    scheduled_at = models.DateTimeField(null=True, blank=True, verbose_name="Programado Para")
    sent_at = models.DateTimeField(null=True, blank=True, verbose_name="Enviado En")
    delivered_at = models.DateTimeField(null=True, blank=True, verbose_name="Entregado En")

    # Metadatos
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Registro de Notificación"
        verbose_name_plural = "Registros de Notificaciones"
        ordering = ['-created_at']

    def __str__(self):
        return f'{self.client.full_name} - {self.template.name} - {self.status}'
```

El modelo NotificationTemplate permite la creación de plantillas personalizables para diversos propósitos de comunicación. Define los siguientes tipos de plantilla:

- account_statement: Estado de Cuenta
- transaction_alert: Alerta de Transacción
- payment_reminder: Recordatorio de Pago
- welcome: Bienvenida
- security_alert: Alerta de Seguridad
- promotional: Promocional
- custom: Personalizado

Además, permite especificar el canal de envío:

- whatsapp: WhatsApp
- email: Email
- both: Ambos

Con campos para el contenido del mensaje de WhatsApp y el asunto y cuerpo del correo electrónico, soportando variables dinámicas y HTML para los emails.

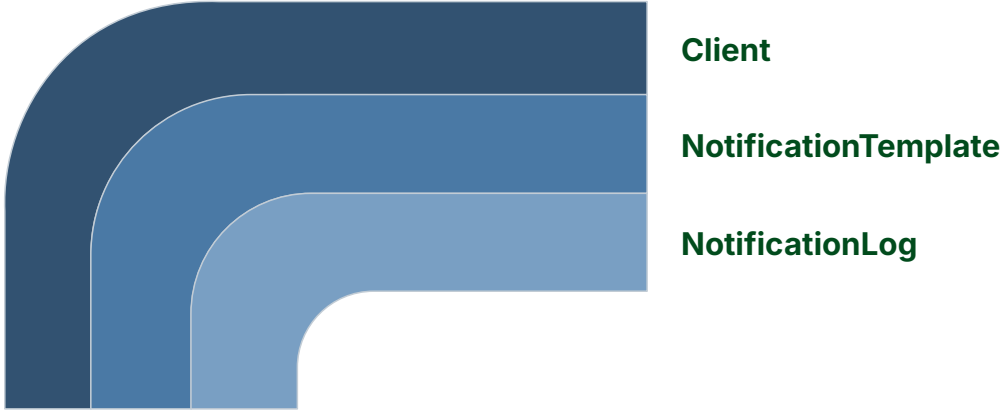
Por otro lado, el modelo NotificationLog centraliza el seguimiento de todas las notificaciones enviadas, proporcionando visibilidad sobre su entrega y posibles fallos. Los estados de notificación contemplados son:

- pending: Pendiente
- processing: Procesando
- sent: Enviado
- delivered: Entregado
- failed: Fallido
- cancelled: Cancelado

Y los canales de registro:

- whatsapp: WhatsApp
- email: Email

Este modelo registra información crucial como el cliente, la plantilla utilizada, el contenido enviado, el canal, el estado de la entrega, mensajes de error, y fechas importantes como cuándo fue programada, enviada y entregada la notificación.



Servicios de WhatsApp - Integración con Twilio

El siguiente código define un servicio de Python para la integración con la API de Twilio, específicamente para el envío de mensajes de WhatsApp. Este servicio permite gestionar la comunicación saliente de WhatsApp, manejar los envíos y actualizar el estado de las notificaciones en el registro. Se ha creado el archivo `notifications/services.py` para encapsular esta lógica.

```
from twilio.rest import Client as TwilioClient
from twilio.base.exceptions import TwilioException
from django.conf import settings
from django.utils import timezone
from .models import NotificationLog
import logging

logger = logging.getLogger(__name__)

class WhatsAppService:
    """Servicio para envío de mensajes por WhatsApp usando Twilio"""

    def __init__(self):
        self.client = TwilioClient(
            settings.TWILIO_ACCOUNT_SID,
            settings.TWILIO_AUTH_TOKEN
        )
        self.from_number = settings.TWILIO_WHATSAPP_FROM

    def send_message(self, to_number, message, notification_log_id=None):
        """
        Envía un mensaje de WhatsApp

        Args:
            to_number (str): Número de destino en formato whatsapp:+1234567890
            message (str): Contenido del mensaje
            notification_log_id (str): ID del registro de notificación

        Returns:
            dict: Resultado del envío con status y detalles
        """
        try:
            # Validar formato del número
            if not to_number.startswith('whatsapp:'):
                to_number = f'whatsapp:{to_number}'

            # Enviar mensaje usando Twilio
            message_instance = self.client.messages.create(
                body=message,
                from_=self.from_number,
                to=to_number
            )

            # Actualizar registro de notificación si se proporciona
            if notification_log_id:
                self._update_notification_log(
                    notification_log_id,
                    status='sent',
                    external_id=message_instance.sid,
                    sent_at=timezone.now()
                )

            logger.info(f'WhatsApp enviado exitosamente. SID: {message_instance.sid}')

            return {
                'success': True,
                'message_sid': message_instance.sid,
                'status': message_instance.status,
                'to': to_number,
                'message': 'Mensaje enviado exitosamente'
            }

        except TwilioException as e:
            error_msg = f'Error de Twilio: {str(e)}'
            logger.error(error_msg)

            # Actualizar registro con error
            if notification_log_id:
                self._update_notification_log(
                    notification_log_id,
                    status='failed',
                    error_message=error_msg
                )

            return {
                'success': False,
                'error': error_msg,
                'error_code': getattr(e, 'code', None)
            }

        except Exception as e:
            error_msg = f'Error inesperado: {str(e)}'
            logger.error(error_msg)

            if notification_log_id:
                self._update_notification_log(
                    notification_log_id,
                    status='failed',
                    error_message=error_msg
                )

            return {
                'success': False,
                'error': error_msg
            }

    def get_message_status(self, message_sid):
        """
        Consulta el estado de un mensaje enviado

        Args:
            message_sid (str): SID del mensaje de Twilio

        Returns:
            dict: Estado actual del mensaje
        """
        try:
            message = self.client.messages(message_sid).fetch()

            return {
                'success': True,
                'sid': message.sid,
                'status': message.status,
                'date_sent': message.date_sent,
                'date_updated': message.date_updated,
                'error_code': message.error_code,
                'error_message': message.error_message
            }

        except TwilioException as e:
            return {
                'success': False,
                'error': f'Error consultando estado: {str(e)}'
            }

    def _update_notification_log(self, log_id, **kwargs):
        """Actualiza un registro de notificación"""
        try:
            log = NotificationLog.objects.get(id=log_id)
            for key, value in kwargs.items():
                setattr(log, key, value)
            log.save()
        except NotificationLog.DoesNotExist:
            logger.error(f'NotificationLog {log_id} no encontrado')
        except Exception as e:
            logger.error(f'Error actualizando NotificationLog: {str(e)}')
```

La clase `WhatsAppService` está diseñada para ser un punto centralizado para todas las operaciones relacionadas con WhatsApp. Utiliza el cliente oficial de Twilio para interactuar con la API, y maneja tanto el envío de mensajes como la consulta de su estado, además de integrar la actualización de los registros de notificación en caso de éxito o fallo.

Inicialización

El constructor `__init__` inicializa el cliente de Twilio con las credenciales de la cuenta (`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`) y el número de WhatsApp de origen (`TWILIO_WHATSAPP_FROM`) configurados en los ajustes de Django.

Envío de Mensajes (`send_message`)

Este método es responsable de enviar un mensaje de WhatsApp a un número específico. Valida el formato del número de destino y utiliza `self.client.messages.create` para realizar el envío. Si se proporciona un `notification_log_id`, actualiza el estado del registro de notificación a 'sent' (enviado) y guarda el SID del mensaje de Twilio y la fecha de envío. Incluye manejo de excepciones para errores de Twilio y otros errores inesperados, actualizando el log a 'failed' en caso de problemas.

Consulta de Estado (`get_message_status`)

Permite consultar el estado de un mensaje enviado previamente utilizando su `message_sid` de Twilio. Devuelve información detallada sobre el estado actual del mensaje, incluyendo la fecha de envío, la última actualización y posibles códigos o mensajes de error.

Actualización de Registros (`update_notification_log`)

Este método auxiliar se encarga de actualizar un registro de notificación existente en la base de datos con nueva información (estado, ID externo, mensaje de error, etc.). Esto asegura que el sistema mantenga un registro preciso del ciclo de vida de cada notificación enviada.

Servicio de Email - Envío con Adjuntos Encriptados

Continuando con la implementación de los servicios de notificación, a continuación se presenta la clase `EmailService`, diseñada para gestionar el envío de correos electrónicos, incluyendo la capacidad de adjuntar archivos PDF y la integración con el sistema de registro de notificaciones.

```
from django.core.mail import EmailMessage
from django.template.loader import render_to_string
from django.utils.html import strip_tags
from django.core.files.base import ContentFile
import smtplib
import logging

logger = logging.getLogger(__name__)

class EmailService:
    """Servicio para envío de correos electrónicos con adjuntos"""

    def send_email_with_attachment(self, to_email, subject, html_content,
                                  pdf_file=None, pdf_filename=None,
                                  notification_log_id=None):
        """
        Envía un correo electrónico con archivo PDF adjunto

        Args:
            to_email (str): Dirección de correo destino
            subject (str): Asunto del correo
            html_content (str): Contenido HTML del correo
            pdf_file (bytes): Contenido del archivo PDF
            pdf_filename (str): Nombre del archivo PDF
            notification_log_id (str): ID del registro de notificación

        Returns:
            dict: Resultado del envío
        """
        try:
            # Crear mensaje de email
            email = EmailMessage(
                subject=subject,
                body=html_content,
                from_email=settings.DEFAULT_FROM_EMAIL,
                to=[to_email],
            )

            # Configurar como HTML
            email.content_subtype = 'html'

            # Adjuntar PDF si se proporciona
            if pdf_file and pdf_filename:
                email.attach(pdf_filename, pdf_file, 'application/pdf')

            # Enviar email
            result = email.send()

            if result == 1: # Éxito
                # Actualizar registro de notificación
                if notification_log_id:
                    self._update_notification_log(
                        notification_log_id,
                        status='sent',
                        sent_at=timezone.now()
                    )

                logger.info(f"Email enviado exitosamente a {to_email}")

            return {
                'success': True,
                'message': 'Email enviado exitosamente',
                'to': to_email
            }
        except smtplib.SMTPException as e:
            error_msg = f"Error SMTP: {str(e)}"
            logger.error(error_msg)

            if notification_log_id:
                self._update_notification_log(
                    notification_log_id,
                    status='failed',
                    error_message=error_msg
                )

            return {
                'success': False,
                'error': error_msg
            }

        except Exception as e:
            error_msg = f"Error inesperado enviando email: {str(e)}"
            logger.error(error_msg)

            if notification_log_id:
                self._update_notification_log(
                    notification_log_id,
                    status='failed',
                    error_message=error_msg
                )

            return {
                'success': False,
                'error': error_msg
            }

    def render_email_template(self, template_name, context):
        """
        Renderiza una plantilla de email con contexto

        Args:
            template_name (str): Nombre de la plantilla
            context (dict): Contexto para la plantilla

        Returns:
            tuple: (html_content, text_content)
        """
        try:
            html_content = render_to_string(template_name, context)
            text_content = strip_tags(html_content)

            return html_content, text_content
        except Exception as e:
            logger.error(f"Error renderizando plantilla {template_name}: {str(e)}")
            return None, None

    def _update_notification_log(self, log_id, **kwargs):
        """Actualiza un registro de notificación"""
        try:
            log = NotificationLog.objects.get(id=log_id)
            for key, value in kwargs.items():
                setattr(log, key, value)
            log.save()
        except NotificationLog.DoesNotExist:
            logger.error(f"NotificationLog {log_id} no encontrado")
        except Exception as e:
            logger.error(f"Error actualizando NotificationLog: {str(e)}")
```

La clase `EmailService` proporciona una interfaz robusta para el envío de correos electrónicos dentro de la aplicación. Gestiona la composición del mensaje, la inclusión de adjuntos y la integración con el sistema de registro de notificaciones para un seguimiento completo de los envíos.

Servicio para Envío de Correos Electrónicos

La clase `EmailService` está diseñada para centralizar y simplificar las operaciones de envío de correos electrónicos, ofreciendo funcionalidades clave para la comunicación con los usuarios de la aplicación.

Envío de Correos con Adjuntos (`send_email_with_attachment`)

Este método permite enviar un correo electrónico con un archivo PDF adjunto a una dirección específica. Construye un objeto `EmailMessage`, configura su contenido como HTML y, si se proporcionan `pdf_file` y `pdf_filename`, adjunta el PDF. Maneja el envío y actualiza el registro de notificación (`notification_log_id`) a 'sent' si es exitoso, o a 'failed' en caso de errores `SMTPException` o cualquier otra excepción inesperada.

Renderizado de Plantillas (`render_email_template`)

Este método es responsable de tomar un nombre de plantilla (`template_name`) y un diccionario de contexto (`context`) para generar el contenido HTML y de texto plano del correo. Utiliza `render_to_string` y `strip_tags` de Django para procesar la plantilla. Si ocurre un error durante el renderizado, registra el error y devuelve `None`.

Actualización de Registros (`_update_notification_log`)

Este método auxiliar se encarga de actualizar un registro de notificación existente en la base de datos con nueva información (estado, ID externo, mensaje de error, etc.). Esto asegura que el sistema mantenga un registro preciso del ciclo de vida de cada notificación enviada. Maneja casos donde el `NotificationLog` no se encuentra o cualquier otro error durante la actualización.



Generación de PDFs con ReportLab - Estados de Cuenta Personalizados

A continuación, se presenta la implementación del servicio de generación de PDFs utilizando la librería ReportLab, enfocándose en la creación de estados de cuenta personalizados. Este código define la clase `PDFReportService`, la cual encapsula la lógica para configurar estilos, obtener datos y construir el documento PDF.

El siguiente bloque de código muestra la estructura y los métodos iniciales de esta clase:

```
from reportlab.lib.pagesizes import letter, A4
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph, Spacer, Image
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_LEFT, TA_RIGHT
from django.conf import settings
from django.utils import timezone
from clients.models import Client, Account, Transaction
import io
import os
from datetime import datetime, timedelta

class PDFReportService:
    """Servicio para generar reportes PDF personalizados"""

    def __init__(self):
        self.styles = getSampleStyleSheet()
        self.setup_custom_styles()

    def setup_custom_styles(self):
        """Configura estilos personalizados para el PDF"""
        # Estilo para el título principal
        self.styles.add(ParagraphStyle(
            name='CustomTitle',
            parent=self.styles['Heading1'],
            fontSize=18,
            spaceAfter=30,
            alignment=TA_CENTER,
            textColor=colors.darkblue
        ))

        # Estilo para subtítulos
        self.styles.add(ParagraphStyle(
            name='CustomSubtitle',
            parent=self.styles['Heading2'],
            fontSize=14,
            spaceAfter=20,
            alignment=TA_LEFT,
            textColor=colors.darkgreen
        ))

        # Estilo para información del cliente
        self.styles.add(ParagraphStyle(
            name='ClientInfo',
            parent=self.styles['Normal'],
            fontSize=10,
            spaceAfter=10,
            alignment=TA_LEFT
        ))

    def generate_account_statement(self, client_id, account_id, start_date=None, end_date=None):
        """
        Genera un estado de cuenta en PDF

        Args:
            client_id (str): ID del cliente
            account_id (str): ID de la cuenta
            start_date (date): Fecha de inicio del período
            end_date (date): Fecha de fin del período

        Returns:
            bytes: Contenido del PDF generado
        """
        try:
            # Obtener datos del cliente y cuenta
            client = Client.objects.get(id=client_id)
            account = Account.objects.get(id=account_id, client=client)

            # Configurar fechas por defecto (último mes)
            if not end_date:
                end_date = timezone.now().date()
            if not start_date:
                start_date = end_date - timedelta(days=30)

            # Obtener transacciones del período
            transactions = Transaction.objects.filter(
                account=account,
                transaction_date__range=[start_date, end_date]
            ).order_by('-transaction_date')

            # Crear buffer para el PDF
            buffer = io.BytesIO()

            # Crear documento PDF
            doc = SimpleDocTemplate(
                buffer,
                pagesize=A4,
                rightMargin=72,
                leftMargin=72,
                topMargin=72,
                bottomMargin=18
            )

            # Construir contenido del PDF
            story = []

            # Encabezado con logo (si existe)
            self._add_header(story)

            # Título del documento
            title = Paragraph("ESTADO DE CUENTA", self.styles['CustomTitle'])
            story.append(title)
            story.append(Spacer(1, 20))

            # Información del cliente
            self._add_client_info(story, client, account)

            # Información del período
            period_info = f"Período: {start_date.strftime('%d/%m/%Y')} - {end_date.strftime('%d/%m/%Y')}"
            story.append(Paragraph(period_info, self.styles['ClientInfo']))
            story.append(Spacer(1, 20))

            # Resumen de la cuenta
            self._add_account_summary(story, account, transactions)

            # Tabla de transacciones
            self._add_transactions_table(story, transactions)

            # Pie de página
            self._add_footer(story)

            # Construir PDF
            doc.build(story)

            # Obtener contenido del buffer
            pdf_content = buffer.getvalue()
            buffer.close()

            return pdf_content

        except Exception as e:
            raise Exception(f"Error generando PDF: {str(e)}")

    def _add_header(self, story):
        """Añade encabezado con logo al PDF"""
        # Aquí puedes añadir un logo si tienes uno
        # logo_path = os.path.join(settings.STATIC_ROOT, 'images', 'logo.png')
        # if os.path.exists(logo_path):
        #     logo = Image(logo_path, width=2*inch, height=1*inch)
        #     story.append(logo)
        #     story.append(Spacer(1, 20))
        pass

    def _add_client_info(self, story, client, account):
        """Añade información del cliente al PDF"""
        client_data = [
            ['Cliente:', client.full_name],
            ['Identificación:', client.identification_number],
            ['Email:', client.email],
            ['Teléfono:', client.phone_number],
            ['Cuenta:', f'{account.account_number} - {account.get_account_type_display()}'],
        ]

        client_table = Table(client_data, colWidths=[2*inch, 4*inch])
        client_table.setStyle(TableStyle([
            ('ALIGN', (0, 0), (-1, -1), 'LEFT'),
            ('FONTNAME', (0, 0), (0, -1), 'Helvetica-Bold'),
            ('FONTSIZE', (0, 0), (-1, -1), 10),
            ('BOTTOMPADDING', (0, 0), (-1, -1), 6),
        ]))

        story.append(client_table)
        story.append(Spacer(1, 20))
```

La clase `PDFReportService` es fundamental para la creación automatizada de documentos PDF, permitiendo la generación de estados de cuenta detallados para clientes y sus respectivas cuentas. Está diseñada para ser flexible, con estilos personalizables y la capacidad de integrar diversos elementos de contenido.

PDFReportService: Servicio de Generación de Reportes PDF Esta clase centraliza la funcionalidad para crear documentos PDF dinámicos. Su objetivo es proporcionar una manera robusta y configurable de generar estados de cuenta y otros reportes, asegurando uniformidad y profesionalismo en la presentación.	Inicialización y Estilos (<code>__init__</code> y <code>setup_custom_styles</code>) Al instanciar <code>PDFReportService</code>, se inicializan los estilos predeterminados de ReportLab y se configuran estilos personalizados como 'CustomTitle', 'CustomSubtitle' y 'ClientInfo'. Estos estilos definen el formato (fuente, tamaño, color, alineación) para diferentes elementos textuales en el PDF, facilitando un diseño coherente.	Generación de Estados de Cuenta (<code>generate_account_statement</code>) Este método es el motor principal para producir un estado de cuenta. Recibe IDs de cliente y cuenta, y rangos de fechas opcionales. Recupera los datos pertinentes del cliente, la cuenta y las transacciones, configura un documento PDF con márgenes definidos y organiza el contenido en una "historia" (story) que incluye encabezado, título, información del cliente y del período, resumen de cuenta y una tabla de transacciones.
Adición de Encabezado (<code>_add_header</code>) Este método auxiliar está reservado para añadir un encabezado personalizado al documento, incluyendo la opción de integrar un logo si se especifica. Aunque actualmente no tiene una implementación activa de logo, proporciona el punto de extensión para futuras personalizaciones visuales en la parte superior del PDF.	Adición de Información del Cliente (<code>_add_client_info</code>) Este método se encarga de formatear y añadir los datos relevantes del cliente y la cuenta al documento PDF. Utiliza una tabla de dos columnas para presentar información como el nombre del cliente, identificación, email, teléfono y los detalles de la cuenta bancaria, aplicando estilos específicos para asegurar una lectura clara.	



Métodos Auxiliares del PDF - Tablas y Resúmenes

Continuando con la exploración de la clase PDFReportService, esta sección profundiza en los métodos auxiliares que son cruciales para la construcción detallada de los documentos PDF, específicamente en la generación de resúmenes de cuenta, tablas de transacciones y el pie de página, así como un método para generar comprobantes de transacción individuales.

```
def _add_account_summary(self, story, account, transactions):
    """Añade resumen de la cuenta al PDF"""
    # Calcular totales
    total_deposits = sum(t.amount for t in transactions if t.transaction_type in ['deposit', 'transfer_in', 'interest'])
    total_withdrawals = sum(t.amount for t in transactions if t.transaction_type in ['withdrawal', 'transfer_out', 'payment', 'fee'])

    summary_data = [
        ['RESUMEN DE CUENTA', ''],
        ['Saldo Actual:', f"${account.balance:,.2f}"],
        ['Total Ingresos:', f"${total_deposits:,.2f}"],
        ['Total Egresos:', f"${total_withdrawals:,.2f}"],
        ['Número de Transacciones:', str(len(transactions))],
    ]

    summary_table = Table(summary_data, colWidths=[3*inch, 2*inch])
    summary_table.setStyle(TableStyle([
        ('ALIGN', (0, 0), (-1, -1), 'LEFT'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, 0), 12),
        ('FONTNAME', (0, 1), (0, -1), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 1), (-1, -1), 10),
        ('BACKGROUND', (0, 0), (-1, 0), colors.lightgrey),
        ('GRID', (0, 0), (-1, -1), 1, colors.black),
        ('VALIGN', (0, 0), (-1, -1), 'MIDDLE'),
    ]))

    story.append(summary_table)
    story.append(Spacer(1, 20))

def _add_transactions_table(self, story, transactions):
    """Añade tabla de transacciones al PDF"""
    if not transactions:
        no_transactions = Paragraph("No hay transacciones en el período seleccionado.", self.styles['Normal'])
        story.append(no_transactions)
        return

    # Encabezado de la tabla
    data = [['Fecha', 'Descripción', 'Tipo', 'Monto', 'Saldo']]

    # Añadir transacciones
    for transaction in transactions:
        date_str = transaction.transaction_date.strftime('%d/%m/%Y %H:%M')
        amount_str = f"${transaction.amount:,.2f}"
        balance_str = f"${transaction.balance_after:,.2f}"

        # Colorear montos según el tipo
        if transaction.transaction_type in ['deposit', 'transfer_in', 'interest']:
            amount_str = f"+{amount_str}"
        elif transaction.transaction_type in ['withdrawal', 'transfer_out', 'payment', 'fee']:
            amount_str = f"-{amount_str}"

        data.append([
            date_str,
            transaction.description[:30] + '...' if len(transaction.description) > 30 else transaction.description,
            transaction.get_transaction_type_display(),
            amount_str,
            balance_str
        ])

    # Crear tabla
    transactions_table = Table(data, colWidths=[1.2*inch, 2.5*inch, 1.3*inch, 1*inch, 1*inch])

    # Aplicar estilos
    table_style = [
        ('BACKGROUND', (0, 0), (-1, 0), colors.darkblue),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, 0), 10),
        ('FONTNAME', (0, 1), (-1, -1), 'Helvetica'),
        ('FONTSIZE', (0, 1), (-1, -1), 8),
        ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
        ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
        ('GRID', (0, 0), (-1, -1), 1, colors.black),
    ]

    # Alternar colores de filas
    for i in range(1, len(data)):
        if i % 2 == 0:
            table_style.append(('BACKGROUND', (0, i), (-1, i), colors.lightgrey))

    transactions_table.setStyle(TableStyle(table_style))

    story.append(Paragraph("DETALLE DE TRANSACCIONES", self.styles['CustomSubTitle']))
    story.append(transactions_table)
    story.append(Spacer(1, 20))

def _add_footer(self, story):
    """Añade pie de página al PDF"""
    footer_text = f"""

    Este documento fue generado automáticamente el {timezone.now().strftime('%d/%m/%Y a las %H:%M')}

    Para consultas, contacte a nuestro servicio al cliente

    CONFIDENCIAL: Este documento contiene información privada del cliente

    """

    footer = Paragraph(footer_text, self.styles['Normal'])
    story.append(Spacer(1, 30))
    story.append(footer)

def generate_transaction_receipt(self, transaction_id):
    """
    Genera un comprobante de transacción individual

    Args:
        transaction_id (str): ID de la transacción

    Returns:
        bytes: Contenido del PDF del comprobante
    """
    try:
        transaction = Transaction.objects.get(id=transaction_id)
        client = transaction.account.client

        buffer = io.BytesIO()
        doc = SimpleDocTemplate(buffer, pagesize=letter)
        story = []

        # Título
        title = Paragraph("COMPROBANTE DE TRANSACCIÓN", self.styles['CustomTitle'])
        story.append(title)
        story.append(Spacer(1, 30))

        # Información de la transacción
        transaction_data = [
            ['Número de Referencia:', transaction.reference_number],
            ['Fecha y Hora:', transaction.transaction_date.strftime('%d/%m/%Y %H:%M:%S')],
            ['Cliente:', client.full_name],
            ['Cuenta:', transaction.account.account_number],
            ['Tipo de Transacción:', transaction.get_transaction_type_display()],
            ['Descripción:', transaction.description],
            ['Monto:', f"${transaction.amount:,.2f}"],
            ['Saldo Después:', f"${transaction.balance_after:,.2f}"],
        ]

        transaction_table = Table(transaction_data, colWidths=[2.5*inch, 3.5*inch])
        transaction_table.setStyle(TableStyle([
            ('ALIGN', (0, 0), (-1, -1), 'LEFT'),
            ('FONTNAME', (0, 0), (0, -1), 'Helvetica-Bold'),
            ('FONTSIZE', (0, 0), (-1, -1), 12),
            ('BOTTOMPADDING', (0, 0), (-1, -1), 8),
            ('GRID', (0, 0), (-1, -1), 1, colors.black),
            ('BACKGROUND', (0, 0), (0, -1), colors.lightgrey),
        ]))

        story.append(transaction_table)
        story.append(Spacer(1, 40))

        # Nota de seguridad
        security_note = Paragraph(
            "¡IMPORTANTE: Conserve este comprobante para sus registros. "
            "En caso de discrepancias, presente este documento como respaldo.",
            self.styles['Normal']
        )
        story.append(security_note)

        doc.build(story)
        pdf_content = buffer.getvalue()
        buffer.close()

        return pdf_content

    except Exception as e:
        raise Exception(f"Error generando comprobante: {str(e)}")
```

Los métodos auxiliares en reports/services.py son componentes fundamentales que facilitan la estructuración y el formateo de los documentos PDF. Cada método está diseñado para encapsular una funcionalidad específica, desde el cálculo y la presentación de un resumen de cuenta hasta la generación detallada de tablas de transacciones y el manejo de los elementos del pie de página.

_add_account_summary: Resumen de la Cuenta

Este método calcula y presenta un resumen de cuenta a cada documento PDF. Suma los ingresos (depósitos, transferencias recibidas, intereses) y egresos (retiros, transferencias enviadas, pagos, tarifas) del período. Luego, organiza esta información vital—saldo actual, totales de ingresos y egresos, y el número de transacciones—en una tabla bien formateada y estilizada, proporcionando una vista rápida del estado financiero de la cuenta.

_add_transactions_table: Tabla de Transacciones

La función principal de este método es listar todas las transacciones apropiado del período seleccionado. Si no hay transacciones, muestra un mensaje apropiado. Prepara los datos de la tabla, incluyendo la fecha, una descripción truncada, el tipo, el monto (con formato de signo +/- según sea ingreso o egreso) y el saldo posterior a la transacción. Aplica estilos de tabla complejos, incluyendo un encabezado distintivo y un sombreado alterno para las filas, mejorando la legibilidad.

_add_footer: Pie de Página Dinámico

Este método es responsable de añadir el pie de página a cada documento PDF. Genera texto que incluye la fecha y hora exactas de la creación del documento, información de contacto para consultas y una nota de confidencialidad. Utiliza etiquetas HTML básicas dentro de un párrafo para formatear el texto, garantizando que los documentos contengan información estándar y relevante al final.

generate_transaction_receipt: Comprobante Individual

Más allá del estado de cuenta general, este método permite generar un comprobante PDF para una transacción individual. Recupera los detalles específicos de una transacción y el cliente asociado. Compone un documento con un título claro, presenta todos los datos relevantes de la transacción (número de referencia, fecha, cliente, cuenta, tipo, descripción, monto, saldo final) en una tabla estilizada y concluye con una nota de seguridad importante para el cliente.



Sistema de Encriptación de PDFs - Seguridad con PyPDF2

Para asegurar la confidencialidad de los documentos generados, se ha implementado un robusto sistema de encriptación de PDFs utilizando la librería PyPDF2. Este sistema permite proteger los informes con contraseñas, garantizando que solo los usuarios autorizados puedan acceder a su contenido. El módulo 'encryption.py' contiene tanto el servicio de encriptación como un generador de contraseñas flexible.

Crear reports/encryption.py:

```
from PyPDF2 import PdfReader, PdfWriter
import io
import logging

logger = logging.getLogger(__name__)

class PDfEncryptionService:
    """Servicio para encriptar PDFs con contraseña"""

    def encrypt_pdf(self, pdf_content, password):
        """
        Encripta un PDF con una contraseña

        Args:
            pdf_content (bytes): Contenido del PDF sin encriptar
            password (str): Contraseña para encriptar el PDF

        Returns:
            bytes: PDF encriptado
        """
        try:
            # Crear buffer de entrada con el PDF original
            input_buffer = io.BytesIO(pdf_content)

            # Leer el PDF
            pdf_reader = PdfReader(input_buffer)

            # Crear escritor para el PDF encriptado
            pdf_writer = PdfWriter()

            # Copiar todas las páginas al escritor
            for page_num in range(len(pdf_reader.pages)):
                page = pdf_reader.pages[page_num]
                pdf_writer.add_page(page)

            # Encriptar con contraseña
            pdf_writer.encrypt(password)

            # Crear buffer de salida
            output_buffer = io.BytesIO()

            # Escribir PDF encriptado
            pdf_writer.write(output_buffer)

            # Obtener contenido encriptado
            encrypted_content = output_buffer.getvalue()

            # Cerrar buffers
            input_buffer.close()
            output_buffer.close()

            logger.info("PDF encriptado exitosamente")
            return encrypted_content

        except Exception as e:
            logger.error(f"Error encriptando PDF: {str(e)}")
            raise Exception(f"Error en encriptación: {str(e)}")

    def decrypt_pdf(self, encrypted_pdf_content, password):
        """
        Desencripta un PDF con contraseña (para verificación)

        Args:
            encrypted_pdf_content (bytes): PDF encriptado
            password (str): Contraseña para desencriptar

        Returns:
            bool: True si la contraseña es correcta
        """
        try:
            input_buffer = io.BytesIO(encrypted_pdf_content)
            pdf_reader = PdfReader(input_buffer)

            # Verificar si el PDF está encriptado
            if pdf_reader.is_encrypted:
                # Intentar desencriptar con la contraseña
                success = pdf_reader.decrypt(password)
                input_buffer.close()
                return success
            else:
                input_buffer.close()
                return True # PDF no encriptado

        except Exception as e:
            logger.error(f"Error verificando contraseña: {str(e)}")
            return False

    def get_pdf_info(self, pdf_content):
        """
        Obtiene información básica del PDF

        Args:
            pdf_content (bytes): Contenido del PDF

        Returns:
            dict: Información del PDF
        """
        try:
            input_buffer = io.BytesIO(pdf_content)
            pdf_reader = PdfReader(input_buffer)

            info = {
                'num_pages': len(pdf_reader.pages),
                'is_encrypted': pdf_reader.is_encrypted,
                'metadata': pdf_reader.metadata if pdf_reader.metadata else {}
            }

            input_buffer.close()
            return info

        except Exception as e:
            logger.error(f"Error obteniendo información del PDF: {str(e)}")
            return {'error': str(e)}

class PasswordGenerator:
    """Generador de contraseñas basado en datos del cliente"""

    @staticmethod
    def generate_from_identification(identification_number):
        """
        Genera contraseña basada en número de identificación

        Args:
            identification_number (str): Número de identificación del cliente

        Returns:
            str: Contraseña generada
        """
        # Usar el número de identificación como contraseña base
        # En producción, podrías añadir más complejidad
        return identification_number.strip()

    @staticmethod
    def generate_complex_password(client):
        """
        Genera contraseña más compleja basada en datos del cliente

        Args:
            client (Client): Objeto cliente

        Returns:
            str: Contraseña compleja generada
        """
        # Combinar diferentes elementos del cliente
        base = client.identification_number

        # Añadir primeras letras del nombre (en mayúsculas)
        name_initials = ".join([name[0].upper() for name in client.first_name.split()])

        # Añadir últimos 4 dígitos del teléfono si existe
        phone_suffix = client.phone_number[-4:] if len(client.phone_number) >= 4 else "0000"

        # Combinar elementos
        complex_password = f"{base}{name_initials}{phone_suffix}"

        return complex_password

    @staticmethod
    def validate_password_strength(password):
        """
        Valida la fortaleza de una contraseña

        Args:
            password (str): Contraseña a validar

        Returns:
            dict: Resultado de la validación
        """
        result = {
            'is_valid': True,
            'score': 0,
            'issues': []
        }

        # Verificar longitud mínima
        if len(password) < 8:
            result['issues'].append("Debe tener al menos 8 caracteres")
            result['is_valid'] = False
            else:
                result['score'] += 1

        # Verificar si contiene números
        if any(char.isdigit() for char in password):
            result['score'] += 1
        else:
            result['issues'].append("Debe contener al menos un número")

        # Verificar si contiene letras
        if any(char.isalpha() for char in password):
            result['score'] += 1
        else:
            result['issues'].append("Debe contener al menos una letra")

        # Verificar si contiene mayúsculas y minúsculas
        if any(char.isupper() for char in password) and any(char.islower() for char in password):
            result['score'] += 1
        else:
            result['issues'].append("Debe contener mayúsculas y minúsculas")

        # Verificar caracteres especiales
        special_chars = "@#%&*()_+=[{}];/.,<>?"
        if any(char in special_chars for char in password):
            result['score'] += 1

        # Determinar nivel de seguridad
        if result['score'] >= 4:
            result['strength'] = 'Fuerte'
        elif result['score'] >= 3:
            result['strength'] = 'Moderada'
        elif result['score'] >= 2:
            result['strength'] = 'Débil'
        else:
            result['strength'] = 'Muy débil'
            result['is_valid'] = False

        return result
```

PDfEncryptionService: Manejo Seguro de Documentos

La clase PDfEncryptionService centraliza las operaciones de seguridad para los documentos PDF, ofreciendo métodos clave para encriptar, desencriptar y verificar la información de los archivos.

encrypt_pdf(self, pdf_content, password) Este método es el núcleo del servicio de seguridad. Recibe el contenido de un PDF (como bytes) y una contraseña, para luego generar una versión encriptada del documento. Utiliza PdfReader para leer el PDF original y PdfWriter para copiar sus páginas y aplicar la encriptación, asegurando que el PDF resultante esté protegido.	decrypt_pdf(self, encrypted_pdf_content, password) Diseñado principalmente para verificación interna o procesos de autenticación, este método intenta desencriptar un PDF. Retorna True si la contraseña proporcionada es correcta y permite el acceso al documento encriptado, o False en caso contrario, sin exponer el contenido del PDF.	get_pdf_info(self, pdf_content) Proporciona metadatos esenciales sobre cualquier PDF, ya sea encriptado o no. Este método es útil para obtener información como el número de páginas, el estado de encriptación y otros metadatos incrustados en el PDF, lo que ayuda en la gestión y auditoría de documentos.
--	--	--

PasswordGenerator: Estrategias de Generación y Validación de Contraseñas

Complementando el servicio de encriptación, la clase PasswordGenerator ofrece métodos estáticos para crear contraseñas a partir de datos del cliente y para evaluar la fortaleza de las mismas, lo que es crucial para implementar políticas de seguridad robustas.

generate_from_identification(identification_number) Este método genera una contraseña simple basada directamente en el número de identificación del cliente. Aunque es conveniente, la documentación sugiere que en un entorno de producción, se debería añadir más complejidad para mejorar la seguridad.	generate_complex_password(client) Para una mayor seguridad, este método crea una contraseña más robusta combinando el número de identificación del cliente con las iniciales en mayúscula de su nombre y los últimos cuatro dígitos de su número de teléfono, resultando en una contraseña más difícil de adivinar.	validate_password_strength(password) Evalúa la fortaleza de una contraseña según varios criterios: longitud mínima, presencia de números, letras (mayúsculas y minúsculas) y caracteres especiales. Devuelve un diccionario que indica si la contraseña es válida, su puntuación, una descripción de su fortaleza (Fuerte, Moderada, Débil, Muy débil) y una lista de los problemas encontrados.
--	---	--



Configuración de Celery - Tareas Asíncronas

Crear notification_hub/celery.py:

```
import os
from celery import Celery
from django.conf import settings

# Configurar Django settings para Celery
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'notification_hub.settings')

# Crear instancia de Celery
app = Celery('notification_hub')

# Configurar Celery usando settings de Django
app.config_from_object('django.conf:settings', namespace='CELERY')

# Autodescubrir tareas en todas las aplicaciones Django
app.autodiscover_tasks()

# Configuración adicional de Celery
app.conf.update(
    # Configuración de zona horaria
    timezone='America/Bogota',
    enable_utc=True,

    # Configuración de serialización
    task_serializer='json',
    accept_content=['json'],
    result_serializer='json',

    # Configuración de resultados
    result_expires=3600, # 1 hora

    # Configuración de routing
    task_routes={
        'notifications.tasks.send_whatsapp_notification': {'queue': 'whatsapp'},
        'notifications.tasks.send_email_notification': {'queue': 'email'},
        'notifications.tasks.generate_and_send_report': {'queue': 'reports'},
    },

    # Configuración de workers
    worker_prefetch_multiplier=1,
    task_acks_late=True,

    # Configuración de retry
    task_default_retry_delay=60, # 1 minuto
    task_max_retries=3,
)

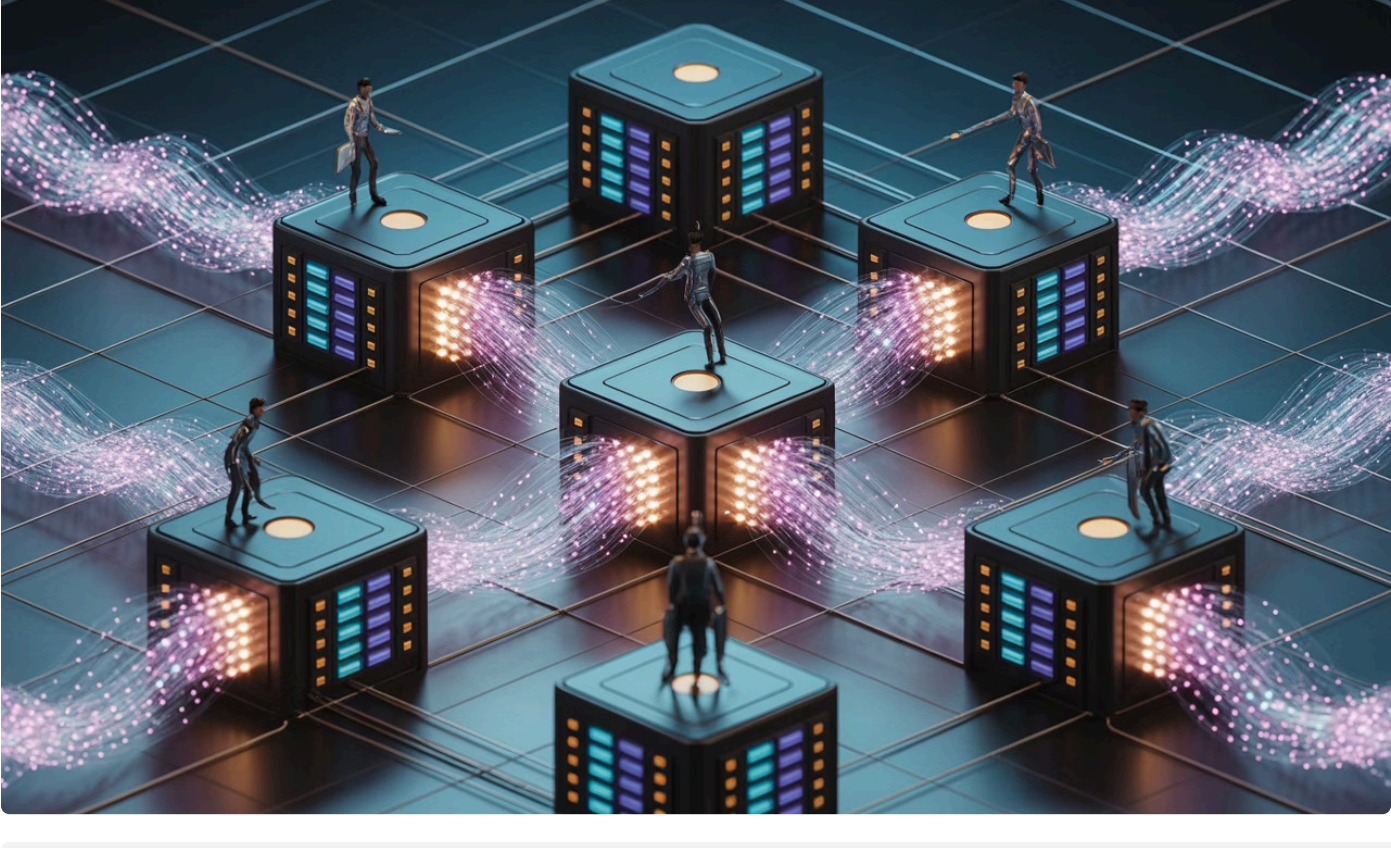
@app.task(bind=True)
def debug_task(self):
    """Tarea de debug para verificar configuración"""
    print(f'Request: {self.request!r}')
```

Modificar notification_hub/__init__.py:

```
# Importar Celery para que Django lo reconozca
from .celery import app as celery_app

__all__ = ('celery_app',)
```

Crear notifications/tasks.py:



```
from celery import shared_task
from django.utils import timezone
from django.core.files.base import ContentFile
from .models import NotificationLog, NotificationTemplate
from .services import WhatsAppService, EmailService
from reports.services import PDFReportService
from reports.encryption import PDFEncryptionService, PasswordGenerator
from clients.models import Client, Account
import logging

logger = logging.getLogger(__name__)

@shared_task(bind=True, max_retries=3)
def send_whatsapp_notification(self, notification_log_id):
    """
    Tarea asíncrona para enviar notificación por WhatsApp

    Args:
        notification_log_id (str): ID del registro de notificación

    Returns:
        dict: Resultado del envío
    """
    try:
        # Obtener registro de notificación
        notification_log = NotificationLog.objects.get(id=notification_log_id)

        # Actualizar estado a procesando
        notification_log.status = 'processing'
        notification_log.save()

        # Verificar que el cliente acepta WhatsApp
        if not notification_log.client.accepts_whatsapp:
            notification_log.status = 'cancelled'
            notification_log.error_message = 'Cliente no acepta notificaciones por WhatsApp'
            notification_log.save()
            return {'success': False, 'error': 'Cliente no acepta WhatsApp'}

        # Inicializar servicio de WhatsApp
        whatsapp_service = WhatsAppService()

        # Enviar mensaje
        result = whatsapp_service.send_message(
            to_number=notification_log.client.whatsapp_formatted,
            message=notification_log.message_content,
            notification_log_id=notification_log_id
        )

        if result['success']:
            logger.info(f"WhatsApp enviado exitosamente: {notification_log_id}")
            return {'success': True, 'message_sid': result.get('message_sid')}
        else:
            # Reintentar si es un error temporal
            if 'temporary' in result.get('error', '').lower():
                raise self.retry(countdown=60, exc=Exception(result['error']))

            return {'success': False, 'error': result['error']}

    except NotificationLog.DoesNotExist:
        logger.error(f"NotificationLog {notification_log_id} no encontrado")
        return {'success': False, 'error': 'Registro de notificación no encontrado'}

    except Exception as e:
        logger.error(f"Error enviando WhatsApp {notification_log_id}: {str(e)}")

        # Actualizar registro con error
        try:
            notification_log = NotificationLog.objects.get(id=notification_log_id)
            notification_log.status = 'failed'
            notification_log.error_message = str(e)
            notification_log.save()
        except:
            pass

        # Reintentar si no se han agotado los intentos
        if self.request.retries < self.max_retries:
            raise self.retry(countdown=60 * (self.request.retries + 1), exc=e)

        return {'success': False, 'error': str(e)}

@shared_task(bind=True, max_retries=3)
def send_email_notification(self, notification_log_id):
    """
    Tarea asincrona para enviar notificación por email

    Args:
        notification_log_id (str): ID del registro de notificación

    Returns:
        dict: Resultado del envío
    """
    try:
        # Actualizar estado
        notification_log.status = 'processing'
        notification_log.save()

        # Verificar que el cliente acepta emails
        if not notification_log.client.accepts_email:
            notification_log.status = 'cancelled'
            notification_log.error_message = 'Cliente no acepta notificaciones por email'
            notification_log.save()
            return {'success': False, 'error': 'Cliente no acepta emails'}

        # Inicializar servicio de email
        email_service = EmailService()

        # Preparar contenido del email
        pdf_content = None
        pdf_filename = None

        if notification_log.pdf_file:
            pdf_content = notification_log.pdf_file.read()
            pdf_filename = f'documento_{notification_log.client.identification_number}.pdf'

        # Enviar email
        result = email_service.send_email_with_attachment(
            to_email=notification_log.client.email,
            subject=notification_log.subject,
            html_content=notification_log.message_content,
            pdf_file=pdf_content,
            pdf_filename=pdf_filename,
            notification_log_id=notification_log_id
        )

        if result['success']:
            logger.info(f"Email enviado exitosamente: {notification_log_id}")
            return {'success': True}
        else:
            return {'success': False, 'error': result['error']}

    except Exception as e:
        logger.error(f"Error enviando email {notification_log_id}: {str(e)}")

        # Actualizar registro con error
        try:
            notification_log = NotificationLog.objects.get(id=notification_log_id)
            notification_log.status = 'failed'
            notification_log.error_message = str(e)
            notification_log.save()
        except:
            pass

        # Reintentar
        if self.request.retries < self.max_retries:
            raise self.retry(countdown=60 * (self.request.retries + 1), exc=e)

        return {'success': False, 'error': str(e)}
```


Tarea Completa de Generación y Envío de Reportes

Esta sección detalla la implementación de la tarea principal encargada de la generación, encriptación y envío de reportes, junto con funciones auxiliares y tareas de mantenimiento del sistema de notificaciones.

- **1. Obtener Componentes**
Recupera el cliente, cuenta y plantilla de notificación.
- **2. Generar PDF**
Crea el estado de cuenta en formato PDF.
- **3. Encriptar PDF**
Aplica encriptación con contraseña al documento.
- **4. Preparar Contexto**
Define los datos dinámicos para las plantillas de mensaje.
- **5. Enviar por Canales**
Distribuye el reporte a través de los canales configurados (WhatsApp, Email).
- **6. Verificar Éxito**
Confirma si al menos un envío fue exitoso.

La función generate_and_send_report es una tarea asíncrona de Celery que orquesta todo el proceso. Se encarga de coordinar la generación de documentos, su seguridad mediante encriptación y la distribución multicanal, garantizando reintentos en caso de fallos.

****Continuar en notifications/tasks.py - Tarea principal:****

```
@shared_task(bind=True, max_retries=2)
def generate_and_send_report(self, client_id, account_id, template_id,
                             start_date=None, end_date=None, channels=['email']):
    """
    Tarea completa para generar PDF, encriptarlo y enviarlo por múltiples canales

    Args:
        client_id (str): ID del cliente
        account_id (str): ID de la cuenta
        template_id (str): ID de la plantilla de notificación
        start_date (str): Fecha de inicio (formato YYYY-MM-DD)
        end_date (str): Fecha de fin (formato YYYY-MM-DD)
        channels (list): Canales de envío ['whatsapp', 'email']

    Returns:
        dict: Resultado del proceso completo
    """
    try:
        # Obtener objetos necesarios
        client = Client.objects.get(id=client_id)
        account = Account.objects.get(id=account_id, client=client)
        template = NotificationTemplate.objects.get(id=template_id)

        logger.info(f"Iniciando generación de reporte para cliente {client.full_name}")

        # Convertir fechas si se proporcionan
        from datetime import datetime
        if start_date:
            start_date = datetime.strptime(start_date, '%Y-%m-%d').date()
        if end_date:
            end_date = datetime.strptime(end_date, '%Y-%m-%d').date()

        # 1. Generar PDF del estado de cuenta
        pdf_service = PDFReportService()
        pdf_content = pdf_service.generate_account_statement(
            client_id=client_id,
            account_id=account_id,
            start_date=start_date,
            end_date=end_date
        )

        # 2. Encriptar PDF con número de identificación
        encryption_service = PDPEncryptionService()
        password = PasswordGenerator.generate_from_identification(client.identification_number)
        encrypted_pdf = encryption_service.encrypt_pdf(pdf_content, password)

        # 3. Preparar contexto para plantillas
        context = {
            'client_name': client.full_name,
            'account_number': account.account_number,
            'account_type': account.get_account_type_display(),
            'balance': account.balance,
            'period_start': start_date.strftime('%d/%m/%Y') if start_date else 'N/A',
            'period_end': end_date.strftime('%d/%m/%Y') if end_date else 'N/A',
            'password': password,
            'generation_date': timezone.now().strftime('%d/%m/%Y %H:%M'),
        }

        results = {'success': True, 'channels': {}}

        # 4. Enviar por cada canal solicitado
        for channel in channels:
            try:
                if channel == 'whatsapp' and template.whatsapp_message:
                    result = await self._send_whatsapp_with_pdf(
                        client, template, context, encrypted_pdf
                    )
                    results['channels']['whatsapp'] = result

                elif channel == 'email' and template.email_body:
                    result = await self._send_email_with_pdf(
                        client, template, context, encrypted_pdf
                    )
                    results['channels']['email'] = result

            except Exception as e:
                logger.error(f"Error enviando por {channel}: {str(e)}")
                results['channels'][channel] = {'success': False, 'error': str(e)}

        # 5. Verificar si al menos un canal fue exitoso
        successful_channels = [ch for ch, result in results['channels'].items()
                               if result.get('success', False)]

        if not successful_channels:
            results['success'] = False
            results['error'] = 'Falló el envío por todos los canales'

        logger.info(f"Reporte completado para {client.full_name}. Canales exitosos: {successful_channels}")
        return results

    except Exception as e:
        logger.error(f"Error en generate_and_send_report: {str(e)}")

        # Reintentar si no se han agotado los intentos
        if self.request.retries < self.max_retries:
            raise self.retry(countdown=300, exc=e) # 5 minutos

        return {'success': False, 'error': str(e)}
```

Funciones Auxiliares para el Envío

Para modularizar el proceso de envío y permitir la expansión a nuevos canales, se definen funciones auxiliares que manejan la lógica específica para WhatsApp y Email, y una función para el renderizado de plantillas.

La función _send_whatsapp_with_pdf prepara y delega el envío de notificaciones de WhatsApp a una tarea asíncrona, registrando la intención de envío en la base de datos. Es importante notar que el PDF encriptado no se adjunta directamente por WhatsApp, solo el mensaje.

```
async def _send_whatsapp_with_pdf(self, client, template, context, encrypted_pdf):
    """Envía notificación por WhatsApp (sin PDF adjunto)"""
    try:
        # Renderizar mensaje de WhatsApp
        message = self._render_template_content(template.whatsapp_message, context)

        # Crear registro de notificación
        notification_log = NotificationLog.objects.create(
            client=client,
            template=template,
            channel='whatsapp',
            message_content=message,
            status='pending'
        )

        # Enviar tarea asíncrona
        task_result = send_whatsapp_notification.delay(str(notification_log.id))

    return {
        'success': True,
        'notification_log_id': str(notification_log.id),
        'task_id': task_result.id
    }

    except Exception as e:
        return {'success': False, 'error': str(e)}
```

Por otro lado, _send_email_with_pdf se encarga de adjuntar el PDF encriptado al correo electrónico, generando un archivo temporal para su almacenamiento y guardando la contraseña del PDF en el registro de notificación antes de delegar el envío a su tarea asíncrona correspondiente.

```
async def _send_email_with_pdf(self, client, template, context, encrypted_pdf):
    """Envía email con PDF encriptado adjunto"""
    try:
        # Renderizar contenido del email
        subject = self._render_template_content(template.email_subject, context)
        body = self._render_template_content(template.email_body, context)

        # Crear archivo temporal para el PDF
        from django.core.files.base import ContentFile
        pdf_file = ContentFile(encrypted_pdf)
        pdf_filename = f'estado_cuenta_{client.identification_number}.pdf'

        # Crear registro de notificación
        notification_log = NotificationLog.objects.create(
            client=client,
            template=template,
            channel='email',
            message_content=body,
            subject=subject,
            status='pending',
            pdf_password=context['password']
        )

        # Guardar PDF en el registro
        notification_log.pdf_file.save(pdf_filename, pdf_file)

        # Enviar tarea asíncrona
        task_result = send_email_notification.delay(str(notification_log.id))

    return {
        'success': True,
        'notification_log_id': str(notification_log.id),
        'task_id': task_result.id
    }

    except Exception as e:
        return {'success': False, 'error': str(e)}
```

La función _render_template_content es utilizada para procesar el contenido de las plantillas de mensajes (tanto para WhatsApp como para Email) usando el motor de plantillas de Django, insertando dinámicamente los datos proporcionados en el contexto. Esto permite una gran flexibilidad en la personalización de los mensajes.

```
def _render_template_content(self, template_content, context):
    """Renderiza contenido de plantilla con contexto"""
    from django.template import Template, Context

    try:
        template = Template(template_content)
        rendered_content = template.render(Context(context))
        return rendered_content
    except Exception as e:
        logger.error(f"Error renderizando plantilla: {str(e)}")
        return template_content # Devolver contenido original si falla
```

Tareas de Mantenimiento y Actualización

Para asegurar la eficiencia del sistema y la coherencia de los datos, se implementan tareas programadas que se encargan de la limpieza de registros antiguos y la actualización de los estados de envío de las notificaciones.

**Limpieza de Notificaciones Antiguas**

La tarea cleanup_old_notifications elimina registros de notificaciones que superan un período de 30 días, contribuyendo a mantener la base de datos optimizada y relevante.

```
@shared_task
def cleanup_old_notifications():
    """Tarea de limpieza para eliminar notificaciones antiguas"""
    from datetime import timedelta

    try:
        # Eliminar notificaciones de más de 30 días
        cutoff_date = timezone.now() - timedelta(days=30)

        old_notifications = NotificationLog.objects.filter(
            created_at__lt=cutoff_date,
            status__in=['sent', 'delivered', 'failed']
        )

        count = old_notifications.count()
        old_notifications.delete()

        logger.info(f"Eliminadas {count} notificaciones antiguas")
        return {'success': True, 'deleted_count': count}

    except Exception as e:
        logger.error(f"Error en limpieza de notificaciones: {str(e)}")
        return {'success': False, 'error': str(e)}
```

**Actualización de Estados de Notificación**

La tarea update_notification_statuses consulta APIs externas (como Twilio para WhatsApp) para sincronizar los estados de entrega de las notificaciones que fueron marcadas como 'enviadas', actualizando su estado a 'entregada' o 'fallida' según corresponda.

```
@shared_task
def update_notification_statuses():
    """Actualiza estados de notificaciones usando APIs externas"""

    try:
        # Obtener notificaciones enviadas pero no entregadas
        pending_notifications = NotificationLog.objects.filter(
            status='sent',
            channel='whatsapp',
            external_id__isnull=False
        )

        whatsapp_service = WhatsAppService()
        updated_count = 0

        for notification in pending_notifications:
            try:
                # Consultar estado en Twilio
                status_info = whatsapp_service.get_message_status(notification.external_id)

                if status_info['success']:
                    if status_info['status'] == 'delivered':
                        notification.delivered_at = timezone.now()
                        notification.save()
                        updated_count += 1
                    elif status_info['status'] == 'failed':
                        notification.status = 'failed'
                        notification.error_message = status_info.get('error_message', 'Error desconocido')
                        notification.save()
                        updated_count += 1

            except Exception as e:
                logger.error(f"Error actualizando notificación {notification.id}: {str(e)}")
                continue

        logger.info(f"Actualizados {updated_count} estados de notificación")
        return {'success': True, 'updated_count': updated_count}

    except Exception as e:
        logger.error(f"Error actualizando estados: {str(e)}")
        return {'success': False, 'error': str(e)}
```